MultiCharts .NET Engines — User Manual (v3)

For _CustomSignal strategies on dxFeed @ES (Portfolio Trader, IBKR execution)

Source: origin/main @ 629da0d (merged PRs #18–#29)

MultiCharts .NET Engines — User Manual (v3)

Source: `origin/main` at commit `629da0d`, which includes merged PRs #18–#29 (EV-4.6b is PRs #28/#29). Member names and signatures were checked against that tree by script (see `engines-manual-v2-verification.md`). Anything I could not confirm from the code is marked **VERIFY**. No section of this manual reports or implies trading results.

Contents. §1 Quick start · §2 Gate 0 · §3 Ten worked examples (easy → hard) · §4 Engines and helpers · Appendices A–D (trace levels, files, tests, glossary) · E (full public signatures by file and class) · F (VERIFY items and discrepancies) · G (EV-4.6b event coverage).

---

1 — Quick start

1.1 Architecture

```
                        MultiCharts .NET (Chart / Portfolio Trader)
                                       │  CalcBar() per tick / bar close
                                       ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ PortfolioSignalObject                                                        │
│ └─ _CustomSignal  (TraceLevel [Input], journal, sizing, time helpers, M-PORT) │
│      │  StartCalc → startEngines()  : create ONLY the engines you enabled    │
│      │  CalcBar   → stepEngines()   : step them on CLOSED bars               │
│      │                                                                       │
│      │   RollDetector → VpHelpers(rings/FVG) → PriceAction → ExtraPA → Tick →  │
│      │   Xmkt → Daily → Regime → Event → Tod → Avwap                         │
│      │   then LevelStudy → portfolioBar (pm.* keys) → journalScan → CS-8 events│
│      │                                                                       │
│      ├─ YourStrategy : _CustomSignal        (e.g. _Test6, your signals)     │
│      └─ _VolumeProfileBase : _CustomSignal  (VP, VWAP slots, confluence, P3) │
│           └─ _VolumeProfileFuturesBase       (CME session clock, IB/RTH stats,  │
│                                               order flow, CME holiday hooks)  │
└──────────────────────────────────────────────────────────────────────────────┘
   Engines (internal sealed classes, MC-free, unit-tested in tests/*):
   _DailyContext  _RegimeEngine(+_Gmm,_Rng)  _EventCalendar  _AvwapManager
   _PriceActionEngine  _TodNormaliser  _XmktLeg  _TickStream  _FvgDetector
   _BarRing  _LevelStudy  _MtfMath  _CmeSessionClock  _CmeHolidays/_NyseHolidays
   Helpers: RegimeGate/A263Core/AtrTercileTracker (_RegimeGate), _PortMM (+ _PortfolioRiskManager)
```

Some rules hold for every engine:

1. **Lazy.** An engine is created in `StartCalc` only if you enabled it through its **protected config** (never an `[Input]`). If you didn't, its accessor returns `null` (or a neutral state) and logs a one-time Warning.
2. **Closed bars only.** Engines commit on `Bars.Status == EBarState.Close`. An intrabar tick never changes their state.
3. **As-of discipline.** Daily values describe completed prior sessions. `RegimeState.AsOf` is the day whose close produced the value, and that value is usable from the **next** session. AVWAP anchors have a `UsableFrom` time. HTF reads use `HtfClosedIndex`/`htfMapped`.
4. **Journaling.** Engines write facts through `engineSink(ETraceLevel.Journal)`, which is `null` when filtered out, so they format no strings. The base never blocks orders on your behalf. For example, `EventBlackout` is only exposed.
5. **Same assembly.** The engine types and accessors are `internal`, which works because MC compiles every study into one assembly. Use them as locals (`_DailyContext d = DailyContext;`). Don't expose them in public or protected signatures of a public class.

1.2 Minimal `_CustomSignal` skeleton

```
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _MySignal : _CustomSignal
    {
        private const string EntryName = "MyLE", ExitName = "MyLX";
        private IOrderPriced m_le;
        private IOrderMarket m_lx;

        public _MySignal(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();                      // sets TraceLevel = None, NYSE holiday function
```

```
            m_le = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, EntryName, EOrderAction.Bu
y));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, ExitName,
EOrderAction.Sell));
            // engine configs may be set here (pattern A) ...
        }

        protected override void StartCalc()
        {
            // ... or here, BEFORE base.StartCalc() (pattern B) — engines are created inside base.StartCalc()
            base.StartCalc();
        }

        protected override void CalcBar()
        {
            base.CalcBar();                      // steps all enabled engines first
            if (Bars.Status != EBarState.Close) return;
            // read engines (values as of this closed bar), decide, Send() orders for the next bar
        }
    }
}
```

`TraceLevel` is the only `[Input]` that `_CustomSignal` defines (`[Input] public virtual ETraceLevel TraceLevel`). Set it to **Journal** in Format Signals. **VERIFY:** `_CustomSignal.Create()` sets `TraceLevel = ETraceLevel.None`, so a code default must be assigned after `base.Create()`. Whether the value saved in the dialog then overrides it is MC behaviour I have not confirmed in this repo.

1.3 Enabling an engine lazily and setting its config

There are two real patterns in the repo:

```
// Pattern A — in Create() (as _EventCalendarTest does)
protected override void Create()
{
    base.Create();
    EventConfig.enabled = true;
    EventConfig.minTier = EEventTier.T3;
    RollDetection = ERollDetection.Series;
}

// Pattern B — in StartCalc() before base.StartCalc() (as _Test6 / _MtfProbe do)
protected override void StartCalc()
{
    RegimeGate.ApplyArm(RegimeGateConfig, RegimeConfig);   // must precede base.StartCalc
    NeedsDaily = true;
    DailyConfig.atrLength = 20;
    HtfStreams = new[] { 2 };
    base.StartCalc();
}
```

Engine	Switch (protected, not an Input)	Config object	Accessor
M-DAILY	`NeedsDaily = true`	`DailyConfig`	`DailyContext`, `DailyGapPct`
G-R Regime	`RegimeConfig.enabled = true` (forces Daily + HV{20,63})	`RegimeConfig`	`Regime` (RegimeState copy)
M-EVENT	`EventConfig.enabled = true`	`EventConfig`	`EventCalendar`, `EventBlackout`, `EventFlattenNow`
M-AVWAP	`AvwapEvents = EAvwapAnchor.…` (≠ None)	`AvwapLimits`	`AvwapManager`
PRICEACTION	`PriceActionConfig.enabled = true` / `ExtraPriceAction.Add(...)`	`PriceActionConfig`	`PriceActionEngine`, `PriceActionFor(ds, role)`
M-TOD	`TodMetrics = ETodMetric.…`	`TodSessions`, `TodMinSessions`, `TodBucketMinutes`	`TodNormaliser`

Engine	Switch (protected, not an Input)	Config object	Accessor
M-XMKT	`XmktLegs.Add(new XmktLegConfig{…})`	`XmktLegConfig`	`XmktLeg(role)`, `XmktLeaderSwing(role)`
M-TICK	`TickConfig.enabled = true`	`TickConfig`	`TickStats`
M-MTF	`HtfStreams = new[]{2}`	—	`HtfClosedIndex`, `HtfClosed`, `HtfClosedBar`, `htfMapped`, `HtfForming`
Rolls	`RollDetection = ERollDetection.Series`	`RollGapTicks`, `RollDates`	`rollOn(ds)`, `RollBars`, `IsRollWindow(day)`
FVG	`Fvgs.enabled = true`	`Fvgs` (FvgConfig)	`FvgDetector`, `fvgLevel(f, edge)`
Bar rings	`requireRing(ds, bars, consumer)` (after base.StartCalc)	`BarRings`	`Ring(ds)`, `ringBarOf(ds, ago)`
A2-33 level study	`LevelStudyConfig.enabled = true`	`LevelStudyConfig`	`levelStudyAdd(...)`
M-PORT	automatic in Portfolio Trader (`PortfolioPublish`)	virtual `Portfolio*` props	`portfolioArm`, `portfolioStop`

Dependencies are enabled for you. Regime forces `NeedsDaily`. AVWAP `Swing`/`Breakout` anchors force a PriceAction instance. AVWAP `VolumeClimax`, or a P2 placebo on Data1, forces TOD `Volume`. `XmktLegConfig.leaderSwing` creates a PA on the leg.

1.4 Reading outputs without lookahead

```
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;            // 1. closed bars only

    _DailyContext d = DailyContext;                        // 2. null if NeedsDaily == false
    DateTime today = tradingDayKey(Bars.Time[0]);
    bool atrOk = d != null && d.Valid && d.PrevDate < today; // 3. t-1: completed PRIOR session
    double atr = atrOk ? d.Atr20d : double.NaN;

    RegimeState r = Regime;                                // 4. usable iff AsOf < today
    bool regimeOk = r.Valid && !r.UsingFallback && r.AsOf < today;

    int i = HtfClosedIndex(2);                             // 5. HTF: last CLOSED bar of Data2
    double htfClose = BarsOfData(2).Close[i];
}
```

Orders sent on bar t's close execute on bar t+1 (`MarketNextBar`, or a `Stop`/`Limit` working from the next bar). Every value above is known at t's close, so there is no lookahead. Never read `BarsOfData(n).Close[0]` of a forming HTF bar. Use `HtfForming(ds)` only when you mean it (it is journaled once).

1.5 Journaling

- Set `TraceLevel = Journal`. That enables Error/Warning/Info, the engines' Journal facts, the per-trade CSV `MCJournal_<runId>.csv`, and CS-8 signal events `MCEvents_<runId>.csv`.
- Guard expensive strings with `if (journalOn()) trace(ETraceLevel.Journal, $"...")`, or use `traceOn(level)`.
- Call `journalArm(entryName, plannedEntry, initialStop, target, context)` next to `Send()` so the trade row carries R and context.
- Call `logSignal(tag, dir, price, stop, context, taken)` to record taken and untaken signals. The base tracks forward MFE/MAE in R at `SignalHorizons` (default {5,15,30,60} bars).

```
if (journalOn())
{
    var ctx = new Dictionary<string, string> { { "atr", atr.ToString("0.00") }, { "regime", r.Label.ToString(
) } };
    journalArm(EntryName, stopPrice, initialStop, double.NaN, ctx);
    logSignal("ORB", +1, stopPrice, initialStop, ctx, true);
}
m_le.Send(stopPrice, qty);
```

1.6 Important for @ES on a plain `_CustomSignal`: the CME session adapter

On a plain `_CustomSignal`, `tradingDayKey()` defaults to the **calendar date** (`ETradingDayRoll.Calendar`). `sessionClock()` defaults to the bar's own time of day, and `isRthBar()` returns **true** for every bar. With a 24-hour Globex @ES chart, that means:

- the Sunday/evening 18:00 ET session is split at midnight;
- M-DAILY aggregates calendar days, not Globex days, and its RTH-close basis becomes "last bar of the day";
- `NoEntryBefore`/`NoEntryAfter`/`FlatBy` are compared in chart time.

`_VolumeProfileFuturesBase` fixes all of this by overriding these virtuals with a `_CmeSessionClock`. A plain `_CustomSignal` can do the same, because every member used here is a real protected virtual or public member:

```
private _CmeSessionClock m_clock;
private _CmeSessionClock Clock
{
    get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBarTimeZone(Bars.Info)); return
m_clock; }
}
protected override void StartCalc() { m_clock = null; /* engine configs … */ base.StartCalc(); }

protected override DateTime tradingDayKey(DateTime barTime) { return Clock.GlobexDate(barTime); }
protected override TimeSpan sessionClock(DateTime barTime) { return Clock.ToEastern(barTime).TimeOfDay; }
protected override bool isRthBar(DateTime barTime) { return
Clock.IsRthValueBarEt(Clock.ToEastern(barTime)); }
protected override int todMinuteOfSession(DateTime barTime)
{
    int m = (int)Clock.ToEastern(barTime).TimeOfDay.TotalMinutes - (int)Clock.GlobexOpen.TotalMinutes;
    return m < 0 ? m + 1440 : m;
}
protected override bool todHalfDay(DateTime d)
{
    return Clock.IsNoRthDay(d) || Clock.HolidayHaltEt(d) != null || Clock.RthEarlyCloseEt(d) != null;
}
```

These bodies mirror `_VolumeProfileFuturesBase` lines 356–512. **VERIFY:** `_Test6` does not override `tradingDayKey`. With a Globex session template, its M-DAILY days are therefore calendar days. Decide whether that is intended (see the pitfalls in 1.7 and Appendix F). Also, on a plain `_CustomSignal`, `PriceActionConfig` keeps the **Library** preset. Call `PriceActionConfig.ApplyPreset(EPriceActionPreset.Futures)` in `Create()` to match the futures bases (LeftInclusive ties, `volumeIncludesCurrent = false`).

1.7 Pitfalls checklist

- [ ] **Lookahead.**
- Read engines only after `base.CalcBar()` and on `EBarState.Close`.
- Daily values require `PrevDate < today`; Regime requires `AsOf < today`; AVWAP requires `UsableFrom <= Bars.Time[0]`; HTF requires `HtfClosedIndex`/`htfMapped`, never `[0]` of a forming bar.
- TOD norms use prior completed sessions only.
- [ ] **Rolls (continuous unadjusted @ES).**
- Set `RollDetection = ERollDetection.Series` (or `Table` + `RollDates`). It is **Off** by default, and then nothing is roll-adjusted.

- `rollOn(1)` is true on the first bar of the new contract. Engines reset or adjust there: Daily `OnRoll` (history difference-adjusted, ATR **reseeded**: `Atr20d` NaN for `atrLength` sessions), PA `roll reset`, AVWAP `invalidated (roll)`, and a Regime refit deferred on the roll day.
- Exclude roll ±1 via `IsRollWindow(day)`. This works only when rings/FVG or RollDetection are active.
- [ ] **Half-days and holidays.**
- Use `_CmeHolidays.Rules.Get(globexDate)`: `.Closed`, `.NoRth`, `.HaltEt`, `.RthCloseEt`.
- TOD excludes half-days only if `todHalfDay` is overridden (the futures base does this, a plain `_CustomSignal` returns false).
- The Event calendar flags `EventDay.isHalfDay`.
- [ ] **Trading day.** On a plain `_CustomSignal` with a Globex template, override `tradingDayKey`/`sessionClock`/`isRthBar` (see 1.6).
- [ ] **Multi-data streams.**
- Orders always go to Data1.
- `HtfStreams` must carry the Data1 root; other markets go through `XmktLegs`.
- A leg with another root never inherits Data1 rolls.
- `requireDataStream` throws at StartCalc if a stream is missing.
- A leg is `Available` only if not stale (`maxStaleSec`, or 2× the leg bar).
- [ ] **Portfolio Trader.**
- `PortfolioPublish` is true only in PT.
- `sizeForRisk` arms M-PORT for Data1. Otherwise call `portfolioArm(side, entry, stop, qty)` next to `Send`, and `portfolioStop(stop)` when you trail.
- Apply `_PortfolioRiskManager` as the PT money-management signal. It sizes via `EntryContracts` and never blocks exits.
- `equityAvailable` in PT uses `Portfolio.MaxRiskEquityPerPosPercent`. The *Percent parameters are fractions (0.01 = 1%).
- [ ] **Validation data.**
- Engines step on closed Data1 bars, so the bar size changes results (`_Test6` journals a bar-size deviation).
- Validate on 1-min with Bar Magnifier, or on tick (`TickConfig` uses a DataLoader 1-tick request; its size outputs stay parked until `sizeOutputsVerified`).
- [ ] **Config timing.** Set configs in `Create()` or before `base.StartCalc()`. After that, the engines are already built. The exception is roll settings, which are read live.
- [ ] **Event table coverage (EV-4.6b, PRs #28/#29).** The engine disables itself (Error line ending `; engine disabled`, neutral answers) only when the event table is missing or its hash does not match `EventRules.ExpectedTableSha`. Coverage holes no longer disable it: each kind and day is `EEventCoverage.NoEvent/Event/Unknown`. Call `CoverageBlocks(day, mask, minTier, out kind)` and treat true as "no trade" (unless `allowPartialCoverage`). See Appendix G.
- [ ] **Time windows on a Globex chart.** ET 18:00–23:59 bars belong to the next trading day. A test like `tod >= 15:55` also needs `tod < 17:00`.
- [ ] **Status Bar numbers.** `asOfBar` arguments (VP, naked levels) take `Bars.FullSymbolData.Current` (1-based), not `Bars.CurrentBar`.
- [ ] **Cost floor (catalogue §2.1).** The minimum stop for @ES is **14 ticks** (costs ≤ 0.10 R at the catalogue's $16.98 round trip). Trades whose structural stop is under the floor are **skipped and logged**, not resized. The §3 examples that trade with a stop (3, 5, 7, 9, 10) measure R from the planned entry and skip with a journal line; see Appendix F for the "max(structural, floor)" wording.
- [ ] **Intraday rules (catalogue §2.1).** No entries 09:30–09:45 unless the idea is an opening idea; no new entries after 15:30; flat by 15:55.
- **Half-days:** on a CME early-close day the intraday examples (3, 5, 7, 9) go flat on the first bar closing at or after `RthCloseEt` − 5 min, the same rule as `A263Config.halfDayExitBefore` (example 6 gets it from `A263Core`).
- **News blackout:** −2/+10 min around tier-1 releases. M-EVENT's `EventBlackout` does this only if you set `EventConfig.blackoutPreMin`/`blackoutPostMin` (both default 0 = no blackout; `blackoutTier` defaults to T1). None of the §3 examples applies it.
- **One live position per instrument; ES and NQ count as one position.** Nothing in the base enforces this. M-PORT's `clusterCap` limits risk per correlation cluster (2% by default); it is a heat cap, not a one-position rule.

- [ ] **Exit orders start on the bar after the fill.** Examples 5, 7, 9 and 10 send their stop and target from the first bar close at which the position exists, so the fill bar itself has no working stop or target (on a 1m chart, one minute). Examples 2 and 3 send the stop together with the entry.

---

2 — Gate 0: the level baseline and the canaries

Gate 0 is a **measurement**, not a trading idea. It answers two questions before any level-based idea in the catalogue is read:

1. **G0-d (level baseline):** does price react to a given level type (prior POC, IB high, weekly VAH, …) more than to matched random prices at the same distance? Scored by the A2-33 matched-control study (`_LevelStudy`).
2. **G0-b (canaries):** do deliberately naive "known loser" rules lose after costs? A canary that shows a **net profit factor > 1.0 is treated as a bug** in fills, costs, lookahead or session handling, not as an edge.

The other Gate-0 items are conventions the examples follow rather than code you run: **G0-a** is the cost model (A2-40 + A2-59; the canary commission/slippage below is the part `_TestGate0` documents), and **G0-c** is roll honesty (roll window excluded, multi-day levels reset at rolls).

> Sources: `_Gate0.Function.CS` (engine), `_TestGate0.Strategy.CS` (host and the setup header quoted below), `tests/Gate0Tests`, PR #20 and its review `specs/pr20-review-analyst2.md`. Catalogue §2.6 defines G0-a…G0-d. **Discrepancy:** the catalogue text names the host `_Test1`; the code on `main` hosts Gate 0 in `_TestGate0` (`_Test1` is a plain `SignalObject`). **KD-02** is listed as a G0-b canary in the catalogue (v2.3) but is **not** in `EGate0Canary`, so it cannot be run yet.

2.1 The pieces

Piece	File · class	Role
Engine	`_Gate0.Function.CS` · `Gate0Engine` (sealed, MC-free)	Builds the level families per RTH session, feeds `_LevelStudy`, runs canaries, bootstrap, verdicts
Bar	`Gate0Bar` (struct)	one closed Data1 bar: ET time, Globex day, OHLCV, roll flag
Host	`_TestGate0.Strategy.CS` · `_TestGate0 : _VolumeProfileFuturesBase`	Converts bars, routes canary orders, journals trades and totals
Study	`_LevelStudy.Function.CS` · `_LevelStudy`	A2-33 matched controls, t, Holm, half-life, terciles
RNG	`_Rng.Function.CS` · `_Rng`	SplitMix64; seed 33
Tests	`tests/Gate0Tests`	unit tests of the engine (run with `bash tests/Gate0Tests/run.sh`)

The engine's public surface (full list in Appendix E under `_Gate0.Function.CS`):

```
// _Gate0.Function.CS (namespace PowerLanguage.Function)
public enum EGate0Mode   { Strategy = 0, EventStudy = 1, ControlRT = 2, ControlFL = 3, Canary = 4 }
public enum EGate0Family { All = 0, PriorProfile = 1, NakedPoc = 2, IbEdge = 3, Overnight = 4, Avwap = 5, Vwa
p = 6,
                           Weekly = 7, CompositeVa = 8, SinglePrint = 9, DvSource = 10, HvnLvn = 11, Mixture
= 12,
                           SigmaMixture = 13 }
public enum EGate0Canary { VW08 = 0, A2_24_i = 1, A2_24_ii = 2, A2_24_iii = 3, DV10 = 4, A2_53 = 5, DV01 =
6, PS03 = 7 }

// class Gate0Engine
public EGate0Mode mode = EGate0Mode.EventStudy;
public int variant;
public double tick = 0.25;
public long seed = 33;
public double vaPct = 0.70;
public Action<string> journal;
public string Configure();                      // null/"" = OK, else an error text (the host throws it)
public void Bar(Gate0Bar b, int mp = 0);        // one CLOSED Data1 bar; mp = market position sign
public List<Gate0Result> Finish();              // scores and journals; call once
public bool InRollWindow(DateTime day);
public static string Verdict(int n, double t, string a2);
```

```
public static string PreRegistration();
public static readonly DateTime DevEnd = new DateTime(2025, 1, 1);
public const int BootstrapDraws = 500, BootstrapBlock = 5;
public const int Controls = 20, ReactionBars5m = 12, MaxAge = 15;
public const int CompositeSessions = 5;
public const double KdeBandwidthTicks = 6, KdeProminence = 0.10;
```

`Configure()` returns these errors (the host turns them into an `ArgumentException` at StartCalc): `tick must be > 0`; `Canary Variant must be 0..7 or 100..123 (A2-53 sweep)`; `level-study Variant must be 0..13`; or the `_LevelStudy.Configure()` error. Mode **Strategy** is not runnable: `_TestGate0.StartCalc` throws `Mode Strategy does not run Gate 0 - use EventStudy/Control*/Canary`.

2.2 Inputs and pre-registered constants

`_TestGate0` has exactly two `[Input]`s, plus the inherited `TraceLevel`:

- **Mode** (`EGate0Mode`, default `EventStudy`):
- `EventStudy`: the real level family `Variant`. Each level registered at the RTH open gets 20 matched controls at 0.9–1.1× its distance from the RTH open price, random side.
- `ControlRT` (negative control, "random time"): each level is replaced by the 1m close at a random bar of the prior RTH session; labels are prefixed `RT:`. Expected: PARK. A PASS is a bug.
- `ControlFL` (negative control, "fake level"): each level shifted by ±{0.5, 1, 2} × ATR14(5m); labels prefixed `FL:`. Expected: PARK. A PASS is a bug.
- `Canary`: G0-b canary number `Variant`, 1 contract.
- **Variant** (int):
- level-study modes: 0 = all families (**overview only, not a verdict run**: controls drop to 5 and every line is prefixed `NOT A VERDICT RUN`); 1–13 = one `EGate0Family` each. Run 1…13 one by one for G0-d.
- Canary mode: 0–7 = `EGate0Canary`; 100–123 = the A2-53 sweep (idx = Variant − 100; gap {0.3, 0.5, 0.8}% [idx / 8]; j = 1 + (idx / 4) % 2; bar 30m if (idx / 2) % 2 == 0 else 15m; vol filter on if idx is even). Canary 5 (A2-53 default) equals 108.
- **TraceLevel**: set it to **Journal**, or nothing is printed.

Everything else is a pre-registered constant in `_Gate0.Function.CS`: seed 33, value area 70%, 500 bootstrap draws in blocks of 5 sessions, 20 controls per level, reaction window 12 × 5m bars after the first touch, sigma = ATR14(5m) frozen at the RTH open, maximum level age 15, composite of 5 sessions, KDE bandwidth 6 ticks and prominence 10%. There is no config object to edit; changing a constant is a new pre-registration.

The level families (Variant 1–13) and their journaled types:

V	Family	Types
1	Prior profile	`POC`, `VAH`, `VAL` (age 1–15)
2	Naked POCs	`NPOC` (age 2–15)
3	Initial balance	`IBH`, `IBL` (prior 09:30–10:30)
4	Overnight	`ONH`, `ONL` (18:00 → 09:30)
5	AVWAP anchors	`AV_PH`, `AV_PL`, `AV_WK`, `AV_MO`, `AV_OPEN`, `AV_PC`
6	VWAP	`VWAP`, `VWAP+1`, `VWAP-1` (prior session, at its close)
7	Weekly profile	`WPOC`, `WVAH`, `WVAL`
8	Composite VA	`CVAH`, `CVAL` (last 5 RTH sessions)
9	Single prints	`SP_HI`, `SP_LO` (primary: interior runs ≥ 4 rows, TPO count 1); `SPV_HI`, `SPV_LO` (sensitivity: volume ≤ 5% of the POC row)
10	DV sources	`DV_ROT_POC_R6`, `DV_ROT_POC_R12`, `DV_TREND_HVN`
11	HVN / LVN	`HVN`, `LVN` (KDE 6 ticks, primary); `HVN_HCV`, `LVN_HCV` (bandwidth h_cv, sensitivity)
12	Mixture (A2-27)	`GMM_MU`, `GMM_EDGE`

V	Family	Types
13	Sigma mixture (A2-32)	`SIG_MU`, `SIG_EDGE`

The canaries (Variant 0–7 in Canary mode; all flat by 15:55 ET, RTH only):

V	Canary	Rule (as documented in the `_TestGate0` header)
0	VW-08	long on a 1m close above RTH VWAP, short below, reverse on the opposite close
1	A2-24 (i)	fade the first VWAP touch after 09:35; stop/target 1R = ATR14(5m)
2	A2-24 (ii)	fade a 1m close beyond VWAP ±2σ; target VWAP frozen at entry; stop 1:1
3	A2-24 (iii)	fade a 1m close ≥ 2 × ATR14(5m) from VWAP; target VWAP frozen; stop 1:1
4	DV-10	weekly VWAP ±1σ fade when both bands moved < 0.5 × ATR30m over 8 × 30m bars; limit at the band; TP weekly VWAP; SL 0.20 × ATR20d
5	A2-53	gap ≥ 0.5%, vol filter on; fade after the 1st 30m bar if the prior close was not retraced; stop-and-reverse at the session extreme ±1 tick (one reversal)
6	DV-01	prior RTH POC after a move-away ≥ 1 × ATR14(5m); limit at the POC, first touch only; SL = TP = 0.15 × ATR20d
7	PS-03	naive 80% rule: open outside the prior VA, enter on the first 30m close back inside; target the prior POC; stop 0.25 × VA width beyond the entry edge

Roll honesty. In level-study modes all multi-session sources restart at the roll, and events on the session **before** a roll, the roll session and the session after are dropped before scoring. In Canary mode the roll session and the session after are skipped live (journal `G0-b roll-skip {day} total(n)`), and the session before a roll is removed from the "ex-roll" totals afterwards.

2.3 Outputs and journal lines

All lines go to the PowerLanguage Output window and the `_CustomSignal` trace log (Appendix B) when `TraceLevel = Journal`.

- At StartCalc (level-study modes): `G0 PRE-REGISTERED: …` (the DV rotation types and the HVN/LVN bandwidth arms).
- Once, at the first bar of 2025 or the last bar on the chart, whichever comes first:
- `A2-33 …` summary lines from `_LevelStudy` per type;
- one `G0 type(T) levels(n) touched(t) pairs(p) edgeR(e) t() p() holm() ci95[lo,hi] sessions(s) halfLife(h) tTercile(lo/mid/hi) tByAge[age:t/n …] A2-33(rule) => PASS | PARK (reason)` line per type;
- `G0 done mode() family() types() rollDays() sessions()`.
- Canary mode: one `G0-b trade {canary} L|S {entry time} {price} -> {exit time} {price} gross() net() cumGross() cumNet()` line per closed trade, then `G0-b canary(...) entries() rollSkips() cumGross() cumNet() - read NET profit factor in the Strategy Performance Report; > 1.0 = BUG` and four splits: **all**, **ex-roll**, **2016-2024** and **2019-2024**. The two dated splits also exclude the roll window (code: `printTotals`).

Definitions (from the `_TestGate0` header):

- **R** = max over the 12 5m bars after the first touch of |close − level| / sigma, sigma = ATR14(5m) at the RTH open.
- **edgeR** = mean R of touched levels minus the mean R of their touched controls. t and p come from `_LevelStudy`.
- **holm** = Holm step-down across all Gate-0 level types, with m fixed at the full type count (so per-family runs are conservative).
- **tByAge** = t per age 1–15; an `x` marks |t| < 2 (that age is not traded).

- **tTercile** = t in the low / mid / top quality tercile (HVN/LVN prominence, GMM weight; other types have no quality).
- **ci95** = 2.5/97.5 percentiles of edgeR over 500 moving-block bootstrap draws (blocks of 5 sessions).

2.4 How to read the verdict

The verdict is `Gate0Engine.Verdict(n, t, a2)` with n = pairs. In code order:

1. n < 30 → `PARK (no evaluation: n<30)`.
2. 30 ≤ n < 150 → `PARK (inconclusive: 30<=n<150)`.
3. A2-33's rule starts with "trade top" → `PASS (trade top-quality tercile only …)`. This check comes **before** the |t| test, so a top-tercile PASS can carry an overall |t| < 2.
4. t is NaN or |t| < 2 → `PARK (kill: edge ~ 0, |t| < 2 - level type not traded)`.
5. A2-33's rule starts with "keep" → `PASS (keep …)`.
6. Otherwise → `PARK (kill: <A2-33 rule or "A2-33 drop">)`.

How to use it:

- A level type is usable by a G0-d-gated idea only with a **PASS from a single-family run (Variant 1–13) in EventStudy mode**. Variant 0 lines are never verdicts.
- `ControlRT` and `ControlFL` must come back PARK for the same family. A control PASS means the harness, not the market, produced the edge: stop and investigate.
- Keep only the ages without an `x` in `tByAge`; if t decays with age, keep ages up to the half-life.
- For **canaries**, the verdict is the **net** profit factor in the Strategy Performance Report over the development window: > 1.0 = bug to investigate, ≤ 1.0 = the plumbing behaves. Compare the "ex-roll" split with "all" to see whether roll sessions are driving anything.
- Gate 0 says nothing about whether any idea makes money. A PASS only says a level type reacts more than its matched controls.

2.5 Step by step: an @ES Gate-0 baseline

1. **Import** (PowerLanguage .NET Editor → File → Import, then compile), in the header's order: `_Rng.Function.CS`; `_LevelStudy.Function.CS`; `_BarRing`, `_ProfileMath`, `_ProfileStats`; `_Gmm`; `_LevelRef`, `_RollDetector`; the `_CustomSignal` / `_VolumeProfileBase` / `_VolumeProfileFuturesBase` chain and their helper functions; then `_Gate0.Function.CS` and `_TestGate0.Strategy.CS`.
2. **Chart:** @ES from dxFeed, continuous and **not back-adjusted** (the levels and roll logic assume raw prices). Data1 = **1-minute** bars (recommended; the 2016–2024 window exists only as 1m history) or tick bars. Never bars longer than 1 minute: the engine re-aggregates Data1 into 1m/5m/30m itself. No Data2.
3. **Session:** the "CME Globex" template (18:00–17:00 ET). RTH is derived internally. Any chart time zone works.
4. **Date range:** 2016-01-01 (or the first 1m bar) to 2024-12-31. The engine hard-stops at 2025-01-01 (`DevEnd`), so the holdout stays unseen even if more data is loaded. Allow about 5 months of warm-up (ATR20d needs 20 RTH sessions, HV100 about 100).
5. **Insert `_TestGate0`.** Format Signals → Inputs: `TraceLevel = Journal`, `Mode = EventStudy`, `Variant = 1`. Properties: Max bars back 1 is enough (all state is internal).
6. **Run** and read the Output window or the trace log. Record the `G0 PRE-REGISTERED:` line and the `G0 type(...)` lines.
7. **Repeat** for Variant 2…13 one by one, then each family again with `Mode = ControlRT` and `Mode = ControlFL`.
8. **Canaries:** `Mode = Canary`, Variant 0–7 (and 100–123 for the A2-53 sweep). In Format Signals → Properties set: - commission **$2.25 per contract per side**; slippage **1 tick per contract per side**; fixed **1 contract**; - **Bar Magnifier ON at 1 tick** where tick history exists (2024+). Before that MC uses its intrabar O-H-L-C assumption. Canaries that rest stop/limit orders (A2-24 i–iii, DV-10, DV-01, PS-03, the A2-53 reversal) need it for honest stop-versus-target ordering; - if your

MC version offers it, "fill limit order when the trade price goes beyond the limit price" (1 tick): DV-01 and DV-10 require a 1-tick trade-through.

9. **Read** the canary's net profit factor in the Strategy Performance Report and the journal splits.

Pitfalls:

- Leaving `Variant = 0` and reading its lines as verdicts.
- A back-adjusted series: the levels are wrong and roll detection misfires.
- Data1 longer than 1 minute: the 5m/30m re-aggregation and the 12 × 5m reaction window break.
- TraceLevel left at None: the run completes and prints nothing.
- Loading data past 2024: harmless (hard stop), but the "last bar" trigger fires at the first 2025 bar, so the results appear there, not at the chart's end.
- Bar Magnifier off on canaries with resting orders: stop-versus-target ordering inside a 1m bar is then MC's assumption.
- **Cost per round trip.** The header's MC settings ($2.25 commission and 1 tick = $12.50 slippage, each **per side**) give $4.50 + $25.00 = **$29.50** per ES round trip. The catalogue's cost floor assumes $16.98 (IBKR round trip + 1 tick in total). That is a G0-a conflict (Appendix F). If you want to match the catalogue, set slippage to 0.5 tick per side ($6.25 per side, $12.50 per round trip), which gives $17.00. Don't change the code: these are MC settings.
- **Cost floor.** @ES stops under 14 ticks are skipped and logged, not resized (catalogue §2.1; §1.7).
- **VERIFY:** whether G0-a's full A2-40 cost model (including the 2-tick slippage at 09:30–09:35) is applied anywhere; MC's fixed per-side slippage cannot model a time-of-day slippage.

2.6 Which examples in §3 depend on Gate 0

Ideas that trade a **level type** are G0-d-gated: they are not read until that level type has a Gate-0 PASS. In §3 these are examples 7 (PS-21: naked POC/VAH/VAL), 9 (PS-13/DV-03: rotation POC, family 10), 10 (MT-02: weekly VAH, family 7), and the AVWAP arms of example 3 (AV-08, family 5). Example 8 (A2-29) is a **conflict in the catalogue**: G0-d's table lists the A2-25…A2-32 range as gated, but A2-29's own entry says "Gate (v2): none (classifier)". A2-29 is a classifier that does not trade a level, so the entry is taken here, and the conflict is listed in Appendix F. The others (A2-69, TS-01, A2-50, A2-57, A2-63) do not trade a level and are not G0-d-gated. Every example inherits G0-a (costs) and G0-b (canaries must still pass after any engine change).

---

3 — Ten worked examples, easy to hard

Each example implements one catalogue entry as a complete strategy you can import. They are written for @ES on dxFeed continuous **unadjusted** data with the CME Globex session template.

Read these first:

- **Compile status.** Every example below is **compiled: type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MultiCharts API stubs**. The stubs are mine and are not authoritative: a member MC names differently would only show up in the PowerLanguage editor. **None of the examples has been run in MultiCharts or backtested.** They carry no results, and nothing here says any of them makes money.
- **Rules.** The rules quoted are the catalogue's. Where the catalogue leaves something open, the code makes a choice and labels it in a comment ("assumption", "coding choice", or "illustrative"). Those labels are listed again under each example.
- **Performance figures** quoted by the books or the catalogue are deliberately not repeated here.
- **Orders** go to Data1. HTF streams are read through `HtfClosedBar` (closed bars only).
- **Gate 0.** Examples 3 (AVWAP arms), 7, 8, 9 and 10 trade or classify on level types that are G0-d-gated (§2.6): run their Gate-0 family first.

#	ID	Idea	Difficulty	Engines used	File
1	A2-69	Kaufman 3-day trade	easy	session clock, holidays, journal	`_Ex01KaufmanThreeDay.Strategy.CS`
2	TS-01	European-open overnight drift	easy	HTF, holidays, M-EVENT	`_Ex02EuropeanOpenDrift.Strategy.CS`
3	AV-08	Week/month VWAP regime layer	easy–medium	VWAP slots, rolls	`_Ex03PeriodAvwapLayer.Strategy.CS`
4	A2-50	Same-slot relative-volume gate	medium	M-TOD, M-EVENT	`_Ex04OpeningRvGate.Strategy.CS`
5	A2-57	60m turn + first 12m oversold	medium	HTF (2 streams)	`_Ex05TwoTimeframeTurn.Strategy.CS`
6	A2-63	Open + k × ATR20d breakout, RegimeGate	medium	M-DAILY, G-R, RegimeGate	`_Ex09OpenAtrBreakout.Strategy.CS`
7	PS-21	Naked POC/VAH/VAL first touch + 5m rejection	medium–hard	VP, naked levels, HTF	`_Ex06NakedLevelReaction.Strategy.CS`
8	A2-29	Entropy day type at 11:00	medium–hard	range profile, entropy	`_Ex08EntropyDayType.Strategy.CS`
9	PS-13 / DV-03	Rotation POC first-touch retest	hard	range profile, HTF, own daily ATR	`_Ex07RotationVolumeLevel.Strategy.CS`
10	MT-02	Weekly accepted value + 60m trigger	hard	weekly profiles, M-DAILY, HTF	`_Ex10WeeklyValueAcceptance.Strategy.CS`

The file names keep the numbers they were written under; the table gives the reading order.

Common pitfalls for all ten:

- **Evening wrap.** On a Globex chart, bars stamped 18:00–23:59 ET belong to the **next** Globex day. A check like `tod >= 15:55` is also true at 19:00, so the examples write `tod >= X && tod < 17:00`.
- **Closed bars.** Every example returns early unless `Bars.Status == EBarState.Close`, and calls `base.CalcBar()` first.

- **Rolls.** Examples that set `RollDetection = ERollDetection.Series` clear multi-day state in `rollOn(1)`. With RollDetection on, M-DAILY's ATR is reseeded at each roll and stays NaN for `atrLength` sessions (§4.3).
- **Configs** are set in `Create()` or before `base.StartCalc()`.

3.1 Example 1 (`_Ex01KaufmanThreeDay`) — A2-69 Kaufman 3-day trade (easy)

Catalogue rules (A2-69, core arm). Setup: C(t−1) < C(t−2) < C(t−3), two lower RTH closes. Entry (long only): if the open O(t) < C(t−1), buy at 09:31 (the first 1m bar's open); otherwise buy at 15:59 (MOC proxy). Exit at 15:59 on the following session. No stop in the core. The SMA, profit-take, spike and Friday-short arms are not coded.

Chart. @ES 1m, dxFeed continuous unadjusted, CME Globex template. Inputs: only `TraceLevel`. Settings are in the protected `A269Config`.

Walk-through.

- The RTH close of each session is the close of its last RTH bar (`isRthBar`, through 16:00 ET or the early close), collected per Globex day. NoRth days never set up.
- At the start of each Globex day the setup is tested on the last three completed RTH closes. At the close of the first RTH bar (close-stamped 09:31), if that bar's open is below C1, a market order goes in for the next bar's open.
- Otherwise it waits for the bar close-stamped 15:59 (one minute before an early close) and sends a market order for the next bar.
- The exit is sent the same way at 15:59 on any later session (normally the next one).
- With TraceLevel Journal, `journalArm` records the planned entry and `logSignal` writes an `MCEvents` row for each entry.

Pitfalls.

- "Lower open" compares the RTH open with the prior **RTH** close, not the Globex close.
- On an early-close day the "15:59" bar is one minute before `RthCloseEt`.
- **Coding choice (labelled):** RollDetection is Series and `rollOn(1)` clears the close history, so the setup never compares closes from two contracts. The catalogue does not say this.
- A 1m entry "at 09:31" is the next bar's open after the 09:31 bar closes. That is the closest MC can do with market-next-bar orders.

Full strategy — `_Ex01KaufmanThreeDay.Strategy.CS`:

```
// Manual v3 example 1 - A2-69 Kaufman 3-day trade (core arm: open-else-close entry, exit next close, long
only).
// Chart: @ES 1m, dxFeed continuous UNADJUSTED, CME Globex session template, Bar Magnifier off (1m is the
fill bar).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;
using System.Globalization;

namespace PowerLanguage.Strategy
{
    public class _Ex01KaufmanThreeDay : _CustomSignal
    {
        // ---- protected config (no [Input]s besides TraceLevel) ----
        protected sealed class A269Config
        {
            public int contracts = 1;
            public TimeSpan rthOpen = new TimeSpan(9, 30, 0);
            public TimeSpan closeBar = new TimeSpan(15, 59, 0);   // ET close-stamp of the bar whose close
sends the "15:59" order
        }
        protected A269Config Cfg { get; } = new A269Config();

        private IOrderMarket m_le, m_lx;
        private _CmeSessionClock m_clock;
```

```
        private readonly List<double> m_closes = new
List<double>();   // 16:00 ET RTH closes C(t-1), C(t-2), C(t-3) ... newest first
        private DateTime m_day = DateTime.MinValue, m_entryDay = DateTime.MinValue;
        private double m_rthClose = double.NaN;   // last RTH value-window close of the current trading day
        private bool m_setup, m_openChecked, m_waitClose;

        public _Ex01KaufmanThreeDay(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        // CME session adapter (section 1.6): Globex trading day + ET clock + RTH value window (09:30-16:00
ET, early close aware).
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "K3LE", EOrderAct
ion.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "K3LX", EOrderAction.Se
ll));
        }

        protected override void StartCalc()
        {
            m_clock = null;
            RollDetection = ERollDetection.Series;   // unadjusted series: detect splices
            base.StartCalc();
            m_closes.Clear(); m_rthClose = double.NaN; m_day = m_entryDay = DateTime.MinValue;
            m_setup = m_openChecked = m_waitClose = false;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime et = Clock.ToEastern(Bars.Time[0]);
            DateTime day = tradingDayKey(Bars.Time[0]);
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan closeBar = cme.RthCloseEt.HasValue ? cme.RthCloseEt.Value - TimeSpan.FromMinutes(1) : Cf
g.closeBar;   // half-day

            if (rollOn(1))
            {
                m_closes.Clear(); m_rthClose =
double.NaN;   // the series jumps by the roll gap: restart the 3-close count (journaled below)
                if (journalOn()) trace(ETraceLevel.Journal, $"A2-69 roll at {Bars.Time[0]:yyyy-MM-dd HH:mm}:
close history cleared");
            }

            // 1. new trading day: push the completed session's RTH close as C(t-1), then evaluate the setup
once
            if (day != m_day)
            {
                m_day = day; m_openChecked = false; m_waitClose = false;
                if (!double.IsNaN(m_rthClose)) { m_closes.Insert(0, m_rthClose); if (m_closes.Count > 3) m_cl
oses.RemoveAt(3); }
                m_rthClose = double.NaN;
                m_setup = !cme.NoRth && m_closes.Count == 3 && m_closes[0] < m_closes[1] && m_closes[1] < m_c
loses[2];
                if (journalOn())
                    trace(ETraceLevel.Journal, $"A2-69 {day:yyyy-MM-dd} closes({string.Join(",", m_closes)})
setup({m_setup}) noRth({cme.NoRth}) halfDay({cme.RthCloseEt.HasValue})");
            }

            bool rth = isRthBar(Bars.Time[0]);
            if (rth) m_rthClose = Bars.Close[0];   // ends as the 16:00 (or early-close) bar close

            // ET 18:00-23:59 bars belong to the NEXT trading day, so "after 15:59" must also be "before
17:00"
            bool atClose = et.TimeOfDay >= closeBar && et.TimeOfDay < new TimeSpan(17, 0, 0);

            int pos = StrategyInfo.MarketPosition;

            // 2. exit: on the session after the entry day, at the 15:59 bar
```

```
            if (pos > 0 && day > m_entryDay && atClose) { m_lx.Send(); return; }
            if (pos != 0 || !m_setup) return;

            // 3. first RTH bar (close-stamped 09:31): O(t) is that bar's open; lower open -> buy the next
bar's open
            if (!m_openChecked && rth)
            {
                m_openChecked = true;
                double o = Bars.Open[0];
                if (o < m_closes[0]) { arm(o, "open"); return; }
                m_waitClose = true;
            }
            // 4. otherwise buy at the 15:59 bar (MOC proxy)
            if (m_waitClose && atClose) { m_waitClose = false; arm(Bars.Close[0], "close"); }
        }

        private void arm(double planned, string how)
        {
            m_setup = false; m_entryDay = m_day;
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "c1", m_closes[0].ToString() }, { "how", how } }
;
                journalArm("K3LE", planned, double.NaN, double.NaN, ctx);
                logSignal("A2-69", +1, planned, double.NaN, ctx, true);
            }
            m_le.Send(Cfg.contracts);
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.2 Example 2 (`_Ex02EuropeanOpenDrift`) — TS-01 European-open overnight drift (easy)

Catalogue rules (TS-01). Buy at 01:30 ET, sell at 04:30 ET every trading day. Skip CME holidays and, in the event variant, the nights before tier-1 08:30 releases. No stop, but a disaster stop at 2 × ATR60m. Sweep: window {01:30–04:30, 02:00–03:00} × event filter {off, on}.

Chart. Data1 = @ES 1m, Data2 = @ES 60m (same symbol and session), dxFeed unadjusted, 24h Globex template. `[Input] Variant` 1–4: 1 = 01:30–04:30, 2 = 02:00–03:00, 3 and 4 = the same windows with the event filter.

Walk-through.

- `HtfStreams = {2}` makes Data2 a same-instrument HTF stream, read with `HtfClosedBar`.
- The entry is a market order sent at the close of the **first closed bar** stamped at or after one minute before the window start (and before the window end). Normally that is the bar one minute before the start, so it fills at the window start. If bars are missing, the entry is late, and the journal's `sendBar` field shows when. If the window passes with no closed bar in it, the night is journaled as `skip: no bar` and not traded. The exit is sent at the close of the first bar stamped at or after the window end (and before 17:00 ET).
- Days that CME has Closed, NoRth days and halt days are skipped (`_CmeHolidays.Rules.Get`).
- Event variant: the engine is enabled with the Macro class. The night is skipped if a T1 event at `EventRules.T0830` is on the coming trading day (`EventsOn`), or if `CoverageBlocks(day, Macro, T1, out hole)` reports an Unknown coverage day (EV-4.6b). If the event engine is missing or `Disabled`, the filter **fails closed** (no trade), because a disabled engine answers neutrally and would let every night through.
- Disaster stop: planned price − 2 × ATR60m from closed 60m bars, sent with the entry and re-sent every bar.

Pitfalls.

- **Assumption (labelled):** ATR60m length 14, as a simple average of true range. The catalogue does not give the length.
- The window sits in the evening-wrapped part of the Globex day: 01:30 ET belongs to the same Globex date as the following RTH session, so "the night before an 08:30 release" means the same Globex date.
- `EventsOn` is as-of: an event added to the table after the bar is not visible.
- GDP/PCE coverage holes 2016–24 (EV-4.6b) are the case `CoverageBlocks` handles.
- RollDetection stays Off here, so an ATR60m window that spans a roll includes the roll gap.
- The catalogue's TS-01 is written for **MES** (kill line $2.47) on 30m/60m bars. This example uses @ES on 1m with a 60m stream, so its costs are ES costs (§2.5).

Full strategy — `_Ex02EuropeanOpenDrift.Strategy.CS`:

```
// Manual v3 example 2 - TS-01 European-open overnight drift (long 01:30-04:30 ET, disaster stop 2 x ATR60m).
// Chart: Data1 = @ES 1m, Data2 = @ES 60m (same symbol, same session), dxFeed continuous UNADJUSTED, 24h CME
Globex template.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex02EuropeanOpenDrift : _CustomSignal
    {
        /// <summary>Sweep (catalogue): 1 = 01:30-04:30, 2 = 02:00-03:00, 3 = 1 + event filter, 4 = 2 + event
filter.</summary>
        [Input] public int Variant { get; set; }

        protected sealed class Ts01Config
        {
            public int contracts = 1;
            public double disasterAtr = 2.0;
            public int atrLength = 14;       // ATR60m length is not in the catalogue: an assumption of this
example
        }
        protected Ts01Config Cfg { get; } = new Ts01Config();

        private IOrderMarket m_le, m_lx;
        private IOrderPriced m_lxStp;
        private _CmeSessionClock m_clock;
        private TimeSpan m_start, m_end;
        private bool m_filter;
        private DateTime m_doneDay = DateTime.MinValue;
        private double m_stop = double.NaN;

        public _Ex02EuropeanOpenDrift(object _ctx) : base(_ctx) { Variant = 1; }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "TS01LE", EOrderA
ction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "TS01LX",
EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "TS01DS", EOrderAction.Sell,
OrderExit.FromAll));
            // M-EVENT: created only when enabled (StartCalc). T1 only; coverage holes of lower tiers never
block (EV-4.6b).
            EventConfig.minTier = EEventTier.T1;
            EventConfig.classes = EEventClassMask.Macro;
        }

        protected override void StartCalc()
        {
```

```
            if (Variant < 1 || Variant > 4) throw new
ArgumentException("_Ex02EuropeanOpenDrift: Variant must be 1-4");
            bool narrow = Variant == 2 || Variant == 4;
            m_start = narrow ? new TimeSpan(2, 0, 0) : new TimeSpan(1, 30, 0);
            m_end = narrow ? new TimeSpan(3, 0, 0) : new TimeSpan(4, 30, 0);
            m_filter = Variant >= 3;
            EventConfig.enabled = m_filter;   // set before base.StartCalc: the engine is built there
            HtfStreams = new[] {
2 };         // M-MTF: StartCalc fails loudly if Data2 is missing or another root
            m_clock = null;
            base.StartCalc();
            m_doneDay = DateTime.MinValue; m_stop = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime et = Clock.ToEastern(Bars.Time[0]);
            DateTime day = tradingDayKey(Bars.Time[0]);   // the 01:30 ET bar belongs to the Globex day that
opened 18:00 ET the evening before
            TimeSpan tod = et.TimeOfDay;
            int pos = StrategyInfo.MarketPosition;

            if (pos > 0)
            {
                if (tod >= m_end && tod < new TimeSpan(17, 0, 0)) { m_lx.Send(); return; }
                if (!double.IsNaN(m_stop)) m_lxStp.Send(m_stop);   // re-send every bar (MC cancels unsent
orders)
                return;
            }
            m_stop = double.NaN;
            if (day == m_doneDay) return;
            // the entry order is sent on the close of the FIRST closed bar stamped at or after m_start - 1
min (normally that
            // bar, so it fills at the m_start bar's open); a missing bar delays the entry, it does not skip
the night
            TimeSpan sendFrom = m_start - TimeSpan.FromMinutes(1);
            if (tod >= m_end && tod < new TimeSpan(17, 0, 0))
            {
                m_doneDay = day;   // the window passed with no closed bar in it
                if (journalOn()) trace(ETraceLevel.Journal, $"TS-01 {day:yyyy-MM-dd} skip: no bar in
{sendFrom:hh\\:mm}-{m_end:hh\\:mm} ET");
                return;
            }
            if (tod < sendFrom || tod >= m_end) return;
            m_doneDay = day;

            string skip = skipReason(day);
            double atr = atrClosed(2, Cfg.atrLength);
            if (skip == null && !(atr > 0)) skip = "ATR60m warm-up";
            if (skip != null) { if (journalOn()) trace(ETraceLevel.Journal, $"TS-01 {day:yyyy-MM-dd} skip:
{skip}"); return; }

            double planned = Bars.Close[0];
            m_stop = planned - Cfg.disasterAtr * atr;   // planned from the signal close; the fill is the
next bar's open
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "variant", Variant.ToString() }, { "atr60",
atr.ToString("F2") }, { "sendBar", tod.ToString(@"hh\:mm") } };
                journalArm("TS01LE", planned, m_stop, double.NaN, ctx);
                logSignal("TS-01", +1, planned, m_stop, ctx, true);
            }
            m_le.Send(Cfg.contracts);
            m_lxStp.Send(m_stop);   // also protects the entry bar
        }

        private string skipReason(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            if (cme.Closed || cme.NoRth || cme.HaltEt.HasValue) return "CME holiday/halt " + cme.Name;
            if (!m_filter) return null;
            _EventCalendar ev = EventCalendar;
            if (ev == null || ev.Disabled) return "event filter on but M-EVENT disabled";   // fail closed
            EEventKind hole;
            if (ev.CoverageBlocks(day, EEventClassMask.Macro, EEventTier.T1, out hole)) return "coverage
```

```
Unknown " + hole;
            foreach (EventRec r in ev.EventsOn(day, EEventClassMask.Macro))
                if (r.tier == EEventTier.T1 && r.etTime.TimeOfDay == EventRules.T0830) return "T1 08:30 " +
r.KindName;
            return null;
        }

        /// <summary>Simple-average true range over the last n CLOSED bars of stream ds (needs n + 1 closed
bars).</summary>
        private double atrClosed(int ds, int n)
        {
            if (BarsOfData(ds).CurrentBar <= n + 1) return double.NaN;
            double sum = 0;
            for (int k = 0; k < n; k++)
            {
                DateTime t; double o, h, l, c, v, pc;
                HtfClosedBar(ds, k + 1, out t, out o, out h, out l, out pc, out v);
                HtfClosedBar(ds, k, out t, out o, out h, out l, out c, out v);
                sum += Math.Max(h - l, Math.Max(Math.Abs(h - pc), Math.Abs(l - pc)));
            }
            return sum / n;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.3 Example 3 (`_Ex03PeriodAvwapLayer`) — AV-08 week/month VWAP regime layer (easy–medium)

Catalogue rules (AV-08). A layer, not a strategy: `aboveWTD && aboveMTD` allows longs only, `belowWTD && belowMTD` allows shorts only. It is applied to host events (PS-01, PS-07, MT-01, AV-02). Sweep: {WTD only, WTD+MTD, MTD+QTD}. Multi-day anchors reset at rolls.

Chart. @ES 1m, dxFeed unadjusted, 24h Globex template. The class derives from `_VolumeProfileFuturesBase` and sets the two VWAP slots on Data1 in `Create`: `Vwap1Type = EVwapType.Week`, `Vwap2Type = EVwapType.Month`. `StartCalc` throws if they are changed.

Walk-through.

- Only the **long** side of the layer is coded: `layerLong` = both slots active, close > WTD VWAP and close > MTD VWAP, and both periods "clean".
- A period is clean only if no roll happened since the week start (Monday Globex date) or the month start, because the Week/Month VWAPs do **not** reset at a roll. RollDetection is Series.
- The host is a close above the prior 20-bar high between 09:45 and 15:00 ET, with a 16-tick stop (above the 14-tick cost floor), flat at 15:55, or at `RthCloseEt` − 5 min on an early-close day (`flatAt`). **It is illustrative only, not a catalogue rule**: it exists so the layer has events to filter.
- Host signals the layer blocks are logged with `logSignal(..., false)`, so the ablation (gated vs ungated) can be run from `MCEvents` without a second backtest.

Pitfalls.

- AV-08's test is a layer ablation on real hosts. Don't read anything into the illustrative host.
- The Week VWAP slot starts at the week's first bar; the catalogue's A2-07 merge anchors at Sunday 18:00. On a Globex chart those are the same bar.
- Not coded: the short side (`belowWTD && belowMTD`), the WTD-only arm and the MTD+QTD arm (`EVwapType.Quarter`).
- **Coding choice (labelled):** `wtdClean`/`mtdClean` switch the layer off for the rest of a week or month that holds a roll. The catalogue says multi-day anchors reset at rolls. The other reading is to re-anchor both VWAPs at `rollOn(1)`; it is not coded (Appendix F).

Full strategy — `_Ex03PeriodAvwapLayer.Strategy.CS`:

```
// Manual v3 example 3 - AV-08 week-open / month-open VWAP regime layer (aboveWTD && aboveMTD for longs
only).
// The host trigger below is ILLUSTRATIVE ONLY (not a catalogue rule): AV-08 is a layer for PS-01 / PS-07 /
MT-01 / AV-02.
// Chart: @ES 1m, dxFeed continuous UNADJUSTED, 24h CME Globex template.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex03PeriodAvwapLayer : _VolumeProfileFuturesBase
    {
        protected sealed class Av08Config
        {
            public int contracts = 1;
            public int hostLookback = 20;                       // illustrative host: close above the prior
20-bar high
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 0, 0), flat = new TimeSpan(1
5, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);   // early-close days: flat at
RthCloseEt - 5 min
            public double stopTicks = 16;                       // illustrative host stop (>= the ES cost
floor of 14 ticks)
        }
        protected Av08Config Cfg { get; } = new Av08Config();

        private IOrderMarket m_le, m_lx;
        private IOrderPriced m_lxStp;
        private DateTime m_lastRollDay = DateTime.MinValue;
        private double m_stop = double.NaN;

        public _Ex03PeriodAvwapLayer(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "AV08LE", EOrderA
ction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "AV08LX",
EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "AV08LS", EOrderAction.Sell,
OrderExit.FromAll));
            // The VWAP slots are [Input]s of the VP base: these are defaults, the Format Signal dialog can
override them.
            Vwap1Type = EVwapType.Week;  Vwap1DataStream = 1;   // WTD: resets on the first bar of a new
Monday-start Globex week
            Vwap2Type = EVwapType.Month; Vwap2DataStream = 1;   // MTD
            Vwap3Type = EVwapType.Disabled;
        }

        protected override void StartCalc()
        {
            base.StartCalc();
            if (Vwap1Type != EVwapType.Week || Vwap2Type != EVwapType.Month || Vwap1DataStream != 1 || Vwap2D
ataStream != 1)
                throw new ArgumentException("_Ex03PeriodAvwapLayer: needs Vwap1Type = Week and Vwap2Type =
Month on Data1");
            m_lastRollDay = DateTime.MinValue; m_stop = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], day = GlobexDate(t);
            TimeSpan tod = ToEastern(t).TimeOfDay;
            if (rollOn(1)) m_lastRollDay = day;
            int pos = StrategyInfo.MarketPosition;
            TimeSpan flat = flatAt(day);

            if (pos > 0)
            {
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) m_lx.Send();
                else if (!double.IsNaN(m_stop)) m_lxStp.Send(m_stop);
```

```
                return;
            }
            if (tod < Cfg.from || tod >= Cfg.to || tod >= flat || Bars.CurrentBar <= Cfg.hostLookback + 1) re
turn;

            // --- the AV-08 layer (closed bar: the VWAP slots include this bar) ---
            double wtd = CurrentVwap1, mtd = CurrentVwap2, c = Bars.Close[0];
            DateTime weekStart = day.AddDays(-(((int)day.DayOfWeek + 6) % 7));   // Monday of the Globex week
            DateTime monthStart = new DateTime(day.Year, day.Month, 1);
            // Week/Month VWAPs do NOT reset at a roll. CODING CHOICE: the layer is off for the rest of a
week/month that holds a roll
            // (the catalogue says multi-day anchors reset at rolls; the alternative is to re-anchor the
VWAPs at rollOn(1))
            bool wtdClean = m_lastRollDay < weekStart, mtdClean = m_lastRollDay < monthStart;
            bool layerLong = Vwap1Active && Vwap2Active && wtdClean && mtdClean && c > wtd && c > mtd;

            // --- illustrative host event ---
            double hh = double.MinValue;
            for (int i = 1; i <= Cfg.hostLookback; i++) hh = Math.Max(hh, Bars.High[i]);
            if (!(c > hh)) return;

            var ctx = new Dictionary<string, string> { { "wtd", wtd.ToString("F2") }, { "mtd", mtd.ToString("
F2") },
                { "wtdClean", wtdClean.ToString() }, { "mtdClean", mtdClean.ToString() } };
            if (!layerLong) { if (journalOn()) logSignal("AV-08", +1, c, double.NaN, ctx, false); return; }
            m_stop = c - Cfg.stopTicks * tickSize();
            if (journalOn()) { journalArm("AV08LE", c, m_stop, double.NaN, ctx); logSignal("AV-08", +1, c, m_
stop, ctx, true); }
            m_le.Send(Cfg.contracts);
            m_lxStp.Send(m_stop);
        }

        /// <summary>Flat time for a Globex day: Cfg.flat, or on a CME early-close day RthCloseEt -
halfDayExitBefore if earlier
        /// (_CmeHolidays.Rules.Get(day).RthCloseEt; same rule as A263Config.halfDayExitBefore).</summary>
        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.4 Example 4 (`_Ex04OpeningRvGate`) — A2-50 same-slot relative-volume gate (medium)

Catalogue rules (A2-50). RV30 = Σ volume(09:30–10:00) / SSB(N = 20) of the same 30-minute window. HIGH if RV30 ≥ q_hi, LOW if ≤ q_lo, else MID. q_hi/q_lo are the trailing 250-session 70th/30th percentiles, rolled daily and causal. Computed once at 10:00; 08:30-report days are a separate stratum. It is a module that routes hosts, so it places no orders.

Chart. Data1 = @ES **30m** (bars end 10:00, 10:30, …); the strategy throws on another bar size. dxFeed unadjusted, 24h Globex template.

Walk-through.

- M-TOD is enabled for Volume with `TodSessions = 20`, `TodMinSessions = 20` and bucket 0 (the host then uses the Data1 bar size, 30 minutes).
- `todHalfDay` is overridden so early-close days stay out of the baseline.
- At the 10:00 bar close, RV30 = that bar's volume / the mean of the last 20 values in `TodNormaliser.VolumePool(todMinuteOfSession(t))` (prior sessions only; `base.CalcBar()` runs first). Half-days and NoRth days get no state.

- RV30 is compared with the trailing 250 completed sessions' values (strictly earlier), then pushed. The state is HIGH, MID or LOW, exposed as `Rv30State` and journaled.
- The event engine is used only to **tag** the day as an 08:30-report day (or as unknown coverage), so the stratum can be split later.

Pitfalls.

- **Assumption (labelled):** nearest-rank percentiles.
- The first 250 sessions after the TOD warm-up have no state (warm-up).
- On a 1m chart the 09:30–10:00 sum would need 30 buckets; the 30m chart makes it one bucket. The 15-minute window arm needs a 15m chart.

Full strategy — `_Ex04OpeningRvGate.Strategy.CS`:

```
// Manual v3 example 4 - A2-50 same-slot relative-volume opening gate (module). Computes RV30 once per
session at the 10:00 ET
// bar close and journals HIGH / MID / LOW. It places NO orders: hosts read Rv30State (A2-50 routes hosts, it
is not a strategy).
// Chart: Data1 = @ES 30m (bars end 10:00, 10:30, ...), dxFeed continuous UNADJUSTED, 24h CME Globex
template. A 30m chart is used
// because a 30m TOD bucket on a 1m chart groups bar ENDS 09:31-10:00 into two buckets (09:31-09:59 and
10:00-10:29).
using System;
using System.Collections.Generic;
using System.Linq;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex04OpeningRvGate : _CustomSignal
    {
        public enum ERvState { Unknown = 0, Low = 1, Mid = 2, High = 3 }

        protected sealed class A250Config
        {
            public TimeSpan slotEnd = new TimeSpan(10, 0, 0);   // ET close-stamp of the 09:30-10:00 bar
            public int ssbSessions = 20;                         // SSB(N = 20)
            public int pctSessions = 250;                        // trailing percentile window
            public double qHi = 0.70, qLo = 0.30;
        }
        protected A250Config Cfg { get; } = new A250Config();

        /// <summary>Today's gate state (Unknown before 10:00 ET, on warm-up, half-days and holidays).</
summary>
        public ERvState Rv30State { get; private set; }
        public double Rv30 { get; private set; }

        private _CmeSessionClock m_clock;
        private readonly List<double> m_history = new List<double>();   // prior sessions' RV30, oldest first
        private DateTime m_day = DateTime.MinValue;

        public _Ex04OpeningRvGate(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }
        /// <summary>Half-days are kept out of the same-slot pools (M-TOD) and out of the gate.</summary>
        protected override bool todHalfDay(DateTime sessionDate) { return _CmeHolidays.Rules.Get(sessionDate)
.RthCloseEt.HasValue; }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            EventConfig.enabled =
true;               // only to tag 08:30-report days (reported as a separate stratum)
            EventConfig.classes = EEventClassMask.Macro;
        }

        protected override void StartCalc()
        {
```

```
            TodMetrics = ETodMetric.Volume;
            TodSessions = Cfg.ssbSessions;
            TodMinSessions = Cfg.ssbSessions;        // no SSB until 20 completed sessions
            TodBucketMinutes = 0;                    // 0 = Data1 bar size (30)
            m_clock = null;
            base.StartCalc();
            m_history.Clear(); m_day = DateTime.MinValue; Rv30State = ERvState.Unknown; Rv30 = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();                          // steps M-TOD first: the pool below never holds today
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], day = tradingDayKey(t);
            if (day != m_day) { m_day = day; Rv30State = ERvState.Unknown; Rv30 = double.NaN; }
            if (Clock.ToEastern(t).TimeOfDay != Cfg.slotEnd) return;
            if (Bars.Info.Resolution.Size != 30) throw new ArgumentException("_Ex04OpeningRvGate: Data1 must
be 30-minute bars");

            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            if (cme.NoRth || cme.RthCloseEt.HasValue) { journal(day, "skip half-day/no-RTH", double.NaN, doub
le.NaN); return; }

            IList<double> pool = TodNormaliser.VolumePool(todMinuteOfSession(t));
            if (pool == null || pool.Count < Cfg.ssbSessions) { journal(day, "SSB warm-up", double.NaN, doubl
e.NaN); return; }
            double ssb = pool.Skip(pool.Count - Cfg.ssbSessions).Average();   // mean of the last 20
completed sessions' 09:30-10:00 volume
            if (!(ssb > 0)) return;
            double rv = Bars.Volume[0] / ssb;
            Rv30 = rv;

            if (m_history.Count >= Cfg.pctSessions)
            {
                var w = m_history.Skip(m_history.Count - Cfg.pctSessions).OrderBy(x => x).ToList();   //
prior 250 only (causal)
                double hi = nearestRank(w, Cfg.qHi), lo = nearestRank(w, Cfg.qLo);
                Rv30State = rv >= hi ? ERvState.High : rv <= lo ? ERvState.Low : ERvState.Mid;
                journal(day, Rv30State.ToString(), hi, lo);
            }
            else journal(day, "percentile warm-up " + m_history.Count + "/" + Cfg.pctSessions, double.NaN, do
uble.NaN);
            m_history.Add(rv);
            if (m_history.Count > Cfg.pctSessions + 5) m_history.RemoveAt(0);
        }

        /// <summary>Nearest-rank quantile of a sorted list (the catalogue does not fix the quantile method:
an assumption).</summary>
        private static double nearestRank(List<double> sorted, double q)
        {
            int k = (int)Math.Ceiling(q * sorted.Count) - 1;
            return sorted[Math.Max(0, Math.Min(sorted.Count - 1, k))];
        }

        private void journal(DateTime day, string state, double hi, double lo)
        {
            if (!journalOn()) return;
            string rep = "n/a";
            _EventCalendar ev = EventCalendar;
            if (ev != null && !ev.Disabled)
            {
                EEventKind hole;
                if (ev.CoverageBlocks(day, EEventClassMask.Macro, EEventTier.None, out hole)) rep = "unknown(
" + hole + ")";
                else rep = ev.EventsOn(day, EEventClassMask.Macro).Any(r => r.etTime.TimeOfDay ==
EventRules.T0830) ? "yes" : "no";
            }
            trace(ETraceLevel.Journal, $"A2-50 {day:yyyy-MM-dd} RV30({Rv30:F3}) qHi({hi:F3}) qLo({lo:F3})
state({state}) report0830({rep})");
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.5 Example 5 (`_Ex05TwoTimeframeTurn`) — A2-57 60m turn + first 12m oversold (medium)

Catalogue rules (A2-57, base configuration). Trend: EMA(13) of 60m closes (closed bars, 24h). Up-turn = the EMA slope changes from ≤ 0 to > 0. Oscillator: raw %K(3) on 12m. Entry: the **first** 12m bar after the up-turn with %K(3) ≤ 20, then a buy stop at that bar's high + 1 tick, valid for the next two 12m bars; RTH 09:45–15:00 only; one trade per up-turn. Stop: the lower of the signal-bar low and the prior 12m low − 1 tick; break-even at +1R; then trail to protect 50% of the peak open profit. Exit: the 60m slope turns ≤ 0, the trail, or 15:55.

Chart. Data1 = @ES 1m, Data2 = @ES 12m, Data3 = @ES 60m (`HtfStreams = {2, 3}`), dxFeed unadjusted, 24h Globex template.

Walk-through.

- Each new closed 60m bar updates the EMA and its slope. An up-turn arms the "first oversold" search.
- Each new closed 12m bar computes %K(3). The first one at ≤ 20 inside 09:45–15:00 becomes the signal bar. An oversold bar outside that window **forfeits** the up-turn.
- The buy stop is sent on every 1m bar until two 12m bars have closed after the signal bar.
- The structural stop is min(signal-bar low, prior 12m low) − 1 tick. If trigger − stop is under the 14-tick cost floor (`costFloorTicks`), the signal is skipped and journaled, and that up-turn is used up.
- In the trade, the stop moves to break-even at +1R, then to entry + 50% of the peak open profit. The position exits on a 60m slope ≤ 0 or at 15:55 (`RthCloseEt` − 5 min on an early-close day, `flatAt`).
- RollDetection is Series. `rollOn(1)` clears a pending trigger, so a buy stop never spans the splice. It also **re-seeds the 60m EMA**: `m_ema`, `m_slope`, `m_ema60Count` and `m_up` are reset and a journal line is written. No slope turn counts until 3 × 13 new closed 60m bars, so the roll gap can't make a false turn.

Pitfalls.

- **Assumptions (labelled):** the EMA warm-up (3 × 13 closed 60m bars before the first turn counts), and the reading that an oversold bar outside 09:45–15:00 uses up "the first OS".
- The protective stop is sent from the bar after the fill. On a 1m chart the fill bar itself is unprotected for that minute.
- **Rolls.** Without the re-seed, the first EMA updates on the new contract include the roll gap, and an up-turn in the slope can come from the splice alone. **Coding choice (labelled):** re-seed rather than ignore turns for 3 × 13 bars. The two differ only in whether the warm-up uses old-contract closes. During the warm-up the slope exit is off too, which doesn't matter while the position is flat by 15:55 before a session-start splice.
- Run MC Probe P2 (HTF bar-end stamping) first, as the catalogue says (§4.12 VERIFY).

Full strategy — `_Ex05TwoTimeframeTurn.Strategy.CS`:

```
// Manual v3 example 5 - A2-57 60m EMA(13) slope up-turn + FIRST 12m %K(3) <= 20, buy stop above the signal
bar (long arm, base config).
// Chart: Data1 = @ES 1m, Data2 = @ES 12m, Data3 = @ES 60m (same symbol and 24h Globex template), dxFeed
continuous UNADJUSTED.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex05TwoTimeframeTurn : _CustomSignal
    {
        protected sealed class A257Config
        {
            public int contracts = 1;
            public int emaLength = 13, stochLength = 3, validBars = 2;
            public double osLevel = 20;
```

```
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 0, 0), flat = new TimeSpan(1
5, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);   // early-close days: flat at
RthCloseEt - 5 min
            public int costFloorTicks = 14;  // catalogue 2.1: ES cost floor; a structural stop under it is
skipped and logged, not resized
            public double trailKeep = 0.5;   // after +1R: protect 50% of the peak open profit
        }
        protected A257Config Cfg { get; } = new A257Config();

        private IOrderPriced m_leStp, m_lxStp;
        private IOrderMarket m_lx;
        private _CmeSessionClock m_clock;
        private DateTime m_last12 = DateTime.MinValue, m_last60 = DateTime.MinValue;
        private double m_ema = double.NaN, m_slope = double.NaN;
        private int m_ema60Count;
        private bool m_up, m_osTaken;          // 60m state; the first OS after this up-turn was already seen
        private double m_trigger = double.NaN, m_planStop = double.NaN; private int m_validLeft;
        private double m_stop = double.NaN, m_entry = double.NaN, m_risk = double.NaN, m_peak = double.NaN;
        private bool m_slopeDown;

        public _Ex05TwoTimeframeTurn(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            RollDetection =
ERollDetection.Series;   // unadjusted series: rollOn(1) below needs detection on (default is Off)
            m_leStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "A257LE",
EOrderAction.Buy));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A257LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A257LX",
EOrderAction.Sell));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2, 3 };
            m_clock = null;
            base.StartCalc();
            m_last12 = m_last60 = DateTime.MinValue; m_ema = m_slope = double.NaN; m_ema60Count = 0;
            m_up = m_osTaken = m_slopeDown = false; m_trigger = m_planStop = double.NaN; m_validLeft = 0;
            m_stop = m_entry = m_risk = m_peak = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            TimeSpan tod = Clock.ToEastern(Bars.Time[0]).TimeOfDay;
            TimeSpan flat = flatAt(tradingDayKey(Bars.Time[0]));
            bool rthWindow = tod >= Cfg.from && tod < Cfg.to && tod < flat;

            if (rollOn(1))
            {
                m_trigger = double.NaN; m_validLeft = 0;   // never carry a level across the splice
                // re-seed the 60m EMA from the new contract: no slope turn counts until 3 x emaLength new
closed 60m bars
                m_ema = m_slope = double.NaN; m_ema60Count = 0; m_up = false;
                if (journalOn()) trace(ETraceLevel.Journal,
$"A2-57 roll {Bars.Time[0]:yyyy-MM-dd HH:mm}: 60m EMA re-seeded");
            }
            step60();
            bool new12 = step12(rthWindow);

            int pos = StrategyInfo.MarketPosition;
            if (pos > 0) { manage(tod, flat); return; }
            m_stop = m_entry = m_risk = m_peak = double.NaN;
            if (new12 && m_validLeft > 0) m_validLeft--;                    // "valid for the next 2 12m
bars"
```

```
            if (m_validLeft > 0 && rthWindow && m_up && !double.IsNaN(m_trigger)) m_leStp.Send(m_trigger,
Cfg.contracts);
        }

        /// <summary>New CLOSED 60m bar: update EMA(13) and the slope state.</summary>
        private void step60()
        {
            DateTime t; double o, h, l, c, v;
            HtfClosedBar(3, 0, out t, out o, out h, out l, out c, out v);
            if (t == m_last60) return;
            m_last60 = t;
            double prevEma = m_ema, prevSlope = m_slope, a = 2.0 / (Cfg.emaLength + 1);
            m_ema = double.IsNaN(m_ema) ? c : m_ema + a * (c - m_ema);
            m_ema60Count++;
            if (double.IsNaN(prevEma) || m_ema60Count < 3 * Cfg.emaLength) return;   // EMA warm-up (3 x
length: an assumption)
            m_slope = m_ema - prevEma;
            bool turnUp = !double.IsNaN(prevSlope) && prevSlope <= 0 && m_slope > 0;
            if (turnUp) { m_up = true; m_osTaken = false; if (journalOn()) trace(ETraceLevel.Journal,
$"A2-57 60m up-turn {t:yyyy-MM-dd HH:mm} ema({m_ema:F2})"); }
            if (m_slope <= 0) { m_up = false; m_slopeDown = true; m_trigger = double.NaN; m_validLeft = 0; }
else m_slopeDown = false;
        }

        /// <summary>New CLOSED 12m bar: the first %K(3) &lt;= 20 after the up-turn arms the buy stop.
Returns true on a new 12m bar.</summary>
        private bool step12(bool rthWindow)
        {
            DateTime t; double o, h, l, c, v;
            HtfClosedBar(2, 0, out t, out o, out h, out l, out c, out v);
            if (t == m_last12) return false;
            m_last12 = t;
            if (!m_up || m_osTaken || BarsOfData(2).CurrentBar <= Cfg.stochLength + 1) return true;
            double hh = double.MinValue, ll = double.MaxValue, prevLow = double.NaN;
            for (int k = 0; k < Cfg.stochLength; k++)
            {
                DateTime tk; double ok, hk, lk, ck, vk;
                HtfClosedBar(2, k, out tk, out ok, out hk, out lk, out ck, out vk);
                hh = Math.Max(hh, hk); ll = Math.Min(ll, lk);
                if (k == 1) prevLow = lk;
            }
            double pctK = hh > ll ? 100.0 * (c - ll) / (hh - ll) : 50.0;
            if (pctK > Cfg.osLevel) return true;
            m_osTaken = true;                       // FIRST oversold only: later ones are ignored (the key
rule)
            if (!rthWindow) { if (journalOn()) trace(ETraceLevel.Journal, $"A2-57 first OS {t:HH:mm} outside
09:45-15:00: turn forfeited"); return true; }
            double tick = tickSize();
            double trigger = h + tick, planStop = Math.Min(l, double.IsNaN(prevLow) ? l : prevLow) - tick;
            if (trigger - planStop < Cfg.costFloorTicks *
tick)   // under the cost floor: skipped and logged (this up-turn is used up)
            {
                if (journalOn()) trace(ETraceLevel.Journal,
$"A2-57 {t:yyyy-MM-dd HH:mm} skip: stop {(trigger - planStop) / tick:F0} ticks < cost floor
{Cfg.costFloorTicks}");
                return true;
            }
            m_trigger = trigger; m_planStop = planStop;
            m_validLeft = Cfg.validBars + 1;        // decremented on this bar's new12 pass, leaving 2 full
12m bars
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "pctK", pctK.ToString("F1") }, { "ema60",
m_ema.ToString("F2") } };
                journalArm("A257LE", m_trigger, m_planStop, double.NaN, ctx);
                logSignal("A2-57", +1, m_trigger, m_planStop, ctx, true);
            }
            return true;
        }

        private void manage(TimeSpan tod, TimeSpan flat)
        {
            if (double.IsNaN(m_entry))
            {
                m_entry = StrategyInfo.AvgEntryPrice; m_stop = m_planStop; m_risk = m_entry - m_stop; m_peak
= m_entry;
                m_trigger = double.NaN; m_validLeft = 0;   // one trade per up-turn
```

```
            }
            if ((tod >= flat && tod < new TimeSpan(17, 0, 0)) || m_slopeDown) { m_lx.Send(); return; }
            m_peak = Math.Max(m_peak, Bars.High[0]);
            if (m_risk > 0 && m_peak - m_entry >= m_risk)
            {
                double trail = m_entry + Cfg.trailKeep * (m_peak - m_entry);   // >= break-even once +1R is
reached
                m_stop = Math.Max(m_stop, Math.Max(m_entry, trail));
            }
            m_lxStp.Send(m_stop);
        }

        /// <summary>Flat time for a Globex day: Cfg.flat, or on a CME early-close day RthCloseEt -
halfDayExitBefore if earlier
        /// (_CmeHolidays.Rules.Get(day).RthCloseEt; same rule as A263Config.halfDayExitBefore).</summary>
        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.6 Example 6 (`_Ex09OpenAtrBreakout`) — A2-63 open + k × ATR20d breakout with RegimeGate (medium)

Catalogue rules (A2-63, core). Buy stop at O + k × ATR20d, O = the 09:30 ET RTH open; active 09:30–15:30; one entry per day; exit 15:55. Core: k 0.5, 30m bars, ATR 20. The G-R consumer `_RegimeGate` (C1) adds arms U (ungated), G and A, pre-registered in `specs/regime-consumer-prereg.md`; its test has **not** been run.

Chart. Data1 = @ES 30m (the book's size) or 1m, dxFeed unadjusted, 24h Globex template. `[Input] GateArm` 0 = U, 1 = G, 2 = A.

Walk-through. This is `_Test6` cut down to show the API; `_Test6` stays the pre-registered harness.

- `StartCalc` validates the arm, then calls `RegimeGate.ApplyArm(GateCfg, RegimeConfig)` **before** `base.StartCalc()`, because the engines are built there. It sets `NeedsDaily` and `DailyConfig.atrLength`.
- `AtrTercileTracker` gets one value per completed day (`daily.PrevDate`, `daily.Atr20d`).
- On the latch bar (`A263Core.IsLatchBar`, the first bar closing after 09:30 ET) it reads ATR and as-of date, asks `RegimeGate.Decide(arm, Regime, day, tercile)` and latches the open, threshold and gate decision in `A263Core.Latch`.
- With TraceLevel Journal it writes `RegimeGate.Line(...)` plus the core's suffix, and `journalArm` with `RegimeGate.TradeFields` when an allowed signal exists.
- `A263Core.Step` returns the stop price to send or the exit.

Pitfalls.

- Like `_Test6`, this example is a plain `_CustomSignal` that converts time with `ToZone(..., "Eastern Standard Time")` and uses calendar dates. It does not override `tradingDayKey`, so M-DAILY days are calendar days on a Globex chart (**VERIFY**, §4.2 and Appendix F).
- RollDetection is Off, as in `_Test6`: the roll gap enters the daily ATR. With it on, `Atr20d` is NaN for 20 sessions after each roll (§4.3). The catalogue's "Atr20d, roll-gap removed" does not match either case exactly; see the discrepancies in Appendix F.
- Change the gate config only before `base.StartCalc()`; the hash (`RegimeGate.Hash`) records it.

- **Not coded:** the host H2 "disaster-stop-at-the-open" arm named in A2-63's gate line (a disaster stop at O). `A263Config` has no disaster stop, so the core has no stop and the cost floor doesn't apply.

Full strategy — `_Ex09OpenAtrBreakout.Strategy.CS`:

```
// Manual v3 example 6 - A2-63 long-only open + k x ATR20d breakout, exit 15:55 ET, with the _RegimeGate arm
switch (G-R consumer C1).
// Chart: Data1 = @ES 30m (book) or 1m, dxFeed continuous UNADJUSTED, 24h CME Globex template.
// The pre-registered harness is _Test6; this example is the same wiring cut down to show the API (A263Core
does the rules).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex09OpenAtrBreakout : _CustomSignal
    {
        [Input] public int GateArm { get; set; }      // 0 = U (ungated), 1 = G, 2 = A (see RegimeGate pre-
registration)

        protected RegimeGateConfig GateCfg { get; } = new RegimeGateConfig();
        protected A263Config HostCfg { get; } = new A263Config();   // k 0.5, ATR 20, 09:30-15:30 active,
exit 15:55

        private IOrderPriced m_le; private IOrderMarket m_lx;
        private A263Core m_core; private AtrTercileTracker m_terc;
        private string m_hash;

        public _Ex09OpenAtrBreakout(object _ctx) : base(_ctx) { GateArm = 0; }

        protected override void Create()
        {
            base.Create();
            m_le = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "A263LE",
EOrderAction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A263LX",
EOrderAction.Sell));
        }

        protected override void StartCalc()
        {
            if (GateArm < 0 || GateArm > 2) throw new ArgumentException("GateArm must be 0, 1 or 2");
            GateCfg.arm = (ERegimeGateArm)GateArm;
            string err = GateCfg.Validate(); if (err != null) throw new ArgumentException(err);
            RegimeGate.ApplyArm(GateCfg, RegimeConfig);   // BEFORE base.StartCalc: the engines are built
there
            NeedsDaily = true;                            // M-DAILY, lazily created by the base
            DailyConfig.atrLength = HostCfg.atrLength;
            base.StartCalc();
            m_core = new A263Core(HostCfg);
            m_terc = new AtrTercileTracker(RegimeConfig.atrPctLookback, RegimeConfig.K);
            m_hash = RegimeGate.Hash(RegimeConfig, GateCfg, HostCfg);
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime et = ToZone(Bars.Time[0], "Eastern Standard Time");
            _DailyContext daily = DailyContext;
            if (daily != null) m_terc.AddDay(daily.PrevDate, daily.Atr20d);
            bool isLong = StrategyInfo.MarketPosition > 0;

            if (m_core.IsLatchBar(et))   // first bar closing after 09:30 ET: O = its open
            {
                double atr = daily != null ? daily.Atr20d :
double.NaN;      // RollDetection is Off here (as in _Test6): roll gaps enter the ATR; with it on, NaN for
atrLength sessions after a roll
                DateTime asOf = daily != null ? daily.PrevDate : DateTime.MinValue;
                _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(et.Date);
                RegimeGateDecision d = RegimeGate.Decide(GateCfg.arm, Regime, et.Date, m_terc.LastDate == asO
f ? m_terc.Tercile : -1);
                m_core.Latch(et, Bars.Open[0], atr, asOf, d.allow, cme.RthCloseEt, cme.NoRth);
```

```
                if (journalOn())
                {
                    trace(ETraceLevel.Journal, RegimeGate.Line(d, m_core.HasSignal, m_hash, GateCfg.placeboSe
ed) + m_core.Suffix());
                    if (d.allow && m_core.HasSignal) journalArm("A263LE", m_core.Threshold, double.NaN, doubl
e.NaN, RegimeGate.TradeFields(d, m_hash, GateCfg.placeboSeed));
                }
            }

            A263Action a = m_core.Step(et, isLong);
            if (a.exit) m_lx.Send();
            else if (a.sendStop) m_le.Send(a.stop, HostCfg.contracts);
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.7 Example 7 (`_Ex06NakedLevelReaction`) — PS-21 naked POC/VAH/VAL first touch (medium–hard)

Catalogue rules (PS-21). Market and timeframe: ES/NQ, GC, CL; 5m. Level: the nearest naked POC/VAH/VAL ≤ 20 sessions old. Entry: first touch ± δ, then a 5m rejection close. Stop: level + floor. Target: 1R. Sweep: age {≤ 5, ≤ 20} × level {POC, VAH/VAL}. Multi-day levels reset at rolls. The catalogue expects a kill and keeps it as a control for EX-02. **G0-d-gated** (families 1 and 2).

Chart. Data1 = @ES 1m with the MC Volume Profile **on** (session profiles), Data2 = @ES 5m, dxFeed unadjusted, 24h. Derives from `_VolumeProfileFuturesBase`, `TrackNakedPoc = true` (StartCalc throws otherwise).

Walk-through.

- The level search uses `RecentCompletedProfiles(1, maxAge, barNo - 1)`: profiles completed **as of the previous bar**. `barNo` is `Bars.FullSymbolData.Current`, the 1-based Status Bar number that `asOfBar` expects (not `Bars.CurrentBar`).
- Profiles that cross a roll or start at or before the last roll bar are skipped.
- Naked tests: `VpExtra.IsNakedPoc(asOfBar)` for POCs, `IsNakedVah`/`IsNakedVal` for value edges. A price already traded once is kept in a set and never used again (first touch only).
- Side: price above the level means support (long), below means resistance (short). The nearest candidate to the prior close wins.
- A touch is the 1m low (or high) within `touchTicks` of the level. The decision waits for the first **closed** 5m bar at or after the touch: a close back on the entry side is a rejection, and the trade is armed at market; anything else is discarded and logged as untaken.
- Stop = level − side × 4 ticks (`stopBufferTicks`); target = 1R from the 1m close of the decision bar. If that R is under the 14-tick cost floor (`costFloorTicks`), the trade is skipped and journaled. Short entries mirror longs. Entries until 15:30; flat at 15:55, or at `RthCloseEt` − 5 min on an early-close day (`flatAt`).

Pitfalls.

- **Assumption (labelled):** the 4-tick buffer beyond the level. With the 14-tick cost floor, a trade is only taken when the 1m close is at least 10 ticks from the level.
- **First touch uses up the level.** The level goes into `m_used` at the touch, so a touch that is discarded (no rejection) or skipped (under the cost floor) uses up that level too.
- Stop and target are sent from the bar after the fill (§1.7), so the fill bar has no working exit.
- `asOfBar` is the Status Bar number. Passing `Bars.CurrentBar` silently shifts the as-of point once MaxBarsBack is not 0.
- If the touch bar also closes the 5m bar that holds it, the rejection is checked on the same bar.
- PS-21's K-FL control with distance matching is mandatory (catalogue); it is not coded here.

Full strategy — `_Ex06NakedLevelReaction.Strategy.CS`:

```
// Manual v3 example 7 - PS-21 first touch of the nearest naked POC / VAH / VAL (<= 20 sessions) + 5m
rejection close, target 1R.
// Chart: Data1 = @ES 1m with the MultiCharts Volume Profile ON (session profiles), Data2 = @ES 5m, dxFeed
continuous UNADJUSTED, 24h.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex06NakedLevelReaction : _VolumeProfileFuturesBase
    {
        public enum ELevelSet { Poc = 1, ValueEdges = 2 }

        protected sealed class Ps21Config
        {
            public int contracts = 1;
            public int maxAgeSessions = 20;          // sweep {5, 20}
            public ELevelSet levels = ELevelSet.Poc; // sweep {POC, VAH/VAL}
            public int touchTicks = 1;               // delta of "first touch +/- delta"
            public int stopBufferTicks = 4;          // structural stop: this far beyond the level (an
assumption of this example)
            public int costFloorTicks = 14;          // catalogue 2.1: ES cost floor; a stop under it is
skipped and logged, not resized
            public TimeSpan from = new TimeSpan(9, 30, 0), to = new TimeSpan(15, 30, 0), flat = new
TimeSpan(15, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);   // early-close days: flat at
RthCloseEt - 5 min
        }
        protected Ps21Config Cfg { get; } = new Ps21Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private int m_lastRollBar;                       // 1-based Status Bar # of the last roll bar
        private double m_level = double.NaN; private int m_side; private DateTime m_touch5 = DateTime.MinValu
e, m_last5 = DateTime.MinValue;
        private readonly HashSet<double> m_used = new HashSet<double>();   // first touch only: a level is
never traded twice
        private double m_stop = double.NaN, m_target = double.NaN;

        public _Ex06NakedLevelReaction(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            TrackNakedPoc = true;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS21LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS21SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS21LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS21SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS21LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS21SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS21LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS21ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2 };
            base.StartCalc();
            if (!TrackNakedPoc) throw new ArgumentException("_Ex06NakedLevelReaction: TrackNakedPoc must be
true");
            m_lastRollBar = 0; m_level = double.NaN; m_side = 0; m_touch5 = m_last5 = DateTime.MinValue; m_us
ed.Clear();
            m_stop = m_target = double.NaN;
```

```
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            int barNo = Bars.FullSymbolData.Current;           // 1-based Status Bar #: the unit of asOfBar
(NOT Bars.CurrentBar)
            if (rollOn(1)) { m_lastRollBar = barNo; m_level = double.NaN; m_side = 0; }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            TimeSpan flat = flatAt(GlobexDate(Bars.Time[0]));
            TimeSpan entryEnd = Cfg.to < flat ? Cfg.to : flat;
            double tick = tickSize();
            int pos = StrategyInfo.MarketPosition;

            if (pos != 0)
            {
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else m_sx.Send()
; return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }

            // a. pending touch: wait for the first 5m bar that CLOSES after the touch (closed 5m bars only)
            DateTime t5; double o5, h5, l5, c5, v5;
            HtfClosedBar(2, 0, out t5, out o5, out h5, out l5, out c5, out v5);
            bool new5 = t5 != m_last5; m_last5 = t5;
            if (m_side != 0 && new5 && t5 >= m_touch5) { checkReject(c5, tod, entryEnd, tick); return; }
            if (m_side != 0 || tod < Cfg.from || tod >= entryEnd) return;

            // b. levels naked as of the PREVIOUS bar, from the last maxAgeSessions completed profiles, none
older than the last roll
            double best = double.NaN; int side = 0; double c1 = Bars.Close[1];
            foreach (VpExtra e in RecentCompletedProfiles(1, Cfg.maxAgeSessions, barNo - 1))
            {
                if (e.crossesRoll || e.startBar <= m_lastRollBar) continue;   // multi-day levels reset at
rolls
                var cand = new List<double>();
                if (Cfg.levels == ELevelSet.Poc) { if (e.IsNakedPoc(barNo - 1)) cand.Add(e.poc); }
                else { if (IsNakedVah(e, barNo - 1)) cand.Add(e.vah); if (IsNakedVal(e, barNo - 1))
cand.Add(e.val); }
                foreach (double lv in cand)
                {
                    if (m_used.Contains(lv)) continue;
                    int s = c1 > lv ? +1 : c1 < lv ? -1 : 0;     // approached from above = support (long);
from below = resistance (short)
                    if (s == 0) continue;
                    if (double.IsNaN(best) || Math.Abs(c1 - lv) < Math.Abs(c1 - best)) { best = lv; side =
s; }
                }
            }
            if (double.IsNaN(best)) return;
            bool touched = side > 0 ? Bars.Low[0] <= best + Cfg.touchTicks * tick : Bars.High[0] >= best - Cf
g.touchTicks * tick;
            if (!touched) return;
            m_used.Add(best); m_level = best; m_side = side; m_touch5 = Bars.Time[0];
            if (journalOn()) trace(ETraceLevel.Journal, $"PS-21 touch {Bars.Time[0]:yyyy-MM-dd HH:mm}
level({best}) side({side})");
            if (new5 && t5 >= m_touch5) checkReject(c5, tod, entryEnd, tick);   // the touch bar also closed
the 5m bar that holds it
        }

        /// <summary>The first 5m close at or after the touch decides: back on the entry side of the level =
rejection, else discard.
        /// Either way the level stays in m_used: a discarded touch uses the level up for good.</summary>
        private void checkReject(double c5, TimeSpan tod, TimeSpan entryEnd, double tick)
        {
            bool reject = m_side > 0 ? c5 > m_level : c5 < m_level;
            if (reject && tod < entryEnd) arm(m_side, Bars.Close[0], tick);
            else if (journalOn()) logSignal("PS-21", m_side, m_level, double.NaN, null, false);
            m_side = 0; m_level = double.NaN;
        }

        private void arm(int side, double planned, double tick)
        {
            double stop = m_level - side * Cfg.stopBufferTicks * tick;
```

```
            double r = Math.Abs(planned - stop);
            if (r < Cfg.costFloorTicks * tick)   // under the cost floor: skipped and logged, not resized
            {
                if (journalOn()) trace(ETraceLevel.Journal, $"PS-21 {Bars.Time[0]:yyyy-MM-dd HH:mm} skip:
stop {r / tick:F0} ticks < cost floor {Cfg.costFloorTicks}");
                return;
            }
            m_stop = stop;
            m_target = planned + side * r;                      // 1R from the planned (signal-close) price
            var ctx = new Dictionary<string, string> { { "level", m_level.ToString() } };
            if (journalOn()) { journalArm(side > 0 ? "PS21LE" : "PS21SE", planned, m_stop, m_target, ctx); lo
gSignal("PS-21", side, planned, m_stop, ctx, true); }
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
        }

        /// <summary>Flat time for a Globex day: Cfg.flat, or on a CME early-close day RthCloseEt -
halfDayExitBefore if earlier
        /// (_CmeHolidays.Rules.Get(day).RthCloseEt; same rule as A263Config.halfDayExitBefore).</summary>
        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.8 Example 8 (`_Ex08EntropyDayType`) — A2-29 entropy day type at 11:00 (medium–hard)

Catalogue rules (A2-29). At 11:00 ET, take the developing RTH profile 09:30–11:00 and its H_norm and n_eff as percentiles versus the same-time values of the trailing 250 sessions. H_norm pct ≥ 80 → trend/dispersed; H_norm pct ≤ 30 **and** n_eff pct ≤ 30 → balance; otherwise no change. It is a classifier (module), so no orders.

Chart. Data1 = @ES 1m, dxFeed unadjusted, 24h Globex template. `RangeProfiles.enabled = true`.

Walk-through.

- At the bar closing at 11:00 ET (skipping NoRth days), `RangeProfileAfter(1, t0, t)` builds the profile of 1m bars ending in `(09:30, 11:00]`: an exclusive start, end-stamped bars.
- `RangeStatsOf(e)` gives the bar count; `ProfileEntropy(e, st.bars)` gives the Miller–Madow `HnormMm` and `Neff`, with N = contributing bars (the VP-4 default).
- Two `_EntropyTracker`s (lookback `ProfileStats.pctLookbackA229` = 250, `pctMinFill` = 0.8) return today's percentile against **earlier** sessions first, and only then push today's value.
- The state (TREND, BALANCE, NEUTRAL, or warm-up/no profile) is exposed as `DayType` and journaled with the raw values.

Pitfalls.

- Percentile first, push second. Doing it the other way includes today in its own baseline.
- `t0 = t − 90 minutes` in chart time; the range is in chart time, not ET.
- Until 80% of 250 sessions are filled, the percentiles are NaN and the line says warm-up.
- **Half-days are in the pool.** Only NoRth days are skipped, so early-close sessions add their 11:00 values to the 250-session baselines (on a half-day, 09:30–11:00 is still complete). M-TOD excludes half-days elsewhere in this manual (`todHalfDay`, §1.7); this example does not.

Full strategy — `_Ex08EntropyDayType.Strategy.CS`:

```
// Manual v3 example 8 - A2-29 developing-profile entropy day type at 11:00 ET (module: journals TREND / BALANCE / NEUTRAL, no orders).
// Chart: Data1 = @ES 1m, dxFeed continuous UNADJUSTED, 24h CME Globex template. Range profiles ON.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex08EntropyDayType : _VolumeProfileFuturesBase
    {
        public enum EDayType { Unknown = 0, Neutral = 1, Trend = 2, Balance = 3 }

        protected sealed class A229Config
        {
            public TimeSpan rthOpen = new TimeSpan(9, 30, 0), decision = new TimeSpan(11, 0, 0);
            public double trendPct = 80, balancePct = 30;
        }
        protected A229Config Cfg { get; } = new A229Config();

        public EDayType DayType { get; private set; }
        private _EntropyTracker m_hnorm, m_neff;
        private DateTime m_day = DateTime.MinValue;
        private int m_slot;

        public _Ex08EntropyDayType(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            RangeProfiles.enabled = true;
        }

        protected override void StartCalc()
        {
            base.StartCalc();
            m_slot = Cfg.decision.Hours * 60 + Cfg.decision.Minutes;   // = _EntropyTracker.Slot(11:00) = 660
            m_hnorm = new _EntropyTracker(ProfileStats.pctLookbackA229, ProfileStats.pctMinFill, new[] { m_slot });
            m_neff = new _EntropyTracker(ProfileStats.pctLookbackA229, ProfileStats.pctMinFill, new[] { m_slot });
            m_day = DateTime.MinValue; DayType = EDayType.Unknown;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], et = ToEastern(t), day = GlobexDate(t);
            if (day != m_day) { m_day = day; DayType = EDayType.Unknown; }
            if (et.TimeOfDay != Cfg.decision) return;
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            if (cme.NoRth) return;

            // developing RTH profile 09:30-11:00: 1m bars ENDING in (09:30, 11:00] -> exclusive t0, in chart time (t is the 11:00 bar end)
            DateTime t0 = t - (Cfg.decision - Cfg.rthOpen);
            VpExtra e = RangeProfileAfter(1, t0, t);
            RangeStats st = RangeStatsOf(e);
            if (e == null || st == null || st.bars < 1) { journal(day, "no profile", st); return; }
            EntropyResult r = ProfileEntropy(e, st.bars);   // Miller-Madow N = contributing bars (VP-4 default basis)

            // percentiles vs the prior 250 sessions FIRST, then push today (strictly earlier sessions only)
            double pH = m_hnorm.Percentile(m_slot, r.HnormMm), pN = m_neff.Percentile(m_slot, r.Neff);
            m_hnorm.Push(m_slot, day, r.HnormMm); m_neff.Push(m_slot, day, r.Neff);
            if (double.IsNaN(pH) || double.IsNaN(pN)) { journal(day, "warm-up", st, r, pH, pN); return; }
            DayType = pH >= Cfg.trendPct ? EDayType.Trend : (pH <= Cfg.balancePct && pN <= Cfg.balancePct) ? EDayType.Balance : EDayType.Neutral;
            journal(day, DayType.ToString(), st, r, pH, pN);
        }

        private void journal(DateTime day, string state, RangeStats st, EntropyResult r = default(EntropyResult), double pH = double.NaN, double pN = double.NaN)
        {
```

```
            if (!journalOn()) return;
            trace(ETraceLevel.Journal, $"A2-29 {day:yyyy-MM-dd} 11:00 HnormMm({r.HnormMm:F4})
Neff({r.Neff:F1}) pctH({pH:F1}) pctNeff({pN:F1}) state({state}) stats({st})");
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.9 Example 9 (`_Ex07RotationVolumeLevel`) — PS-13 / DV-03 rotation POC first-touch retest (hard)

Catalogue rules (PS-13). Filter: a rotation of ≥ 6 bars on 15m with range ≤ 1.5 × ATR15m, then a breakout bar closing outside with range ≥ 1.5 × ATR15m. Level: the highest-volume price in the rotation. Entry: first touch (limit) or first 5m HL after the touch; first touch only; level valid 5 sessions intraday. Stop: 10–20% of the 200-day ATR, floored. Target: before the next cluster. DV-03 (merged) adds a follow-through of ≥ 1 box width before any retest. **G0-d-gated** (family 10).

Chart. Data1 = @ES 1m, Data2 = @ES 15m, dxFeed unadjusted, 24h. `RangeProfiles.enabled = true`, RollDetection Series. This codes the **limit arm** of PS-13.

Walk-through.

- On each new closed 15m bar: ATR15 over the 14 bars before the breakout bar; the box = the high/low of the previous 6 closed 15m bars; reject if the box is taller than 1.5 × ATR15.
- The breakout bar must close outside the box with a range ≥ 1.5 × ATR15.
- `RangeProfileAfter(1, boxStart, boxEnd)` builds the 1m volume profile over the box only, and its POC is the level. `RangeStatsOf` must show coverage ≥ 0.95.
- Stop = level ∓ 0.10 × ATR200d; target = level ± 2R. If 0.10 × ATR200d is under the 14-tick cost floor (`costFloorTicks`), the level is skipped and journaled.
- **Entry window (catalogue §2.1):** the limit is sent only on bar closes from 09:45 to before 15:30 ET (and before an early-close flat time), so it can fill only between 09:45 and 15:30. A position is closed at market at 15:55, or at `RthCloseEt` − 5 min on an early-close day (`flatAt`).
- A buy (sell) limit rests at the level. If a later bar reaches the level and the position is still flat, the level is spent. Once filled, the level is cleared, so it is never traded twice. It also expires after 5 sessions and is cleared at a roll. A touch outside the entry window, with no limit resting (overnight, for example), also spends the level.
- `justArmed` is true only on the bar that armed the level, so that bar's own range can't count as the first touch.
- **ATR200d is computed in the strategy**, not by M-DAILY: per-Globex-day true range, with the roll day's gap dropped, simple average of 200 days. M-DAILY with `atrLength = 200` and RollDetection on would reseed at every roll (about every 63 sessions), so it would never be valid.

Pitfalls.

- **Assumptions (labelled):** ATR15 length 14; coverage 0.95; a fixed 2R target as a stand-in for "before the next cluster".
- **Not coded:** DV-03's follow-through condition, the 5m HL entry arm and the near-HVN-edge level variant.
- The box start is the open time of the oldest box bar. The profile is "after" that time, assuming bars are stamped at their end (Probe P2, **VERIFY**).
- A bar that touches the level without a fill (for example with MC's "fill only when price trades beyond the limit") spends the level.
- **Coding choice (labelled):** a touch outside 09:45–15:30 spends the level (first touch only). The other reading would keep it for the next session's window.
- Stop and target are sent from the bar after the fill (§1.7), so the fill bar has no working exit.

Full strategy — `_Ex07RotationVolumeLevel.Strategy.CS`:

```
// Manual v3 example 9 - PS-13 / DV-03 rotation -> breakout -> first touch of the rotation's highest-volume
price (limit arm).
// Chart: Data1 = @ES 1m, Data2 = @ES 15m (same symbol, 24h Globex), dxFeed continuous UNADJUSTED. Range
profiles ON (RangeProfiles.enabled).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex07RotationVolumeLevel : _VolumeProfileFuturesBase
    {
        protected sealed class Ps13Config
        {
            public int contracts = 1;
            public int rotationBars = 6, atr15Length = 14;   // ATR15m length is not in the catalogue: an
assumption
            public double boxAtr = 1.5, breakoutAtr = 1.5;
            public int atrDailyLength = 200;
            public double stopFraction = 0.10;                 // sweep {0.10, 0.20} x ATR200d
            public int costFloorTicks =
14;                      // catalogue 2.1: ES cost floor; a stop under it is skipped and logged, not resized
            public int validSessions = 5;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 30, 0), flat = new
TimeSpan(15, 55, 0);   // catalogue 2.1 intraday rules
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);   // early-close days: flat at
RthCloseEt - 5 min
            public double targetR = 2.0;                        // "before the next cluster" is not coded
here: a fixed-R stand-in (coding choice)
        }
        protected Ps13Config Cfg { get; } = new Ps13Config();

        private IOrderPriced m_leLmt, m_seLmt, m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private IOrderMarket m_lx, m_sx;
        private DateTime m_last15 = DateTime.MinValue;
        // own ATR200d: per-Globex-day TR with the roll day's TR dropped (M-DAILY reseeds its ATR at every
roll, see section 4)
        private readonly List<double> m_tr = new List<double>();
        private DateTime m_aggDay = DateTime.MinValue; private double m_dH, m_dL, m_dC = double.NaN, m_prevC
= double.NaN; private bool m_dRoll;
        private double m_level = double.NaN; private int m_side; private DateTime m_levelDay; private int m_l
evelSessions;
        private double m_stop = double.NaN, m_target = double.NaN;

        public _Ex07RotationVolumeLevel(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            RangeProfiles.enabled = true;              // M-RANGE: needs the bar ring (ringBars 2880 = two
days of 1m bars)
            m_leLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "PS13LE",
EOrderAction.Buy));
            m_seLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "PS13SE",
EOrderAction.SellShort));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS13LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS13SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS13LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS13ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS13LX",
EOrderAction.Sell, OrderExit.FromAll));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS13SX",
EOrderAction.BuyToCover, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2 };
            base.StartCalc();
            m_last15 = m_aggDay = DateTime.MinValue; m_tr.Clear(); m_dC = m_prevC = double.NaN; m_dRoll = fal
se;
```

```
            m_level = m_stop = m_target = double.NaN; m_side = 0; m_levelSessions = 0;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], day = GlobexDate(t);
            TimeSpan tod = ToEastern(t).TimeOfDay, flat = flatAt(day);
            bool roll = rollOn(1);
            stepDailyAtr(day, roll);
            if (roll) { m_level = double.NaN; m_side = 0; }       // multi-day rotations reset at rolls

            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_level = double.NaN; m_side = 0;                // filled: the level is spent (first touch
only, no re-entry)
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else m_sx.Send()
; return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }
            if (!double.IsNaN(m_level) && m_side != 0 && m_levelSessions > Cfg.validSessions) { m_level = dou
ble.NaN; m_side = 0; }

            step15();

            // first touch only: the resting limit IS the touch; once price reaches the level without a fill
the level is spent.
            // A touch outside the entry window (no limit resting) also spends it.
            if (!double.IsNaN(m_level) && m_side != 0)
            {
                bool traded = m_side > 0 ? Bars.Low[0] <= m_level : Bars.High[0] >= m_level;
                if (traded && Bars.CurrentBar > 1 && !justArmed) { m_level = double.NaN; m_side = 0;
return; }
                justArmed = false;
                // entries 09:45-15:30 ET only (and before an early-close flat time): the limit sent on this
close works on the next bar
                bool window = tod >= Cfg.from && tod < Cfg.to && tod < flat;
                if (window) { if (m_side > 0) m_leLmt.Send(m_level, Cfg.contracts); else
m_seLmt.Send(m_level, Cfg.contracts); }
            }
        }
        // true only on the bar that armed the level: that 1m bar belongs to the breakout, so its own range
must not count
        // as the first touch and use up the level it just created
        private bool justArmed;

        /// <summary>New CLOSED 15m bar: rotation of the previous rotationBars bars, breakout on this bar,
level = POC of the box.</summary>
        private void step15()
        {
            DateTime tb; double ob, hb, lb, cb, vb;
            HtfClosedBar(2, 0, out tb, out ob, out hb, out lb, out cb, out vb);
            if (tb == m_last15) return;
            m_last15 = tb;
            int n = Cfg.rotationBars;
            if (BarsOfData(2).CurrentBar <= n + Cfg.atr15Length + 2) return;
            double atr15 = atrClosed(2, Cfg.atr15Length, 1);   // ATR of the bars BEFORE the breakout bar
            double boxH = double.MinValue, boxL = double.MaxValue; DateTime boxStart = DateTime.MinValue, box
End = DateTime.MinValue;
            for (int k = 1; k <= n; k++)
            {
                DateTime tk; double ok, hk, lk, ck, vk;
                HtfClosedBar(2, k, out tk, out ok, out hk, out lk, out ck, out vk);
                boxH = Math.Max(boxH, hk); boxL = Math.Min(boxL, lk);
                if (k == 1) boxEnd = tk;
            }
            { DateTime tk; double ok, hk, lk, ck, vk; HtfClosedBar(2, n + 1, out tk, out ok, out hk, out lk,
out ck, out vk); boxStart = tk; }
            if (!(atr15 > 0) || boxH - boxL > Cfg.boxAtr * atr15) return;
            int side = cb > boxH ? +1 : cb < boxL ? -1 : 0;
            if (side == 0 || hb - lb < Cfg.breakoutAtr * atr15) return;
            // 1m volume-at-price over the box only: bars ending in (boxStart, boxEnd] (exclusive t0, end-
stamped bars, Probe P2)
```

```
            VpExtra e = RangeProfileAfter(1, boxStart, boxEnd);
            if (e == null || !(e.poc > 0)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-13 {tb:yyyy-MM-
dd HH:mm} no range profile"); return; }
            RangeStats st = RangeStatsOf(e);
            if (st == null || !(st.coverage >= 0.95)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-13
{tb:HH:mm} box coverage {(st == null ? double.NaN : st.coverage):F2} < 0.95: skip"); return; }
            double atrD = atrDaily();
            if (!(atrD > 0)) return;
            double tick = tickSize();
            double risk = Cfg.stopFraction * atrD;
            if (risk < Cfg.costFloorTicks * tick)   // under the cost floor: skipped and logged, not resized
            {
                if (journalOn()) trace(ETraceLevel.Journal, $"PS-13 {tb:yyyy-MM-dd HH:mm} skip: stop {risk /
tick:F0} ticks < cost floor {Cfg.costFloorTicks}");
                return;
            }
            m_level = e.poc; m_side = side; m_levelDay = GlobexDate(Bars.Time[0]); m_levelSessions = 0; justA
rmed = true;
            m_stop = m_level - side * risk; m_target = m_level + side * Cfg.targetR * risk;
            var ctx = new Dictionary<string, string> { { "boxH", boxH.ToString() }, { "boxL",
boxL.ToString() }, { "atr15", atr15.ToString("F2") }, { "atrD", atrD.ToString("F2") } };
            if (journalOn()) { journalArm(side > 0 ? "PS13LE" : "PS13SE", m_level, m_stop, m_target, ctx); lo
gSignal("PS-13", side, m_level, m_stop, ctx, true); }
        }

        private void stepDailyAtr(DateTime day, bool roll)
        {
            if (day != m_aggDay)
            {
                if (m_aggDay != DateTime.MinValue)
                {
                    if (!m_dRoll && !double.IsNaN(m_prevC)) m_tr.Add(Math.Max(m_dH - m_dL,
Math.Max(Math.Abs(m_dH - m_prevC), Math.Abs(m_dL - m_prevC))));
                    else m_tr.Add(m_dH - m_dL);                    // roll day: the gap to the old contract
is not range
                    if (m_tr.Count > Cfg.atrDailyLength) m_tr.RemoveAt(0);
                    m_prevC = m_dC;
                    if (!double.IsNaN(m_level)) m_levelSessions++;
                }
                m_aggDay = day; m_dH = Bars.High[0]; m_dL = Bars.Low[0]; m_dRoll = false;
            }
            m_dH = Math.Max(m_dH, Bars.High[0]); m_dL = Math.Min(m_dL, Bars.Low[0]); m_dC = Bars.Close[0];
            if (roll) { m_dRoll = true; m_prevC = double.NaN; }
        }
        private double atrDaily()
        {
            if (m_tr.Count < Cfg.atrDailyLength) return double.NaN;
            double s = 0; foreach (double x in m_tr) s += x; return s / m_tr.Count;
        }

        private double atrClosed(int ds, int len, int from)
        {
            double sum = 0;
            for (int k = from; k < from + len; k++)
            {
                DateTime t; double o, h, l, c, v, pc;
                HtfClosedBar(ds, k + 1, out t, out o, out h, out l, out pc, out v);
                HtfClosedBar(ds, k, out t, out o, out h, out l, out c, out v);
                sum += Math.Max(h - l, Math.Max(Math.Abs(h - pc), Math.Abs(l - pc)));
            }
            return sum / len;
        }

        /// <summary>Flat time for a Globex day: Cfg.flat, or on a CME early-close day RthCloseEt -
halfDayExitBefore if earlier
        /// (_CmeHolidays.Rules.Get(day).RthCloseEt; same rule as A263Config.halfDayExitBefore).</summary>
        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.10 Example 10 (`_Ex10WeeklyValueAcceptance`) — MT-02 weekly accepted value + 60m trigger (hard)

Catalogue rules (MT-02). Filter: by Tuesday's close, ≥ 2 daily closes above the prior-week VAH. Entry: the first pullback to prior-week VAH ± 0.25 × ATR20d with a 60m higher low, Wednesday–Friday. Stop: back inside the prior-week VA by 0.25 × weekly VA height, floored. Target: prior-week VAH + 1 × weekly VA height. Time stop 7 sessions. Sweep: acceptance days {1, 2} × target {1 × height, trail}. Multi-day levels reset at rolls. **G0-d-gated** (family 7).

Chart. Data1 = @ES 1m, Data2 = @ES 60m, dxFeed unadjusted, 24h. `WeeklyProfiles.enabled = true` (RTH sessions, roll policy Reset, minSessions 3), `NeedsDaily = true`, RollDetection Series. `[Input] AcceptanceDays` 1 or 2 (default 2).

Walk-through.

- The prior week is looked up by **key** (`_WeeklyProfiles.WeekKeyOf`, `WeekKey(i)`, `Week(i)`). A week counts as completed only when the next week's first session completes, so on Monday the prior week may still be `Week(0)`. Weeks that cross a roll, are partial, or have no valid VA are skipped.
- Acceptance: on Monday and Tuesday the RTH close (the 1m bar close-stamped 16:00 ET, or the early close) is compared with the prior-week VAH.
- From Wednesday to Friday during RTH: without enough acceptance closes, the week is done. Otherwise the zone is VAH ± 0.25 × `DailyContext.Atr20d`. The first bar that overlaps it marks the touch.
- Trigger: the first new closed 60m bar after the touch whose low is above the prior 60m low. The entry is a market order; stop = VAH − 0.25 × height; target = VAH + 1 × height. If the R from the signal close (close − stop) is under the 14-tick cost floor (`costFloorTicks`), the week is skipped and journaled.
- **Zone break:** after the touch and before the 60m trigger, any RTH 1m bar that closes below the zone (VAH − 0.25 × ATR20d) cancels the setup for the rest of the week. That is very strict, since one 1m close is enough. It is a coding choice the catalogue doesn't make (**VERIFY** against intent; Appendix F).
- A roll during the week cancels the setup. If `Atr20d` is NaN the week is skipped. The time stop exits after 7 sessions held.

Pitfalls.

- **Assumptions (labelled):** a close below the zone kills the week's setup (**VERIFY** against intent); "60m HL" read as a 60m bar whose low is above the prior 60m bar's low. The v2 8-tick floor is gone: the stop is the structural 0.25 × height, and the catalogue's 14-tick cost floor skips trades instead of widening them.
- Stop and target are sent from the bar after the fill (§1.7), so the fill bar has no working exit.
- Acceptance uses the RTH close (16:00 ET or the early close); being a swing idea, the example does not apply the half-day flat.
- `Atr20d` is NaN for 20 sessions after each roll (§4.3), so weeks right after a roll are skipped and journaled.
- The trail target arm is not coded.

Full strategy — `_Ex10WeeklyValueAcceptance.Strategy.CS`:

```
// Manual v3 example 10 - MT-02 weekly accepted value + 60m trigger (swing, long side as written in the
catalogue).
// Chart: Data1 = @ES 1m, Data2 = @ES 60m, dxFeed continuous UNADJUSTED, 24h CME Globex template. Weekly
profiles ON (RTH sessions).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;
```

```
namespace PowerLanguage.Strategy
{
    public class _Ex10WeeklyValueAcceptance : _VolumeProfileFuturesBase
    {
        [Input] public int AcceptanceDays { get; set; }   // sweep {1, 2}; catalogue default 2

        protected sealed class Mt02Config
        {
            public int contracts = 1;
            public double zoneAtr = 0.25, stopHeight = 0.25, targetHeight = 1.0;
            public int costFloorTicks = 14;                    // catalogue 2.1: ES cost floor; a stop under
it is skipped and logged, not resized
            public int timeStopSessions = 7;
        }
        protected Mt02Config Cfg { get; } = new Mt02Config();

        private IOrderMarket m_le, m_lxTime; private IOrderPriced m_lxStp, m_lxLmt;
        private DateTime m_day = DateTime.MinValue, m_week = DateTime.MinValue, m_last60 = DateTime.MinValue;
        private double m_vah = double.NaN, m_height = double.NaN;
        private int m_accept; private bool m_touched, m_doneWeek, m_rollThisWeek;
        private double m_stop, m_target; private int m_heldSessions;

        public _Ex10WeeklyValueAcceptance(object _ctx) : base(_ctx) { AcceptanceDays = 2; }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            NeedsDaily =
true;                               // Atr20d (NaN for ~20 sessions after each roll: those weeks are skipped)
            WeeklyProfiles.enabled = true;                   // source RthSessions, rollPolicy Reset,
minSessions 3
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "MT02LE", EOrderA
ction.Buy));
            m_lxTime = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "MT02LX", EOrderAct
ion.Sell, OrderExit.FromAll));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "MT02LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "MT02LT",
EOrderAction.Sell, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            if (AcceptanceDays < 1 || AcceptanceDays > 2) throw new
ArgumentException("AcceptanceDays must be 1 or 2");
            HtfStreams = new[] { 2 };
            base.StartCalc();
            m_day = m_week = m_last60 = DateTime.MinValue; m_vah = m_height = double.NaN; m_heldSessions = 0;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], et = ToEastern(t), day = GlobexDate(t);
            bool roll = rollOn(1);
            bool newDay = day != m_day;
            if (newDay) { m_day = day; if (StrategyInfo.MarketPosition > 0) m_heldSessions++; }
            DateTime wk = _WeeklyProfiles.WeekKeyOf(day);
            if (wk != m_week) { m_week = wk; m_vah = double.NaN; m_accept = 0; m_touched = m_doneWeek = m_rol
lThisWeek = false; }
            if (roll) m_rollThisWeek =
true;                  // multi-day levels reset at rolls: no new setup this week

            if (StrategyInfo.MarketPosition > 0)
            {
                if (m_heldSessions >= Cfg.timeStopSessions) { m_lxTime.Send(); return; }
                m_lxStp.Send(m_stop); m_lxLmt.Send(m_target);
                return;
            }
            m_heldSessions = 0;
            if (m_doneWeek || m_rollThisWeek) return;
            if (double.IsNaN(m_vah) && !loadPriorWeek(wk)) return;

            DayOfWeek dow = day.DayOfWeek;
```

```
            TimeSpan tod = et.TimeOfDay;
            bool rth = isRthBar(t);
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan closeBar = cme.RthCloseEt ?? new TimeSpan(16, 0, 0);   // the RTH close bar (close-
stamped 16:00 ET, or the early close)
            // acceptance: RTH closes (the 16:00 ET bar close on 1m) above the prior-week VAH, Monday and
Tuesday
            if ((dow == DayOfWeek.Monday || dow == DayOfWeek.Tuesday) && rth && tod == closeBar &&
Bars.Close[0] > m_vah) m_accept++;
            if (dow < DayOfWeek.Wednesday || dow > DayOfWeek.Friday || cme.NoRth || !rth) return;
            if (m_accept < AcceptanceDays) { m_doneWeek = true; journal("no acceptance"); return; }

            double atr = DailyContext != null ? DailyContext.Atr20d : double.NaN;
            if (!(atr > 0)) { m_doneWeek = true; journal("Atr20d NaN (post-roll seeding)"); return; }
            double zLo = m_vah - Cfg.zoneAtr * atr, zHi = m_vah + Cfg.zoneAtr * atr;
            if (!m_touched && Bars.Low[0] <= zHi && Bars.High[0] >= zLo) m_touched = true;   // first
pullback into the zone
            if (!m_touched) return;
            if (Bars.Close[0] < zLo) { m_doneWeek = true; journal("pullback broke the zone"); return; }   //
coding choice (VERIFY)

            // trigger: the first new CLOSED 60m bar after the touch whose low is above the prior 60m low (a
60m higher low)
            DateTime t60; double o, h, l, c, v, o1, h1, l1, c1, v1;
            HtfClosedBar(2, 0, out t60, out o, out h, out l, out c, out v);
            if (t60 == m_last60) return;
            m_last60 = t60;
            DateTime tPrev; HtfClosedBar(2, 1, out tPrev, out o1, out h1, out l1, out c1, out v1);
            if (!(l > l1)) return;
            m_stop = m_vah - Cfg.stopHeight * m_height; m_target = m_vah + Cfg.targetHeight * m_height;
            if (Bars.Close[0] <= m_stop || Bars.Close[0] >= m_target) { m_doneWeek = true; return; }
            double r = Bars.Close[0] - m_stop;   // R from the planned (signal-close) entry
            if (r < Cfg.costFloorTicks * tickSize()) { m_doneWeek = true; journal($"skip: stop {r /
tickSize():F0} ticks < cost floor {Cfg.costFloorTicks}"); return; }
            m_le.Send(Cfg.contracts); m_doneWeek = true;
            var ctx = new Dictionary<string, string> { { "vah", m_vah.ToString() }, { "height", m_height.ToSt
ring() }, { "atr20d", atr.ToString("F2") }, { "accept", m_accept.ToString() } };
            if (journalOn()) { journalArm("MT02LE", Bars.Close[0], m_stop, m_target, ctx);
logSignal("MT-02", +1, Bars.Close[0], m_stop, ctx, true); }
        }

        /// <summary>Prior-week profile by KEY: a week is completed only when the next week's first session
completes,
        /// so on Monday the prior week is still Week(0) and Week(1) is two weeks back.</summary>
        private bool loadPriorWeek(DateTime wk)
        {
            DateTime prior = wk.AddDays(-7);
            VpExtra e = WeekKey(0) == prior ? Week(0) : WeekKey(1) == prior ? Week(1) : null;
            if (e == null || e.crossesRoll || e.isPartial || !(e.vah > 0) || !(e.val > 0) || e.vah <= e.val)
return false;
            m_vah = e.vah; m_height = e.vah - e.val;
            return true;
        }

        private void journal(string why) { if (journalOn()) trace(ETraceLevel.Journal, $"MT-02 week
{m_week:yyyy-MM-dd} {why}"); }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` 629da0d) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

---

4 — Engines and helpers

> Each chapter covers the same items: **purpose · enable · config defaults · API · timing · journal · tests · snippet**.
>
> Snippets in this part are **fragments, not compiled** (they need a host strategy around them). Every member they use was checked by the verification script. Full public signatures, with file and class, are in **Appendix E**; complete compiled strategies are in §3.

4.1 `_CustomSignal` core services

- **Purpose:** the base for every strategy. It provides tracing, the journals, sizing, trade-management rules, time-zone helpers, the trading-day key, the RUN/RUN-END lines and M-PORT publishing.
- **Enable:** always on. The only `[Input]` is `TraceLevel`.
- **API (real signatures):**
- Tracing: `traceOn(ETraceLevel)`, `trace(ETraceLevel, string)`, `journalOn()`, `engineSink(ETraceLevel)`.
- Journaling: `journalArm(string entryOrderName, double plannedEntry, double initialStop, double target, IDictionary<string,string> context = null)`, `logSignal(string tag, int direction, double price, double stop, IDictionary<string,string> context, bool taken)`, `SignalHorizons` (default `{5,15,30,60}`).
- Trade management (CS-7, properties, not Inputs): `NoEntryBefore`, `NoEntryAfter`, `FlatBy` (`TimeSpan?`, compared with `sessionClock()`), `MaxTradesPerDay`, `DailyLossStopCount`, `DailyLossStopR`, `RiskUnitDollars`, `TimeStopBars`; `entryAllowed(out string reason)`, `mustFlatten()`, `TradesToday`, `LosersToday`, `NetProfitToday`.
- Sizing (CS-4): `sizeForRisk(IInstrument bars, double entry, double stop, double riskFraction, SizingLimits limits)`, where `SizingLimits` has `maxQty`, `maxNotionalFraction`, `unitNotional`, `minStopTicks`, `roundLots`. Also `equityAvailable(double)` and `costFilterPasses(IInstrument, entry, stop, out reason)` with `CostFilterMaxFractionOfR` (default 0 = off).
- Time (CS-6): `ToZone(DateTime barTime, string tzId)`, `inTimeWindow(TimeSpan start, TimeSpan end, string tzId)`, `BarTimeZone`, `resolveBarTimeZone(IInstrumentSettings)`, `TradingDayRoll` (`Calendar` default, or `Auto`), `tradingDayKey(DateTime)` virtual, `sessionClock(DateTime)` virtual, `Holidays` (`_NyseHolidays`).
- **Timing:** the CS-7 counters use `tradingDayKey`. `mustFlatten` is true at or after `FlatBy`, or once `TimeStopBars` have elapsed.
- **Journal:**
- `RUN id(..) build(..) strategy(..) symbol(..) category(..) resolution(..) timeZone(..) firstBar(..) barNumber(..) app(..) inputs[..]`, plus `event[canon] journalSchema(CS-1+evtDayTier,evtMinSince)` when Event is on.
- `RUN-END id(..) lastBar(..) barNumber(..) realTime(..)`.
- `entry blocked: <reason>` at Info.
- **Snippet:**

```
NoEntryBefore = new TimeSpan(9, 35, 0); NoEntryAfter = new TimeSpan(15, 30, 0); FlatBy = new TimeSpan(15,
55, 0);
MaxTradesPerDay = 2; DailyLossStopCount = 2;
// CalcBar:
if (mustFlatten()) m_lx.Send();
string why;
if (StrategyInfo.MarketPosition == 0 && entryAllowed(out why))
{
    int qty = sizeForRisk(Bars, entry, stop, 0.005, new SizingLimits { maxQty = 5, minStopTicks = 4 });
    if (qty > 0) m_le.Send(entry, qty);
}
```

4.2 `_CmeSessionClock` and the CME/NYSE calendars

- **Purpose:** CME Globex session windows in US Eastern time (Asia, Europe, NewYork, Rth, Premarket, PostMarket), the Globex trading date, halt/closed state, and holiday hooks.
- **Enable:** `new _CmeSessionClock(resolveBarTimeZone(Bars.Info))` (as in `_CmeSessionClockTest`). On the futures bases it is built in and exposed as `SessionClock`.
- **Defaults (ET):**
- Globex open 18:00, close 17:00; halt 17:00–18:00.
- Asia 18:00–03:00; Europe 03:00–11:30; NewYork 08:00–17:00.
- `RthStart` 09:30, `RthEnd` 16:15 (session flag), `RthValueEnd` 16:00 (value window).
- `UseCmeHolidayTable = true`.
- **API:**
- Conversion and state: `ToEastern`, `EasternOf`, `State(barTime)` → `GlobexState {Closed, Halt, Open}`, `Sessions(barTime)` → `[Flags] MarketSession`, `InSession`, `InAny`, `GlobexDate(barTime)`, `GlobexDateEt`, `PrimarySessionEt`.
- Session timing: `TimeSinceOpenEt/MinutesSinceOpenEt(et, session)`, `TimeUntilCloseEt/MinutesUntilCloseEt`, `SessionLength`, `IsOpeningRangeEt(et, session, minutes)`, `IsRthAfterInitialBalanceEt(et, ibMinutes)`, `TrySessionBoundsEt(et, session, out open, out close)`, `TryWindow`.
- RTH value window: `RthValueEndEt(globexDate)`, `IsRthValueBarEt(et)` (close-stamped bar in `(RthStart, RthValueEnd]`).
- Holidays: `IsGlobexClosedDay`, `IsNoRthDay` (NYSE holiday that is not a Globex closure), `HolidayHaltEt`, `RthEarlyCloseEt`. Hooks: `GlobexClosedDayHook`, `NoRthDayHook`, `HolidayHaltHook`, `RthEarlyCloseHook`.
- Static: `DefaultGlobexClosedDay`, `InWindow`, `IsSingleFlag`.
- **`_CmeHolidays.Rules`:**
- `Get(date)` returns a `Day` with `Closed`, `NoRth`, `HaltEt`, `RthCloseEt` and `Name`.
- Constants: `HolidayHaltEt` 13:00, `EarlyCloseEt` 13:15, `GoodFridayCloseEt` 09:15, `MourningCloseEt` 09:30.
- Helpers: `ClosedDates(year)`, `Observed`, `NthWeekday`, `LastWeekday`, `Easter`.
- **Futures-base wrappers** (`_VolumeProfileFuturesBase`): `IsRth()`, `IsAsia()`, `IsEurope()`, `IsNewYork()`, `IsLondonNyOverlap()`, `GetGlobexState()`, `GetSessions()`, `MinutesSinceOpen(session)`, `MinutesUntilClose(session)`, `IsOpeningRange(session, minutes)`, `IsRthAfterInitialBalance(ib)`, `TrySessionBounds(...)`, `IsNewGlobexDay()`, `Session` (SessionStats: RTH/ETH/IB, prior-day H/L/C, `IbRange`, `Gap`), `IbMinutes` (60).
- **Timing:** bars are close-stamped. A bar is RTH when its ET close is in `(09:30, value end]`.
- **Journal:** the test signal prints `VPF-7 clock globexDate(..) firstBarEt(..) state(..) sessions(..) noRth(..) rth(..)`.
- **Tests:** `tests/SessionClockTests`, `tests/CalendarTests` (with `gen.sh`).
- **Snippet:**

```
DateTime et = m_clock.EasternOf(Bars.Time[0]);
DateTime gx = m_clock.GlobexDate(Bars.Time[0]);
DateTime o, c;
if (m_clock.TrySessionBoundsEt(et, _CmeSessionClock.MarketSession.Rth, out o, out c) && !m_clock.IsNoRthDay(g
x))
{
    int mins = m_clock.MinutesSinceOpenEt(et, _CmeSessionClock.MarketSession.Rth);
}
```

4.3 M-DAILY `_DailyContext`

- **Purpose:** daily/weekly context: Wilder ATR, SMA200, RSI2, TSMOM, HV and HV percentile, LBR, prior day and week levels, and the gap. At a detected roll the stored history is difference-adjusted and the ATR is **reseeded** (see Timing).
- **Enable:** `NeedsDaily = true`. Regime enables it automatically.
- **`DailyConfig` defaults:**

- `dataStream 0` (aggregate Data1 per `tradingDayKey`), `weeklyStream 0`;
- `atrLength 20`, `smaLength 200`, `rsiLength 2`, `tsmomLookbacks {252,126,63}`;
- `hvLengths {}`, `hvAnnualisation 252`, `hvPctLookback 0`, `lbrRsiLength 0`;
- `closeBasis EDailyClose.RthValue`; `HvOn`; `Validate()`.
- **API:** `Atr20d` (ATR of length `atrLength`, despite the name), `Sma200`, `Rsi2`, `Tsmom(n)`, `Hv(n)`, `HvPercentile(n)`, `Roc1`, `Lbr`, `RthCloseAdj`, `RthClosePrev`, `PrevHigh`/`PrevLow`/`PrevClose`/`PrevDate`/`PrevRange`, `PrevWeekHigh`/`PrevWeekLow`, `Valid`, `Days`, `GapPct(todayOpen)`, and the host's `DailyGapPct`.
- **Timing:** a day is added when the next trading day's first bar arrives. On that bar, `PrevDate` is yesterday, so every value describes completed prior sessions. With a daily data stream, only the closed daily bar (`HtfClosedIndex`) whose day is before today is used. Rolls (only with RollDetection on) call `OnRoll(day, rollBarOpen - PrevClose)` (`_CustomSignal` line 658). `applyRoll` difference-adjusts the stored o/h/l/c/RTH closes and the weekly values, then sets the ATR to NaN and restarts its seed: **`Atr20d` is NaN and `Valid` is false for `atrLength` sessions after every roll.** With `atrLength = 200` and quarterly @ES rolls (about every 63 sessions) the ATR never becomes valid. With RollDetection Off, nothing is adjusted and the roll gap enters the ATR. `PrevClose` is the last bar's close, `RthClosePrev` the RTH close, `RthCloseAdj` the roll-adjusted RTH close.
- **Journal:**
- `DAILY {date} return spans a gap of N business days since {prev}`;
- `DAILY hv{n} first valid {date} …`;
- `DAILY hv{n}pct first valid …`;
- `DAILY roll reset {date}: history adjusted by {d}, ATR reseeding`;
- `DAILY closeBasis=Bar (P3 unverified): …`;
- a Warning: `DAILY Data{n} advanced k bars between Data1 bars`.
- **Tests:** `tests/DailyTests` (`gen.sh` generates the calendars).
- **Snippet:**

```
NeedsDaily = true; DailyConfig.hvLengths = new[] { 20 };      // before base.StartCalc()
// CalcBar
_DailyContext d = DailyContext;
if (d != null && d.Valid && d.PrevDate < tradingDayKey(Bars.Time[0]))
    if (Bars.Close[0] > d.PrevHigh && d.Hv(20) < 0.25) { /* … */ }
```

4.4 G-R regime engine `_RegimeEngine`

- **Purpose:** an HMM (or GMM) market regime on daily features, with hysteresis and an ATR-tercile fallback.
- **Enable:** `RegimeConfig.enabled = true`. This forces M-DAILY with HV {20, 63}.
- **`RegimeConfig` defaults:**
- Model: `method Hmm`, `K 3`, `kSelect Fixed`, `features {Return, LogHv20, Tsmom63z}`.
- Fitting: `fitSessions 1000`, `minSessions 500`, `refit Monthly`, `refitEveryN 21`, `restarts 10`, `maxIter 200`, `tol 1e-6`, `regCovar 1e-6`, `transPrior 1`.
- Filtering and switching: `warmFilter 63`, `switchProb 0.60`, `switchMargin 0.10`.
- Other: `seed 20261021`, `fallback AtrTercile`, `publish false`, `atrPctLookback 250`, `parkinsonBars 10`, `bicDiagnostic true`, `rollSuspectZ 5`, `gapBusinessDays 4`.
- Methods: `Validate()`, `Copy()`, `IndexOf`.
- **Enums:** `ERegimeMethod {Hmm, Gmm}`, `ERegimeKSelect {Fixed, Bic}`, `ERegimeRefit {Monthly, EveryNSessions, Quarterly}`, `ERegimeFallback {AtrTercile, Unknown}`, `ERegimeFeature {Return, LogHv20, Tsmom63z, Parkinson10}`.
- **API:** the `Regime` property returns a copy of `RegimeState`:
- fields: `Valid`, `Label` (0 = calmest, -1 invalid), `RawArgmax`, `Probs`, `AsOf`, `FittedThrough`, `K`, `LogLik`, `Refits`, `UsingFallback`, `RelabelConflicts`;
- methods: `Copy()`, `Neutral(K)`.
- When the engine is disabled, `Regime` returns a neutral state (`Valid=false`, `Label=-1`).
- Engine statics: `AtrPercentile`, `AtrBucket`, `Hysteresis`, `Argmax`.
- **Timing:** the engine is fed once per completed M-DAILY day. `AsOf` is that day, and the value is usable from the **next** session. `FittedThrough < AsOf`. Refits on a roll day are deferred.

- **M-PORT:** with `publish = true`, `pm.regime = Label+1` and `pm.regimeP = Probs` are published at session close.
- **Journal:**
- `REGIME {date} label() raw() p() fb();`
- `REGIME switch a->b {date};`
- `REGIME refit {date} window() K() ll() iters() conv() means();`
- `REGIME diag {date} bicK…;`
- `REGIME fallback reason(…);`
- `REGIME refit deferred {date}: roll day;`
- `REGIME gap …;`
- `REGIME roll adjust suspect …;`
- `REGIME relabel-conflict ….`
- **Tests:** `tests/RegimeTests` (`reference.py`; `run.sh` can `--export gv`), `tests/GmmTests`.
- **Snippet:**

```
RegimeConfig.enabled = true;                    // before base.StartCalc()
// CalcBar
RegimeState r = Regime;
bool usable = r.Valid && !r.UsingFallback && r.AsOf < tradingDayKey(Bars.Time[0]);
int bucket = usable ? r.Label : -1;             // a PRIOR / bucket, never a trigger
```

4.5 RegimeGate / A2-63 helpers (`_RegimeGate.Function.CS`)

- **Purpose:** the pre-registered regime consumer used by `_Test6`: the U/G/A arms, the decision, canonical hashing and the A2-63 host core.
- **Enable:** construct the objects yourself, then call `RegimeGate.ApplyArm(RegimeGateConfig, RegimeConfig)` before `base.StartCalc()`.
- **Defaults:**
- `RegimeGateConfig { arm = ERegimeGateArm.U, placeboSeed = RegimeGate.PlaceboSeed (20261022) }`.
- `A263Config { k 0.5, atrLength 20, rthOpen 09:30, activeEnd 15:30, exitTime 15:55, rthClose 16:00, contracts 1, halfDayExitBefore 5m }`.
- **API:**
- `RegimeGate.Decide(arm, RegimeState, session, atrTercile)` returns a `RegimeGateDecision` with `session`, `asOf`, `arm`, `label`, `raw`, `atrTercile`, `pFilt`, `fb`, `usable`, `wouldBlock`, `allow` and `reason` (an `ERegimeGateReason {Ok, Warmup, Invalid, Fallback}`).
- `RegimeGate.Line`, `TradeFields`, `HashFields`, `Canon`, `Hash`, `Fnv1a64`; constants `BlockedLabel = 2` and `Version = "A263-RG-2"`.
- `AtrTercileTracker(lookback, K)`: `.AddDay(date, atr)`, `.Tercile`, `.LastDate`.
- `A263Core(cfg)`: `IsLatchBar(etClose)`, `Latch(etClose, open, atr, atrAsOf, allow, earlyCloseEt, noRth)`, `Step(etClose, isLong)` returning `A263Action {latched, sendStop, stop, exit}`, plus `HasSignal`, `Threshold`, `Suffix()`.
- `RegimeGateCounts.Add(d, noRth)`, `.Suffix()`.
- **Arms:** U = ungated; G = skip when the prior session's HMM label == 2; A = skip ATR tercile 2 (via `fitSessions = minSessions = 100000`).
- **Hash pins:** U `26b38655261cc0bf`, G `9c1da7b123ca9441`, A `3d9814a5f264c353`.
- **Journal:**
- Header: `REGGATE header version() arm() cfg() seed() symbol() bar() barDeviation() firstBar() canon[]`.
- Per session: `REGGATE {date} arm() asOf() label() raw() pFilt() fb() usable() allow() atrT() wouldBlock() why() signal() cfg() seed()`, followed by `O() atr20d() atrAsOf() atrOk() thr() exitAt() halfDay() noRth()` and the counts `nSess() nUsable() nWarm() nInv() nFb() nBlock() nNoRth()`.
- **Tests:** `tests/RegimeGateTests` (`gen.sh`).
- **Snippet:** see §3 example 6 (`_Ex09OpenAtrBreakout`), a compiled cut-down of `_Test6.CalcBar`.

4.6 M-EVENT `_EventCalendar`

- **Purpose:** a scheduled-event calendar with as-of visibility. It covers:
- macro releases: FOMC, CPI, PPI, NFP, JOLTS, Claims, PCE, GDP, Retail, ISM, Jackson Hole, testimony;
- energy: EIA;
- calendar events: quarterly expiry, roll week, monthly opex, index rebalance, half-day, VIX expiry, Russell recon.
- **Enable:** `EventConfig.enabled = true`.
- **`EventConfig` defaults:**
- Scope: `classes Macro|Calendar`, `minTier T3`, `tierOverride {}`, `csvPath ""`, `allowPartialCoverage false`.
- Blackout: `blackoutPreMin 0`, `blackoutPostMin 0` (0/0 = no blackout), `blackoutTier T1`, `blackoutFlatten false`.
- Other: `publish false`, `includeRussell false`; methods `Validate()`, `Canon()`.
- **Enums:**
- `EEventTier {None=0, T3=1, T2=2, T1=3, C=8}` (T1 is the highest);
- `EEventClassMask {None, Macro=1, Calendar=2, Energy=4, All=7}`;
- `EEventKind` (FomcDecision=1 ... EiaPetroleum=18, QuarterlyExpiry=30 ... RussellRecon=36, Custom=62);
- `EEventFlags` (Sep, Quad, Unscheduled, Scheduled, Delayed, Canceled, Csv, RuleOnly, Pm).
- **Records:**
- `EventRec` fields: `kind`, `tier`, `flags`, `etTime`, `etEnd`, `utc`, `tradingDay`, `knownSinceUtc`, `canceledSinceUtc`, `refPeriod`, `name`, `source`; plus `IsSpan`, `Class`, `KindName`.
- `EventDay` fields: `day`, `maxTier`, `kindsMask`, `hasFomc`, `hasSep`, `isHalfDay`, `isRollWeek`, `isQuarterlyExpiry`, `isOpex`, `isRebalance`, `firstReleaseEt` (TimeSpan?), `count`, `eiaRuleOnly`; plus `Has(kind)`.
- **API:**
- State: `TradingDay`, `EventBlackout`, `EventFlattenNow`, `NowUtc`, `Disabled`.
- Next and previous events: `NextEvent(minTier, out e)`, `PrevEvent(minTier, out e)`, and overloads `(nowUtc, minTier, mask, out e)`.
- Distance in minutes: `MinutesTo(minTier, mask = Macro)`, `MinutesSince`, `MinutesSinceAt`.
- Windows: `IsInWindow(minTier, pre, post, mask = Macro)`, `IsInWindowAt`.
- Day queries: `EventsOn(day, mask = All)`, `EventsOnExPost`, `EventsInRange`, `Classify(day)`, `Classify(day, asOfUtc)`, `ClassifyExPost(day)`.
- Expiry and rolls: `IsRollWeek()`, `IsRollDay()`, `SessionsToExpiry()`, `InSpan(kind)`, `LastT1Instant(out et)`.
- Conversion: `ToEt`, `TradingDayOf`, `SessionOpenUtc`.
- Statics: `RuleRollDate`, `RollDateFor`, and `EventRules` (`IsNyseBusinessDay`, `ThirdFriday`, `OpexDate`, `RollDate`, `VixExpiry`, `TableSha`).
- Host wrappers: `EventBlackout`, `EventFlattenNow` (false when disabled).
- **Timing:**
- Stepped once per closed bar with `[open, close)` in UTC.
- An event is visible only once it is known (`knownSinceUtc <= now`) or has happened. Cancellations are honoured as of now.
- `Classify(day)` is as of the session open.
- The blackout uses the Macro class only, so Energy consumers call `IsInWindow(..., EEventClassMask.Energy)` explicitly.
- Window queries on daily bars return false.
- **The base never blocks orders.** It is your job to act on `EventBlackout`/`EventFlattenNow`.
- **M-PORT keys** (`publish = true`): `pm.evtTier`, `pm.evtMask` (daily); `pm.evtWin`, `pm.evtNext` (per bar).
- **CS-1 extra columns:** `evtDayTier`, `evtMinSince`.
- **Journal:**
- `EVENT created classes() minTier() blackout(pre/post tier) …;`
- `EVENT table missing: …;`
- `EVENT coverage rules(…) …;`
- `EVENT correction …;`

- `EVENT day {yyyy-MM-dd} at {bar} tier() kinds(Kind@HH:mm,…) firstRelease() [rollWeek] [halfDay] [EiaRuleOnly];`
- `EVENT blackout on|off at {bar} {Kind} {et} ET pre() post() [flatten];`
- an Error line ending `…; engine disabled` **only** when the table is missing or its hash mismatches (EV-4.6b);
- `COVGAP {Kind} {from}..{to} Unknown …` once at start, and `COVGAP day {yyyy-MM-dd} Unknown Kind(blocks|belowMinTier|NoEvent)` per affected day.
- **EV-4.6b (PRs #28/#29):** `Coverage(kind, day)`, `CoverageBlocks(day, mask, minTier, out kind)`, `CoverageBlocks(day, mask)`, `CoverageReport(from, to, mask[, minTier], out flagged)`, `CoverageStats()`, `EventRules.CoverageHoles()`, `EventRules.TableSha()`, `EventRules.ExpectedTableSha`, and `EventConfig.tierOverride`. Details in Appendix G.
- **Tests:** `tests/EventTests` (`gen.sh`). Tooling lives in `tools/m-event`.
- **Snippet:**

```
EventConfig.enabled = true; EventConfig.blackoutPreMin = 5; EventConfig.blackoutPostMin = 15;
EventConfig.blackoutFlatten = true;
// CalcBar
if (EventFlattenNow && StrategyInfo.MarketPosition != 0) m_lx.Send();
_EventCalendar ev = EventCalendar;
if (ev != null)
{
    EventDay today = ev.Classify(ev.TradingDay);
    bool fomcDay = today.hasFomc;
    double toT1 = ev.MinutesTo(EEventTier.T1);
    EventRec next;
    if (ev.NextEvent(EEventTier.T2, out next) && journalOn()) trace(ETraceLevel.Journal, $"next
{next.KindName} {next.etTime:HH:mm}");
}
```

4.7 M-AVWAP `_AvwapManager`

- **Purpose:** anchored VWAPs on events, with σ-bands, slope, age, touches and expiry. Optional placebo twins (P1/P2) are available.
- **Enable:** `AvwapEvents = EAvwapAnchor.WeekOpen | EAvwapAnchor.GapOpen | …`.
- **`EAvwapAnchor` flags:** `SwingHigh=1`, `SwingLow=2`, `Breakout=4`, `GapOpen=8`, `VolumeClimax=16`, `WeekOpen=32`, `MonthOpen=64`, `QuarterOpen=128`, `NewsBar=256` (inert), and `Swing`.
- **`AvwapLimits` defaults:**
- Anchors: `maxAnchors 6`, `maxAgeSessions 10`, `replaceSameType true`.
- Bands and slope: `bandSigmas {1,2,3}`, `slopeBars 5`, `dataStream 1`.
- Detection thresholds: `gapAtrFraction 0.5`, `gapFallbackTrMultiple 3`, `climaxZ 3`, `fallbackLength 20`, `backfillBars 2`.
- Placebos: `placebos EAvwapPlacebo.None`, `p2MinPool 10`.
- **API:**
- Manager: `Anchors` (`IReadOnlyList<_Avwap>`), `Latest(EAvwapAnchor type)`, `LastBar`, `FallbackTrAvg`.
- Each `_Avwap` has:
    - identity: `Id`, `Type`, `AnchorTime`, `AnchorBar`, `UsableFrom`, `Direction`;
    - value: `Value`, `Sigma`, `SlopeTicks`, `SlopeAtr`, `Band(k)`, `Valid`;
    - lifecycle: `AgeBars`, `AgeSessions`, `Touched`, `TouchCount`, `Expired`, `ExpireReason`;
    - tags: `Tag` (`AV:{Type}:{yyyyMMddHHmm}`), `TagFor(arm)`;
    - placebos: `P1`, `P1Sigma`, `P2`, `P2Sigma`, `p2Degraded`.
- **Timing:**
- Swing anchors become usable at PA pivot confirmation (`UsableFrom`); other anchors at the anchor time.
- GapOpen uses `isRthBar` and M-DAILY ATR when available.
- All anchors are invalidated on a roll.
- **Dependencies:** Swing and Breakout force PriceAction. VolumeClimax and P2 force TOD Volume.
- **Journal:**
- `AV created {Tag} usable={t} value={v};`
- `AV expired ({why}) {Tag} at {t};`

- `AV invalidated (roll) {Tag} at {t};`
- `AV p2Degraded ….`
- **Tests:** `tests/AvwapTests.`
- **Snippet:**

```
AvwapEvents = EAvwapAnchor.Swing | EAvwapAnchor.GapOpen;
// CalcBar
_AvwapManager am = AvwapManager;
if (am != null)
    foreach (_Avwap a in am.Anchors)
        if (a.Valid && a.UsableFrom <= Bars.Time[0] && Bars.Low[0] <= a.Band(-1) && a.Direction > 0) { /*
long pullback to -1σ */ }
```

4.8 PRICEACTION `_PriceActionEngine`

- **Purpose:** pivots (fractal or ATR zig-zag) on several layers, HH/LH/HL/LL labels, BOS/CHoCH, structure, equal highs/lows, and candle and volume flags.
- **Enable:** `PriceActionConfig.enabled = true`. Extra instances go in `ExtraPriceAction.Add(new PriceActionConfig{ enabled = true, dataStream = 2, role = "htf" })`.
- **`PriceActionConfig` defaults:**
- Stream and pivots: `dataStream 1`, `pivotMode Fractal`, `tieRule Strict`, `layers 2`, `strength 1`.
- ATR and volume: `atrLength 20`, `volumeLength 20`, `volumeIncludesCurrent true`, `atrMultiple 1.5`.
- Candle factors: `factorDoji 0.1`, `factorClose 0.1`, `factorStrong 1.0`, `factorEpsilon 0.05`, `equalFactor 0.1`.
- Breakouts and history: `lookbackPivots 5`, `minBreakoutLayer 1`, `breakoutNeedsStrong true`, `history 50`, `candles true`.
- Presets: `preset Library`, `role ""`; `ApplyPreset(EPriceActionPreset)`, `ApplyPresetDefaults`. The Futures preset uses `LeftInclusive` and `volumeIncludesCurrent = false`, and the futures bases apply it.
- **API:**
- Engine state: `IsWarm`, `Atr`, `VolumeMean`, `VolumeSigma`, `Candle` (ECandle flags), `Has(ECandle)`, `Changed`, `LastBarTime`, `LastBar`.
- Bar events: `NewPivotHigh`, `NewPivotLow`, `BosUp`, `BosDown`, `ChochUp`, `ChochDown`.
- Pivots and structure: `LastHigh(layer = 0)`, `LastLow(layer = 0)`, `High(layer, ago)`, `Low(layer, ago)`, `PivotCount(layer, isHigh)`, `Structure(layer = 0)` → `EStructure {Unknown, Up, Down, Range}`, `DoubleTop(layer)`, `DoubleBottom(layer)`.
- `_Pivot` fields: `isHigh`, `layer`, `seq`, `price`, `extremeTime`, `extremeBar`, `confirmTime`, `confirmBar`, `label` (EPivotLabel), `isEqual`, `equalToSeq`, `legSizeAtr`.
- Lookup: `PriceActionFor(int dataStream, string role = null)`.
- **Timing:** steps on its own stream's closed bars. A pivot exists only from `confirmTime` (after `strength` bars) and is never relabelled. The engine resets on a roll.
- **Journal:**
- `PA{ds} L{layer} PivotHigh|PivotLow #{seq} {price} {label} [EQ#n] ext={t} conf={t};`
- `… BreakoutUp|BreakoutDown L{layer} of #{seq} {price} at {t};`
- `… BOS up|down [CHoCH] at {t};`
- `… roll reset at {t};`
- `PA{ds}-{role} created strength() layers() shared().`
- **Tests:** `tests/PriceActionTests` (`McShim.cs`).
- **Snippet:**

```
PriceActionConfig.ApplyPreset(EPriceActionPreset.Futures); PriceActionConfig.enabled = true;   // Create()
// CalcBar
_PriceActionEngine pa = PriceActionEngine;
if (pa != null && pa.IsWarm && pa.BosUp && pa.Structure(1) == EStructure.Up)
{
    _Pivot hl = pa.LastLow(0);
    double stop = hl != null ? hl.price - 2 * tickSize() : double.NaN;   // tickSize(): protected helper of
_CustomSignal
}
```

4.9 M-TOD `_TodNormaliser`

- **Purpose:** time-of-day norms (z-score, median, percentile, cumulative z) for volume, delta, range and true range per session minute bucket.
- **Enable:** `TodMetrics = ETodMetric.Volume | ETodMetric.Range`.
- **Defaults:** `TodSessions 20`, `TodMinSessions 10`, `TodBucketMinutes 0` (host: 0 = the Data1 minute bar size, else 1). The class field `bucketMinutes` defaults to 1 but the host always overwrites it (`_CustomSignal` ~line 561: 0 → the Data1 minute size, else 1) — VERIFY resolved.
- **API:** `Valid(m)`, `Z(m)`, `Median(m)`, `Pct(m)`, `ZCum(m)`, `Sessions(m)`, `VolumePool(minuteOfSession)`.
- **Timing:** norms come from prior completed sessions. Half-days are excluded (via `todHalfDay`). Range and TR are dropped on the roll-splice bar. Delta requires the futures base order flow.
- **Journal:** `TOD delta tier change a -> b: norm reset at session {day}`.
- **Tests:** `tests/TodTests`.
- **Snippet:**

```
TodMetrics = ETodMetric.Volume;
_TodNormaliser tod = TodNormaliser;
bool climax = tod != null && tod.Valid(ETodMetric.Volume) && tod.Z(ETodMetric.Volume) > 3;
```

4.10 M-XMKT `_XmktLeg`

- **Purpose:** cross-market legs on extra chart streams (for example NQ on Data2): log returns, rolling correlation, beta and relative-strength z. Orders stay on Data1.
- **Enable:** `XmktLegs.Add(new XmktLegConfig { role = "NQ", dataStream = 2 })`.
- **Defaults:** `role "leg"`, `dataStream 2`, `maxStaleSec 0` (= 2× the leg bar; off for non-time bars), `returnBars 1`, `corrLength 60`, `rsLength 60`, `leaderSwing false`.
- **API:** `XmktLeg(role)` returns `_XmktLeg` with `Available`, `LastLegTime`, `LastClose`, `LogReturn`, `Data1LogReturn`, `Correlation`, `Beta`, `RsZ`. `XmktLeaderSwing(role)` returns a PA engine on the leg.
- **Timing:** the leg is fed its last closed bar with `Time <= Data1 time` (`htfMapped`). It becomes unavailable when stale. Leg rolls are not inherited from Data1.
- **Journal:** `XMKT {role} Data{n} available at {t} (staleness limit …)`; Info `XMKT leg … symbol() resolution() dataFeed() maxStale()`.
- **Tests:** `tests/XmktTests`.
- **Snippet:**

```
_XmktLeg nq = XmktLeg("NQ");
if (nq != null && nq.Available && nq.RsZ < -2) { /* ES strong vs NQ */ }
```

4.11 M-TICK `_TickStream`

- **Purpose:** per-Data1-bar tick statistics from a lazy DataLoader 1-tick Trade request: prints, max prints/s, inter-print median/P10, burst z, and the size histogram.
- **Enable:** `TickConfig.enabled = true`.
- **Defaults:** `historyTicks 2,000,000`, `realTime true`, `burstZ 3`, `burstNormLength 200`, `burstMinSamples 30`, `sizeBuckets {1,2,5,10,20,50}`, `largePrintSize 20`, `sizeOutputsVerified false`.
- **API:** `TickStats` (last closed bar, or null) is a `TickBarStats` with `valid`, `quality`, `prints`, `maxPrintsPerSecond`, `interPrintMedianMs`, `interPrintP10Ms`, `burstZ`, `burst`, `sizeHistogram`, `largePrints`, `volume`.
- **Timing:** aggregated over `(Time[1], Time[0]]` at bar close. Late ticks are dropped. The size fields stay NaN/null until `sizeOutputsVerified`.
- **Journal:**
- `TICK DataLoader 1-tick Trade request for {sym}, N ticks back, RT=…`;
- `TICK n late tick(s) dropped …`;
- `TICK data missing/late … outputs invalid`;
- `TICK data valid again …`.

- **VERIFY:** the `Bar` member names are read by reflection (`Time`, `Close`, `TotalVolume`/`Volume`/`Ticks`). This is flagged in the code as "verify in the editor".
- **Tests:** `tests/TickTests`.

4.12 M-MTF (`_MtfMath`, host helpers)

- **Purpose:** closed-bar-only access to higher timeframes of the same instrument.
- **Enable:** `HtfStreams = new[] { 2 }`. StartCalc throws if the stream is missing or carries another root.
- **API:**
- Host: `HtfClosedIndex(ds, ago = 0)`, `HtfClosed(ds, ago = 0)`, `HtfClosedBar(ds, ago, out time, out o, out h, out l, out c, out v)`, `htfMapped(ds, maxBack = 2)` (-1 = none), `HtfForming(ds)`.
- Statics: `_MtfMath.ClosedIndex`, `MapIndex`, `SymbolRoot`, `IsStale`.
- **Journal:** `MTF Data{n} first mapping {t} -> {htf t} (ago i, status s)` (once per session); `MTF forming-bar access on Data{n} (not closedBarsOnly)`.
- **VERIFY:** whether MC stamps HTF bars with their end time ("Probe P2" in the code).
- **Tests:** `tests/MtfTests`.
- **Snippet** (`_MtfProbe`):

```
HtfStreams = new[] { 2 }; base.StartCalc();
// CalcBar
int i = htfMapped(2);
if (i >= 0) { double h60 = BarsOfData(2).High[i]; }
```

4.13 Rolls (`_RollDetector`, `_RollWindow`)

- **Purpose:** detect contract splices on continuous unadjusted futures.
- **Enable:** `RollDetection = ERollDetection.Series | Table | SeriesOrTable` (default `Off`).
- **Settings:** `RollGapTicks` (8); `RollDates` (a HashSet of trading days for Table).
- **Series rule:** a trading-day open that gaps ≥ RollGapTicks from the prior close on days 1–21 of Mar/Jun/Sep/Dec, or a change of `Bars.Info.Name`.
- **API:** `rollOn(ds)`, `RollBars` (1-based Data1 bars), `IsRollWindow(tradingDay)` (roll session ±1; requires rings, FVG or RollDetection).
- **Journal:** Info `VPF-4 roll splice detected tradingDay() bar() name()`; Warning `RollDetection is Off on a futures symbol …`.
- **Tests:** `tests/RollTests`. Event roll dates: `_EventCalendar.RuleRollDate`/`RollDateFor`.

4.14 Bar rings and FVG (`_BarRing`, `_FvgDetector`, `LevelRef`)

- **Rings:**
- `requireRing(ds, bars, consumer)` (after `base.StartCalc()`), then `Ring(ds)` returns a `_BarRing` with `Capacity`, `Count`, `Added`, `Dropped`, `LastBarNumber`, `this[ago]`, `At(idx)`, `IndexAtOrBefore`, `FirstIndexAtOrAfter`, `Covers`, `Window(t0, t1, t0Exclusive)` and `Session(day, rthOnly)`.
- `RingBar` fields: `bar`, `time`, `tradingDay`, `o h l c v up down`, `rth`, `rollBar`.
- `ringBarOf(ds, ago)`.
- `BarRingConfig.capacityBars 0` (auto, capped at `MaxAutoBars` 40000).
- Journal: `RING Data{n} capacity() consumers()`.
- **FVG:**
- Enable with `Fvgs.enabled = true`.
- Defaults: `dataStream 1`, `minTicks 1`, `minAtrFrac 0`, `atrLen 14`, `crossSessions false`, `rthOnly false`, `maxLive 20`, `maxAgeSessions 5`, `fillOn EFvgFillOn.Close`.
- `FvgDetector` has `Live`, `NewThisBar`, `Atr`, `Nearest(price, dir)`, `ToLevel(f, startEdge, stream)`.
- `Fvg` fields: `id`, `dir`, `lo`, `hi`, `start`, `createdTime`, `createdDay`, `touched`, `filled`, `expired`, `invalid`, `fillPct`, `ageSessions`, `Size`, `IsLive`, `Tag` (`FVG:U|D:yyyyMMddHHmm`).
- Use `fvgLevel(f, startEdge)` to get a `LevelRef`.
- **`LevelRef`:**

- fields: `kind` (`ELevelKind`), `price`, `lo`, `hi`, `createdTime` (usable from), `sourceT0`/`sourceT1`, `dataStream`, `tag`, `quality`, `crossesRoll`, `isRollWindow`;
- methods: `Line(...)`, `Zone(...)`, `StudyType`, `StudyKey`.
- **Tests:** `tests/VpHelperTests`.

4.15 Volume-profile bases (`_VolumeProfileBase`, `_VolumeProfileFuturesBase`)

- **Purpose:** MC volume profiles with POC/VA/HVN/LVN, naked POCs, up to three VWAP slots, confluence zones and targets, and the Phase 3 helpers (range, weekly and bar ladders, profile stats). The futures base adds the CME clock, IB/RTH stats, VPF-5 order flow and the CME holiday hooks.
- **Enable:** derive from `_VolumeProfileFuturesBase`. These bases do expose `[Input]`s:
- `HvnPercentile .8`, `LvnPercentile .2`, `MinNodeTicks 3`, `TrackNakedPoc true`, `NakedPocTickTolerance 1`;
- `Vwap1..3 Type/Length/FromDate/DataStream` (Vwap1 = Session on DS1; 2 and 3 Disabled);
- `DevelopingSource`, `DevProfileSourceMode`, `DevProfileDataStream`, `ValueAreaMethod`, `LookaheadAudit false`.
- **Main API:**
- Profiles: `ExtraVolumeProfileForBar([ds,] barIndex)` (1-based), `RecentCompletedProfiles(ds, count, asOfBar)`, `NakedPocs(ds[, asOf])`, `IsNewVolumeProfile(...)`.
- Confluence: `GetConfluenceZones`, `GetTopConfluenceZones([n])`, `GetConfluenceTargets(price, EVpDirection[, n])`, `GetConfluenceTargetZones`.
- VWAP slots: `CurrentVwap1..3`, `CurrentVwap1Sigma`, `Vwap1Active`.
- Auction and value: `ValueRelationship`, `IsPoorHigh`/`IsPoorLow`, `HasUnfinishedAuction`, `SinglePrints`, `ProfileShape`, `ValueMigrationScore`/`ValueMigrationState`, `Composite(stream, n)`, `IsBalanced`, `BalanceRange`.
- Futures base profiles: `RthProfile()`, `GlobexProfile()`, `PriorRthProfile()`.
- Range profiles: `RangeProfile(stream, t0, t1)`, `RangeProfileAfter`, `RangeProfileBars`, `OpenRange`/`CloseRange`/`RangeHandleProfile`, `RangeStatsOf`, `RangeLevels`.
- Weekly: `Week(weeksAgo)`, `WeeklyComposite(n)`, `CompletedWeeks`, `WeekKey`, `WeekLevels`.
- Ladders: `Ladder(ago[, minQuality])`, `LadderAt(time)`, `HtfLadder(ds, ago)`, `LadderProfile(b)`.
- Other: `ProfileEntropy(e, mmN)`, `SessionBars(day, rthOnly)`.
- **Order flow (futures base):**
- Enable with `OrderFlowEnabled = true`; configure `FlowCfg` (`_OrderFlow.FlowConfig`) and `FlowTickSource`.
- API: `Flow`, `Footprint(barsAgo)`, `BarDelta`, `CumDelta`, `DiagonalImbalances`, `StackedImbalances`, `Absorption`.
- **Phase 3 configs:** `RangeProfiles`, `WeeklyProfiles`, `ProfileStats`, `BarLadders` (`enabled false` by default; for the defaults see the §1 table sources and the `configs` in `_RangeProfile`/`_WeeklyProfiles`/`_ProfileStats`/`_BarLadder`).
- **Journal:**
- `VPB-1 own profile: source() mode() approximate() valueArea() audit();`
- `RP build {tag} Data{n} [t0..t1] bars(a/b) poc() va() approx() roll();`
- `RP coverage …;`
- `LADDER HTF Data{n} {t} coverage() -> totals-only`.
- **Tests:** `tests/ProfileTests`, `tests/VpHelperTests`, `tests/OrderFlowTests`.
- **Note:** your strategies inherit `_CustomSignal` directly, so this chapter applies only if you move a strategy onto a VP base.

4.16 M-PORT publisher and `_PortfolioRiskManager` (`_PortMM`)

- **Purpose:** publish `pm.*` keys to `MyPortfolioData` in Portfolio Trader. A money-management signal then sizes and limits risk across strategies.
- **Enable:**
- The publisher is automatic in PT (`PortfolioPublish`).

- Override as needed: `PortfolioFamily`, `PortfolioScore`, `PortfolioHalted`, `PortfolioCluster` (default from the symbol root: ES→EQIDX...), `PortfolioRegime`.
- Apply `_PortfolioRiskManager` as the portfolio's Money Management signal. It has no `[Input]`s; subclass it and override `Config` or `ConfigPath` (JSON).
- **Keys (`_PortKeys`):**
- Identity: `pm.ver`, `pm.id`, `pm.family`, `pm.cluster`.
- Equity and returns: `pm.eq`, `pm.pnl`, `pm.ret`, `pm.uRet`, `pm.vol`.
- Risk: `pm.R$`, `pm.qtyCode`, `pm.riskNew`, `pm.riskOpen`, `pm.openR`.
- State: `pm.state`, `pm.side`, `pm.regime`, `pm.regimeP`, `pm.score`, `pm.stamp`, `pm.day`, `pm.pos`.
- Events: `pm.evt*`.
- Written by the MM: `pm.mult`, `pm.heat`, `pm.dd`, `pm.rho`.
- **`EPortState`:** `{Flat, Armed, Long, Short, Exiting, Halted}`.
- **`PortMmConfig` defaults:**
- Sizing: `sizing RiskParity`, `bookVolTarget 0.10`, `riskPerTrade 0.005`.
- Risk caps: `heatCap 0.04`, `clusterCap 0.02`, `rhoMerge 0.9`, `rhoWin 60`, `rhoMinDays 40`, `maxPositions 6`.
- Drawdown: `ddSteps {5%→0.5, 10%→0.25, 15%→0}`, `ddRecoverHighDays 20`.
- Loss stops and kills: `dailyLossStop 0.02`, `dailyLossFlatten 0.03`, `stratKillR -8`, `stratKillTrades 20`.
- Other: `staleBars 3`, `minContracts 1`, `idmCap 2.5`.
- **API in your strategy:** `portfolioArm(int side, double entry, double stop, int qtyCode)` (called by `sizeForRisk` for Data1), `portfolioStop(double)`.
- **Journal:** Info `PORT publish on: id() family() cluster()`; Journal `PORT book multiplier x at t`. MM lines go to Output (`PORT Warning/Info`).
- **Tests:** `tests/PortMMTests`. Live checks use `_TestPortArm` + `_TestPortMM`.

4.17 A2-33 level study (`_LevelStudy`) and Gate 0 (`_Gate0`)

- **Level study:**
- Enable with `LevelStudyConfig.enabled = true`.
- Defaults: `controls 20`, `distLo .9`, `distHi 1.1`, `reactionBars 60`, `sigmaBars 60`, `touchTicks 0`, `maxAge 15`, `deathR 1`, `seed 33`, `from/to`, `csvPath ""`, `controlsEitherSide false`, `reactionAfterTouch false`.
- Register prior-session levels on the session's first bar with `levelStudyAdd(type, price, ageSessions, quality = NaN, key = null)`.
- Results: Info `A2-33 RESULT events() types() window()` plus a summary per type at RUN-END, and an optional CSV.
- No strategy on `main` enables `LevelStudyConfig`: `_Gate0` drives `_LevelStudy` directly. The catalogue and the `LevelStudyConfig` comment in `_CustomSignal` name `_Test1`, but `_Test1` is a plain `SignalObject` (discrepancy, Appendix F).
- **Gate 0** (`Gate0Engine`, `EGate0Mode/Family/Canary`, `Gate0Bar/Intent/Profile/Vwap/Atr/Session`): an MC-free research harness, hosted by `_TestGate0`. Its use is covered in **§2**.
- **Tests:** `tests/LevelStudyTests`, `tests/Gate0Tests`.

4.18 Utility engines

- `_Gmm`: `Fit`, `Fit1D`, `FitK`, `FitOnce`, `LogDensity`, `Density1D`, `Order`, `ParamCount`; `GmmModel`, `GmmFitConfig`.
- `_Rng`: deterministic `Mix`, `Seed(params long[])`, `Hash(string)`.
- `_ProfileStats`: `Entropy`, `MmN`, `KL`, `KLKde*`, `KdeOnGrid`, `ParkinsonPerBar`, `ParkinsonSessionPrice`, `EwmaForecast`, `AggregateBars`, `WilderAtr`; `GmmSession.FromBars`.
- `_SignalJournal`: a CS-0 probe description only (`PROBE_ID`, `Describe()`).

Appendices

A. `ETraceLevel`

Level	Value	Enables
None	0	nothing (the default after `Create`)
Error	1	errors (for example the Event coverage failure)
Warning	2	+ one-time "engine queried but not enabled" warnings, config warnings
Info	3	+ RUN / RUN-END, `entry blocked:`, roll detected, XMKT leg, PORT publish
Journal	4	+ engine facts, `MCJournal_<runId>.csv`, `MCEvents_<runId>.csv`
Verbose	5	+ equity and account details
Debug	6	+ engine debug sinks (for example PA `trace`), sizing reasons

B. Files and formats

All files go to `%LOCALAPPDATA%\MCSignalLogs\` (fallback: `%TEMP%`).

- **Trace log** `MCSignal{yyyyMMdd-HHmmss}-{pid}.csv`, one per process. Columns: `level,yyyy-MM-dd HH:mm:ss {zone},signalName,symbol,runId,message`.
- **Trade journal** `MCJournal_<runId>.csv`, with the header: `runId,strategy,symbol,tag,side,qty,entryTime,entryPrice,exitTime,exitPrice,exitOrder,barsHeld,profit,com` plus `,evtDayTier,evtMinSince` when M-EVENT is on. The `context` column holds the `journalArm` pairs as `k=v|k=v`.
- **Signal events** `MCEvents_<runId>.csv`: `runId,strategy,symbol,tag,direction,time,bar,price,stop,riskPerUnit,taken,controlMode`, then `,mfeR_{h},maeR_{h}` for each horizon h, then `,completedBy,context` (exact header checked in `_CustomSignal`). The context is `k=v;`.
- **Engine line prefixes:** `DAILY`, `REGIME`, `REGGATE`, `EVENT`, `AV`, `PA{ds}[-role]`, `TOD`, `XMKT`, `TICK`, `MTF`, `RING`, `RP`, `LADDER`, `VPB-1`, `VPF-4`, `VPF-7`, `PORT`, `A2-33`.

C. Running the tests

All tests need the .NET 8 SDK. Each `tests/<Name>/run.sh` does `dotnet run -c Release` in its folder. Where a `gen.sh` exists, it generates `Generated/*.cs` from the real study sources (for example it extracts the `// <calendar-rules …>` blocks), and the build runs it itself.

```
cd MultiCharts.Net
for t in tests/*/run.sh; do echo "== $t"; bash "$t" || break; done
bash tests/RegimeTests/run.sh          # also supports the reference.py export path
bash tests/EventTests/gen.sh            # regenerate only
```

Suites: Avwap, Calendar, Daily, Event, Gate0, Gmm, LevelStudy, Mtf, OrderFlow, PortMM, PriceAction, Profile, RegimeGate, Regime, Roll, SessionClock, Tick, Tod, VpHelper, Xmkt. `gen.sh` exists in Calendar, Daily, Event, Gate0, Gmm, OrderFlow, Profile and RegimeGate.

D. Glossary

- **AsOf**: the date whose completed data produced a value. It is usable strictly after that date.
- **AVWAP**: anchored VWAP from an event bar.
- **BOS / CHoCH**: break of structure / change of character (a BOS against the prior structure).
- **Closed bar**: `Bars.Status == EBarState.Close`. Engines commit only on these.
- **Globex date**: the CME trading date. The 18:00 ET evening session belongs to the next date.

- **Half-day**: a CME early RTH close (`RthCloseEt`, 13:15 ET) or a holiday halt.
- **HTF**: a higher-timeframe stream of the same instrument (`HtfStreams`).
- **Leg**: a cross-market stream (`XmktLegs`).
- **No-RTH day**: Globex is open but there is no cash session (an NYSE holiday).
- **Placebo (P1/P2)**: an AVWAP twin with neutralised volume, used for falsification.
- **Roll splice**: the first bar of a new contract on a continuous unadjusted series.
- **RTH value window**: `(09:30, 16:00]` ET, or up to the early close.
- **Tier**: event importance (T1 > T2 > T3, C = calendar).
- **TOD**: time-of-day normalisation per session-minute bucket.

E. Public signatures by file and class

All 2174 public/protected declarations extracted from `origin/main` 629da0d, in source order, with the source line. Long lines wrap in the PDF.

`_AvwapManager.Function.CS`

(file scope)

```
/* 15 */ public enum EAvwapAnchor { None = 0, SwingHigh = 1 << 0, SwingLow = 1 << 1, Breakout = 1 << 2,
GapOpen = 1 << 3, VolumeClimax = 1 << 4, WeekOpen = 1 << 5, MonthOpen = 1 << 6, QuarterOpen = 1 << 7,
NewsBar = 1 << 8, Swing = SwingHigh | SwingLow }
/* 30 */ public enum EAvwapPeriod { Week, Month, Quarter }
/* 33 */ public static class _AvwapCalendar
/* 51 */ public enum EAvwapPlacebo { None = 0, P1 = 1, P2 = 2 }
/* 54 */ public sealed class AvwapLimits
/* 90 */ public struct AvwapBar
/* 101 */ public struct AvwapPaEvents
/* 110 */ public sealed class _Avwap
/* 149 */ internal sealed class _AvwapManager
```

_AvwapCalendar

```
/* 36 */ public static long PeriodKey(EAvwapPeriod p, DateTime sessionDate)
```

AvwapLimits

```
/* 56 */ public int maxAnchors = 6;
/* 57 */ public int maxAgeSessions = 10;
/* 58 */ public bool replaceSameType = true;
/* 59 */ public double[] bandSigmas = { 1, 2, 3 };
/* 60 */ public int slopeBars = 5;
/* 61 */ public int dataStream = 1;
/* 62 */ public double gapAtrFraction = 0.5;
/* 63 */ public double gapFallbackTrMultiple = 3;
/* 64 */ public double climaxZ = 3;
/* 65 */ public int fallbackLength = 20;
/* 66 */ public int backfillBars = 2;
/* 67 */ public EAvwapPlacebo placebos = EAvwapPlacebo.None;
/* 68 */ public int p2MinPool = 10;
/* 70 */ public string Validate()
```

AvwapBar

```
/* 92 */ public int bar;
/* 93 */ public DateTime time;
/* 94 */ public double o, h, l, c, v;
/* 95 */ public DateTime sessionDate;
/* 96 */ public bool rth;
/* 97 */ public int slot;
```

AvwapPaEvents

```
/* 103 */ public bool swingHigh, swingLow;
/* 104 */ public int swingHighBar, swingLowBar;
/* 105 */ public DateTime swingHighTime, swingLowTime;
/* 106 */ public bool breakoutUp, breakoutDown;
```

_Avwap

```
/* 112 */ internal double sPV, sP2V, sV;
/* 113 */ internal double tS, tS2; internal int tN;
/* 114 */ internal double qPV, qP2V, qV;
/* 115 */ internal bool p2Journaled;
/* 117 */ public double P1 { get; internal set; }
/* 118 */ public double P1Sigma { get; internal set; }
/* 120 */ public double P2 { get; internal set; }
/* 121 */ public double P2Sigma { get; internal set; }
/* 123 */ public bool p2Degraded { get; internal set; }
/* 125 */ public string TagFor(EAvwapPlacebo arm)
/* 126 */ internal double[] hist; internal int histCount, histNext;
/* 127 */ public int Id { get; internal set; }
/* 128 */ public EAvwapAnchor Type { get; internal set; }
/* 129 */ public DateTime AnchorTime { get; internal set; }
/* 130 */ public int AnchorBar { get; internal set; }
/* 131 */ public DateTime UsableFrom { get; internal set; }
/* 132 */ public int Direction { get; internal set; }
/* 133 */ public double Value { get; internal set; }
/* 134 */ public double Sigma { get; internal set; }
/* 135 */ public double SlopeTicks { get; internal set; }
/* 136 */ public double SlopeAtr { get; internal set; }
/* 137 */ public int AgeBars { get; internal set; }
/* 138 */ public int AgeSessions { get; internal set; }
/* 139 */ public bool Touched { get; internal set; }
/* 140 */ public int TouchCount { get; internal set; }
/* 141 */ public bool Expired { get; internal set; }
/* 142 */ public string ExpireReason { get; internal set; }
/* 144 */ public string Tag { get; }
/* 145 */ public double Band(double k)
/* 146 */ public bool Valid { get; }
```

_AvwapManager

```
/* 151 */ public AvwapLimits limits = new AvwapLimits();
/* 152 */ public EAvwapAnchor events = EAvwapAnchor.None;
/* 153 */ public double tickSize = 0.25;
/* 154 */ public string label = "AV";
/* 155 */ public Action<string> journal;
/* 156 */ public Action<string> trace;
/* 157 */ public Action<string> warnOnce;
/* 158 */ public Func<double> dailyAtr;
/* 159 */ public Func<double> volumeZ;
/* 161 */ public Func<int, IList<double>> volumePool;
/* 163 */ public bool Configured { get; private set; }
/* 164 */ public int LastBar { get; }
/* 183 */ public static string P2PoolError(EAvwapPlacebo placebos, int dataStream, bool todRunning)
/* 191 */ public string Configure()
/* 204 */ public void Reset()
/* 216 */ public IReadOnlyList<_Avwap> Anchors { get; }
/* 218 */ public _Avwap Latest(EAvwapAnchor type)
/* 225 */ public double FallbackTrAvg { get; }
/* 228 */ public void OnRoll(DateTime barTime)
/* 239 */ public bool Add(AvwapBar b, AvwapPaEvents pa)
```

`_BarLadder.Function.CS`

(file scope)

```
/* 16 */ public sealed class LadderBar
/* 29 */ public sealed class BarLadderConfig
/* 46 */ public sealed class _BarLadderStore
```

LadderBar

```
/* 18 */ public int bar; public DateTime time; public double tick;
/* 19 */ public _OrderFlow.FlowSource source = _OrderFlow.FlowSource.Unavailable;
/* 20 */ public _OrderFlow.FlowQuality quality = _OrderFlow.FlowQuality.Unknown;
/* 21 */ public long[] rows = new long[0]; public double[] bid = new double[0], ask = new double[0], total =
new double[0];
/* 22 */ public double High, Low, Buy, Sell, Volume, coverage = 1;
/* 23 */ public bool crossesRoll;
/* 24 */ public bool HasBidAsk { get; }
/* 25 */ public bool HasRows { get; }
/* 26 */ public double Delta { get; }
```

BarLadderConfig

```
/* 31 */ public bool enabled = false;
/* 32 */ public int keepBars = 2000;
/* 33 */ public int[] htfStreams = new int[0];
/* 34 */ public _OrderFlow.FlowQuality minQuality = _OrderFlow.FlowQuality.Low;
/* 36 */ public string Validate()
/* 43 */ public string Canon()
```

_BarLadderStore

```
/* 48 */ public readonly BarLadderConfig config;
/* 49 */ public readonly double tick;
/* 53 */ public _BarLadderStore(BarLadderConfig cfg, double tick)
/* 55 */ public int Count { get; }
/* 59 */ public LadderBar Get(RingBar rb, _OrderFlow.FootprintBar fp, bool final = true)
/* 70 */ public bool TryGet(int bar, out LadderBar lb)
/* 71 */ public void Clear()
/* 74 */ public static LadderBar Build(RingBar rb, _OrderFlow.FootprintBar fp, double tick)
/* 92 */ public static LadderBar BarUpDown(RingBar rb, double tick)
/* 111 */ public static LadderBar Merge(IList<LadderBar> parts, IList<RingBar> ringBars, int expectedSlots, i
nt htfBar, DateTime htfTime, double tick)
/* 181 */ public static bool Dense(LadderBar lb, out double[] prices, out double[] vols)
```

`_BarRing.Function.CS`

(file scope)

```
/* 14 */ public struct RingBar
/* 21 */ public sealed class BarRingConfig
/* 30 */ public sealed class _BarRing
```

RingBar

```
/* 16 */ public int bar; public DateTime time; public DateTime tradingDay;
/* 17 */ public double o, h, l, c, v, up, down;
/* 18 */ public bool rth; public bool rollBar;
```

BarRingConfig

```
/* 24 */ public int capacityBars = 0;
/* 25 */ public const int MaxAutoBars = 40000;
/* 27 */ public string Canon()
```

_BarRing

```
/* 37 */ public _BarRing(int capacity)
/* 39 */ public int Capacity { get; }
/* 40 */ public int Count { get; }
/* 42 */ public long Added { get; }
/* 43 */ public bool Dropped { get; }
/* 44 */ public int LastBarNumber { get; }
/* 47 */ public bool Add(RingBar b)
/* 62 */ public void Clear()
/* 65 */ public RingBar this[int ago]
```

```
/* 76 */ public RingBar At(int idx)
/* 79 */ public int IndexAtOrBefore(DateTime t)
/* 91 */ public int FirstIndexAtOrAfter(DateTime t, bool exclusive = false)
/* 104 */ public bool Covers(DateTime t0)
/* 112 */ public List<RingBar> Window(DateTime t0, DateTime t1, bool t0Exclusive = false)
/* 121 */ public List<RingBar> Session(DateTime tradingDay, bool rthOnly = false)
```

`_CmeHolidays.Function.CS`

(file scope)

```
/* 13 */ public sealed class _CmeHolidays : _Holidays
```

_CmeHolidays

```
/* 32 */ public static class Rules
/* 153 */ public _CmeHolidays(CStudyControl _master)
/* 154 */ public _CmeHolidays(CStudyControl _master, int _ds)
/* 156 */ protected override void refreshHolidays(int year)
```

_CmeHolidays.Rules

```
/* 34 */ public struct Day
/* 43 */ public static readonly TimeSpan HolidayHaltEt = new TimeSpan(13, 0, 0);
/* 44 */ public static readonly TimeSpan EarlyCloseEt = new TimeSpan(13, 15, 0);
/* 45 */ public static readonly TimeSpan GoodFridayCloseEt = new TimeSpan(9, 15, 0);
/* 46 */ public static readonly TimeSpan MourningCloseEt = new TimeSpan(9, 30, 0);
/* 68 */ public static Day Get(DateTime date)
/* 107 */ public static List<DateTime> ClosedDates(int year)
/* 119 */ public static DateTime SundayToMonday(DateTime d)
/* 121 */ public static DateTime Observed(DateTime d)
/* 127 */ public static DateTime NthWeekday(int year, int month, DayOfWeek day, int n)
/* 134 */ public static DateTime LastWeekday(int year, int month, DayOfWeek day)
/* 141 */ public static DateTime Easter(int year)
```

_CmeHolidays.Rules.Day

```
/* 36 */ public bool Closed;
/* 37 */ public bool NoRth;
/* 38 */ public TimeSpan? HaltEt;
/* 39 */ public TimeSpan? RthCloseEt;
/* 40 */ public string Name;
```

`_CmeSessionClock.Function.CS`

(file scope)

```
/* 24 */ public sealed class _CmeSessionClock
```

_CmeSessionClock

```
/* 29 */ public enum MarketSession { None = 0, Asia = 1 << 0, Europe = 1 << 1, NewYork = 1 << 2, Rth = 1 <<
3, Premarket = 1 << 4, PostMarket = 1 << 5 }
/* 41 */ public enum GlobexState { Closed, Halt, Open }
/* 49 */ public static readonly TimeSpan DefaultAsiaStart = new TimeSpan(18, 0, 0);
/* 50 */ public static readonly TimeSpan DefaultAsiaEnd = new TimeSpan(3, 0, 0);
/* 51 */ public static readonly TimeSpan DefaultEuropeStart = new TimeSpan(3, 0, 0);
/* 52 */ public static readonly TimeSpan DefaultEuropeEnd = new TimeSpan(11, 30, 0);
/* 53 */ public static readonly TimeSpan DefaultNewYorkStart = new TimeSpan(8, 0, 0);
/* 54 */ public static readonly TimeSpan DefaultNewYorkEnd = new TimeSpan(17, 0, 0);
/* 55 */ public static readonly TimeSpan DefaultRthStart = new TimeSpan(9, 30, 0);
/* 56 */ public static readonly TimeSpan DefaultRthEnd = new TimeSpan(16, 15, 0);
/* 57 */ public static readonly TimeSpan DefaultRthValueEnd = new TimeSpan(16, 0, 0);
/* 58 */ public static readonly TimeSpan DefaultHaltStart = new TimeSpan(17, 0, 0);
/* 59 */ public static readonly TimeSpan DefaultHaltEnd = new TimeSpan(18, 0, 0);
/* 60 */ public static readonly TimeSpan DefaultGlobexOpen = new TimeSpan(18, 0, 0);
```

```
/* 61 */ public static readonly TimeSpan DefaultGlobexClose = new TimeSpan(17, 0, 0);
/* 71 */ public TimeSpan AsiaStart = DefaultAsiaStart, AsiaEnd = DefaultAsiaEnd;
/* 72 */ public TimeSpan EuropeStart = DefaultEuropeStart, EuropeEnd = DefaultEuropeEnd;
/* 73 */ public TimeSpan NewYorkStart = DefaultNewYorkStart, NewYorkEnd = DefaultNewYorkEnd;
/* 74 */ public TimeSpan RthStart = DefaultRthStart, RthEnd = DefaultRthEnd;
/* 76 */ public TimeSpan RthValueEnd = DefaultRthValueEnd;
/* 77 */ public TimeSpan HaltStart = DefaultHaltStart, HaltEnd = DefaultHaltEnd;
/* 78 */ public TimeSpan GlobexOpen = DefaultGlobexOpen, GlobexClose = DefaultGlobexClose;
/* 81 */ public bool UseCmeHolidayTable = true;
/* 84 */ public Func<DateTime, bool> GlobexClosedDayHook;
/* 86 */ public Func<DateTime, bool> NoRthDayHook;
/* 88 */ public Func<DateTime, TimeSpan?> HolidayHaltHook;
/* 90 */ public Func<DateTime, TimeSpan?> RthEarlyCloseHook;
/* 106 */ public _CmeSessionClock(TimeZoneInfo barTimeZone = null)
/* 120 */ public TimeZoneInfo Eastern { get; }
/* 123 */ public TimeZoneInfo BarTimeZone
/* 136 */ public void Invalidate()
/* 142 */ public DateTime ToEastern(DateTime barTime)
/* 156 */ public DateTime EasternOf(DateTime barTime)
/* 157 */ public GlobexState State(DateTime barTime)
/* 159 */ public MarketSession Sessions(DateTime barTime)
/* 160 */ public bool InSession(DateTime barTime, MarketSession session)
/* 161 */ public bool InAny(DateTime barTime, MarketSession sessions)
/* 164 */ public DateTime GlobexDate(DateTime barTime)
/* 171 */ public GlobexState StateEt(DateTime et)
/* 200 */ public MarketSession SessionsEt(DateTime et)
/* 229 */ public DateTime GlobexDateEt(DateTime et)
/* 237 */ public MarketSession PrimarySessionEt(DateTime et)
/* 250 */ public TimeSpan TimeSinceOpenEt(DateTime et, MarketSession session)
/* 258 */ public int MinutesSinceOpenEt(DateTime et, MarketSession session)
/* 264 */ public TimeSpan TimeUntilCloseEt(DateTime et, MarketSession session)
/* 272 */ public int MinutesUntilCloseEt(DateTime et, MarketSession session)
/* 278 */ public TimeSpan SessionLength(MarketSession session)
/* 287 */ public bool IsOpeningRangeEt(DateTime et, MarketSession session, int minutes)
/* 293 */ public bool IsRthAfterInitialBalanceEt(DateTime et, int ibMinutes)
/* 302 */ public bool TrySessionBoundsEt(DateTime et, MarketSession session, out DateTime open, out DateTime
close)
/* 329 */ public bool TryWindow(MarketSession session, out TimeSpan start, out TimeSpan end, out bool wraps)
/* 348 */ public TimeSpan RthValueEndEt(DateTime globexDate)
/* 355 */ public bool IsRthValueBarEt(DateTime et)
/* 364 */ public bool IsGlobexClosedDay(DateTime globexDate)
/* 369 */ public bool IsNoRthDay(DateTime globexDate)
/* 375 */ public TimeSpan? HolidayHaltEt(DateTime globexDate)
/* 381 */ public TimeSpan? RthEarlyCloseEt(DateTime globexDate)
/* 391 */ public static bool DefaultGlobexClosedDay(DateTime globexDate, bool useCmeHolidayTable)
/* 403 */ public static bool InWindow(TimeSpan t, TimeSpan start, TimeSpan end, bool wraps)
/* 409 */ public static bool IsSingleFlag(MarketSession session)
```

`_CustomSignal.Strategy.CS`

(file scope)

```
/* 17 */ public abstract class _CustomSignal : PortfolioSignalObject
```

_CustomSignal

```
/* 23 */ public const double DEFAULT_ACCOUNT_RECALC_INTERVAL = 5;
/* 24 */ public const int DEFAULT_ENTRY_EXIT_LOOKBACK = 63;
/* 27 */ public const string BUILD_TAG = "base-class-rev2";
/* 34 */ public enum ETraceLevel { None = 0, Error, Warning, Info, Journal, Verbose, Debug, }
/* 49 */ public enum EUseFill { First, Last, }
/* 56 */ [Input] public virtual ETraceLevel TraceLevel { get; set; }
/* 233 */ public int IntradayBarCount
/* 238 */ public double IntradayLow
/* 243 */ public DateTime IntradayLowTime
/* 248 */ public double IntradayHigh
/* 253 */ public DateTime IntradayHighTime
/* 259 */ public double IntradayClose
/* 264 */ public DateTime IntradayCloseTime
/* 269 */ public double IntradayOpen
/* 274 */ public DateTime IntradayOpenTime
/* 280 */ public double IntradayVolume
/* 285 */ public Function._Holidays Holidays { get; }
```

```
/* 297 */ public string RunId { get; }
/* 300 */ protected virtual string BuildTag { get; }
/* 302 */ protected ITradingData tradingData
/* 350 */ public _CustomSignal(object _ctx)
/* 359 */ protected bool traceOn(ETraceLevel level)
/* 369 */ public void trace(ETraceLevel traceLevel, string message)
/* 384 */ protected PriceActionConfig PriceActionConfig { get; }
/* 389 */ internal _PriceActionEngine PriceActionEngine
/* 403 */ protected List<PriceActionConfig> ExtraPriceAction { get; }
/* 410 */ protected virtual void presetExtraPriceAction(PriceActionConfig c)
/* 414 */ internal _PriceActionEngine PriceActionFor(int dataStream, string role = null)
/* 495 */ protected virtual bool priceActionRequired()
/* 498 */ protected EAvwapAnchor AvwapEvents { get; set; }
/* 500 */ protected AvwapLimits AvwapLimits { get; }
/* 506 */ internal _AvwapManager AvwapManager
/* 516 */ protected virtual bool isRthBar(DateTime barTime)
/* 521 */ protected ETodMetric TodMetrics { get; set; }
/* 523 */ protected int TodSessions { get; set; }
/* 525 */ protected int TodMinSessions { get; set; }
/* 527 */ protected int TodBucketMinutes { get; set; }
/* 533 */ internal _TodNormaliser TodNormaliser
/* 544 */ protected virtual int todMinuteOfSession(DateTime barTime)
/* 546 */ protected virtual bool todHalfDay(DateTime sessionDate)
/* 549 */ protected virtual double todDelta(out int tier)
/* 586 */ protected bool NeedsDaily { get; set; }
/* 588 */ protected DailyConfig DailyConfig { get; }
/* 598 */ internal _DailyContext DailyContext
/* 608 */ protected double DailyGapPct { get; }
/* 686 */ protected RegimeConfig RegimeConfig { get; }
/* 692 */ protected RegimeState Regime
/* 738 */ protected EventConfig EventConfig { get; }
/* 745 */ internal _EventCalendar EventCalendar
/* 754 */ protected bool EventBlackout { get; }
/* 756 */ protected bool EventFlattenNow { get; }
/* 827 */ protected List<XmktLegConfig> XmktLegs { get; }
/* 833 */ internal _XmktLeg XmktLeg(string role)
/* 841 */ internal _PriceActionEngine XmktLeaderSwing(string role)
/* 914 */ protected TickConfig TickConfig { get; }
/* 921 */ internal TickBarStats TickStats { get; }
/* 995 */ protected LevelStudyConfig LevelStudyConfig { get; }
/* 1026 */ protected bool levelStudyAdd(string type, double price, int ageSessions, double quality = double.N
aN, string key = null)
/* 1055 */ protected BarRingConfig BarRings { get; }
/* 1057 */ protected FvgConfig Fvgs { get; }
/* 1073 */ protected void requireRing(int ds, int bars, string consumer)
/* 1086 */ protected _BarRing Ring(int ds)
/* 1100 */ protected bool IsRollWindow(DateTime tradingDay)
/* 1103 */ protected RingBar ringBarOf(int ds, int ago = 0)
/* 1159 */ internal _FvgDetector FvgDetector
/* 1169 */ protected LevelRef fvgLevel(Fvg f, bool startEdge)
/* 1180 */ protected virtual bool PortfolioPublish { get; }
/* 1182 */ protected virtual string PortfolioFamily { get; }
/* 1185 */ protected virtual int PortfolioRegime
/* 1195 */ protected virtual double PortfolioScore { get; }
/* 1197 */ protected virtual bool PortfolioHalted { get; }
/* 1199 */ protected virtual string PortfolioCluster
/* 1246 */ protected void portfolioArm(int side, double entry, double stop, int qtyCode)
/* 1261 */ protected void portfolioStop(double stop)
/* 1325 */ protected int[] HtfStreams { get; set; }
/* 1330 */ protected int HtfClosedIndex(int ds, int ago = 0)
/* 1332 */ protected double HtfClosed(int ds, int ago = 0)
/* 1334 */ protected void HtfClosedBar(int ds, int ago, out DateTime time, out double open, out double high,
out double low, out double close, out double volume)
/* 1341 */ protected double HtfForming(int ds)
/* 1351 */ protected int htfMapped(int ds, int maxBack = 2)
/* 1386 */ public enum ERollDetection { Off = 0, Series = 1, Table = 2, SeriesOrTable = 3 }
/* 1389 */ protected ERollDetection RollDetection { get; set; }
/* 1394 */ protected int RollGapTicks { get; set; }
/* 1396 */ protected HashSet<DateTime> RollDates { get; }
/* 1403 */ public IList<int> RollBars { get; }
/* 1427 */ protected void stepRollDetector()
/* 1451 */ protected virtual bool rollOn(int ds)
/* 1463 */ protected Action<string> engineSink(ETraceLevel level)
/* 1593 */ protected override void StopCalc()
/* 1601 */ protected override void Create()
/* 1612 */ protected override void Destroy()
/* 1622 */ protected override void StartCalc()
```

```
/* 1676 */ protected override void CalcBar()
/* 1779 */ protected string runLine()
/* 1789 */ protected string inputSnapshot()
/* 1820 */ protected double tickSize()
/* 1846 */ protected bool journalOn()
/* 1853 */ protected void journalArm(string entryOrderName, double plannedEntry, double initialStop, double t
arget, IDictionary<string, string> context = null)
/* 1964 */ protected static void requireUnitInterval(string name, double value)
/* 1970 */ protected static void requireAtLeast(string name, int value, int min)
/* 1976 */ protected static void requireNonNegative(string name, double value)
/* 1982 */ protected static void requireNonNegative(string name, int value)
/* 1988 */ protected void requireDataStream(string name, int stream)
/* 1999 */ protected virtual DateTime tradingDayKey(DateTime barTime)
/* 2010 */ protected double equityAvailable(double maxCustomRiskEquityPerPosPercent = 0.0d)
/* 2082 */ public enum ETradingDayRoll { Calendar = 0, Auto = 1 }
/* 2088 */ protected ETradingDayRoll TradingDayRoll { get; set; }
/* 2093 */ protected TimeZoneInfo BarTimeZone
/* 2099 */ protected DateTime ToZone(DateTime barTime, string tzId)
/* 2111 */ protected bool inTimeWindow(TimeSpan start, TimeSpan end, string tzId)
/* 2118 */ protected DateTime autoTradingDay(DateTime barTime, string category)
/* 2129 */ protected TimeZoneInfo resolveBarTimeZone(IInstrumentSettings info)
/* 2151 */ protected virtual string exchangeTimeZoneId(string exchange)
/* 2162 */ protected static TimeZoneInfo findTz(string windowsId)
/* 2173 */ public enum EControlMode { None = 0, RandomEntry = 1, ShiftLevels = 2, DelayEntryBars = 3 }
/* 2176 */ protected int[] SignalHorizons { get; set; }
/* 2178 */ protected EControlMode ControlMode { get; set; }
/* 2179 */ protected int ControlSeed { get; set; }
/* 2181 */ protected double ControlRandomProbability { get; set; }
/* 2183 */ protected int ControlShiftTicks { get; set; }
/* 2185 */ protected int ControlDelayBars { get; set; }
/* 2207 */ protected void logSignal(string tag, int direction, double price, double stop,
IDictionary<string, string> context, bool taken)
/* 2284 */ protected double controlRandom(int bar)
/* 2297 */ protected bool controlRandomEntry()
/* 2303 */ protected double controlAdjustLevel(double level)
/* 2309 */ protected int controlAdjustDelayBars()
/* 2317 */ protected TimeSpan? NoEntryBefore { get; set; }
/* 2319 */ protected TimeSpan? NoEntryAfter { get; set; }
/* 2321 */ protected TimeSpan? FlatBy { get; set; }
/* 2323 */ protected int MaxTradesPerDay { get; set; }
/* 2325 */ protected int DailyLossStopCount { get; set; }
/* 2327 */ protected double DailyLossStopR { get; set; }
/* 2329 */ protected double RiskUnitDollars { get; set; }
/* 2331 */ protected int TimeStopBars { get; set; }
/* 2340 */ protected virtual TimeSpan sessionClock(DateTime barTime)
/* 2343 */ protected int TradesToday { get; }
/* 2344 */ protected int LosersToday { get; }
/* 2345 */ protected double NetProfitToday { get; }
/* 2385 */ protected bool entryAllowed(out string reason)
/* 2404 */ protected bool mustFlatten()
/* 2432 */ public struct SizingLimits
/* 2446 */ protected int sizeForRisk(IInstrument bars, double entry, double stop, double riskFraction, Sizing
Limits limits)
/* 2484 */ protected double equityAvailable(double fraction, bool includeOpenPnl)
/* 2501 */ protected double CostFilterMaxFractionOfR { get; set; }
/* 2507 */ protected virtual double roundTripCostPerContract()
/* 2513 */ protected bool costFilterPasses(IInstrument bars, double entry, double stop, out string reason)
/* 2532 */ protected static int applyRoundLots(int qty, bool useRoundLots)
/* 2551 */ protected int capByNotional(int qty, double unitNotional, double maxEquityFraction)
/* 2563 */ protected int equityBasedPositionSize(double entryPrice, double maxEquityPerPosPercent = 0.0d, boo
l useRoundLots = false)
/* 2588 */ protected int equityBasedPositionSize(IInstrument bars, double maxEquityPerPosPercent = 0.0d,
bool useRoundLots = false)
/* 2609 */ protected int riskBasedPositionSize(double risk, double maxRiskEquityPerPosPercent, bool useRoundL
ots = false)
/* 2641 */ protected int stopLossBasedPositionSize(IInstrument bars, double stopLossPrice, double maxRiskEqui
tyPerPosPercent = 0.0d, bool useRoundLots = false)
/* 2647 */ protected int stopLossBasedPositionSize(IInstrument bars, double entryPrice, double
stopLossPrice, double maxRiskEquityPerPosPercent, bool useRoundLots = false)
/* 2664 */ protected int volumeBasedPositionSize(IInstrument bars, int volumePeriod, int volumePeriodOffset,
double maxAvgVolumePercent, bool useRoundLots = false)
/* 2694 */ protected double averageTrueRange(IInstrument bars, int length)
/* 2711 */ protected bool isWithinAverageRange(IInstrument bars, int maLength, int atrLength, double atrFacto
r)
/* 2720 */ protected int calendarDayCount(DateTime entryDate, DateTime currentDateTime)
/* 2734 */ protected int tradingDayCount(DateTime entryDate, DateTime currentDateTime)
/* 2759 */ protected virtual bool isTradingHoliday(DateTime date)
```

```
/* 2769 */ protected Tuple<DateTime, double> recentLow(int daysBack, DateTime? earliestDateTime = null, bool
includeToday = true)
/* 2811 */ protected Tuple<DateTime, double> recentHigh(int daysBack, DateTime? earliestDateTime = null,
bool includeToday = true)
/* 2855 */ public DateTime orderEntryTime(params String[] entryOrderNames)
/* 2882 */ public DateTime orderEntryTimeUncached(params String[] entryOrderNames)
/* 2888 */ public enum EOrderNameMatch { Contains = 0, Exact = 1, Prefix = 2 }
/* 2891 */ protected static bool orderNameMatches(string orderName, string wanted, EOrderNameMatch mode)
/* 2903 */ public DateTime orderEntryTime(EOrderNameMatch matchMode, params String[] entryOrderNames)
/* 2964 */ public DateTime orderExitTime(EMarketPositionSide side, EUseFill useFill = EUseFill.Last)
/* 2976 */ public DateTime orderExitTime(EMarketPositionSide side, EUseFill useFill, DateTime notBefore, para
ms string[] exitOrderNames)
/* 3075 */ protected void refreshTradeManagerIfNeeded()
```

_CustomSignal.SizingLimits

```
/* 2434 */ public int maxQty;
/* 2435 */ public double maxNotionalFraction;
/* 2436 */ public double unitNotional;
/* 2437 */ public double minStopTicks;
/* 2438 */ public bool roundLots;
```

`_DailyContext.Function.CS`

(file scope)

```
/* 17 */ public enum EDailyClose { Bar = 0, RthValue = 1 }
/* 20 */ public sealed class DailyConfig
/* 53 */ internal sealed class _DailyContext
/* 309 */ internal sealed class _RthCloseSelector
```

DailyConfig

```
/* 22 */ public int dataStream = 0;
/* 23 */ public int weeklyStream = 0;
/* 24 */ public int atrLength = 20;
/* 25 */ public int smaLength = 200;
/* 26 */ public int rsiLength = 2;
/* 27 */ public int[] tsmomLookbacks = { 252, 126, 63 };
/* 29 */ public int[] hvLengths = { };
/* 30 */ public int hvAnnualisation = 252;
/* 31 */ public int hvPctLookback = 0;
/* 32 */ public int lbrRsiLength = 0;
/* 33 */ public EDailyClose closeBasis = EDailyClose.RthValue;
/* 34 */ public bool HvOn { get; }
/* 36 */ public string Validate()
```

_DailyContext

```
/* 55 */ public DailyConfig config = new DailyConfig();
/* 56 */ public string label = "DAILY";
/* 57 */ public Action<string> journal;
/* 58 */ public Action<string> warnOnce;
/* 69 */ public bool Configured { get; private set; }
/* 86 */ public double Hv(int n)
/* 88 */ public double HvPercentile(int n)
/* 99 */ public double Roc1 { get; }
/* 101 */ public double Lbr
/* 111 */ public double RthCloseAdj { get; }
/* 113 */ public double RthClosePrev { get; }
/* 117 */ public double Atr20d { get; }
/* 118 */ public double Sma200 { get; }
/* 119 */ public double Rsi2 { get; }
/* 120 */ public double PrevHigh { get; }
/* 121 */ public double PrevLow { get; }
/* 122 */ public double PrevClose { get; }
/* 123 */ public DateTime PrevDate { get; }
/* 124 */ public double PrevRange { get; }
/* 125 */ public double PrevWeekHigh { get; }
/* 126 */ public double PrevWeekLow { get; }
```

```
/* 128 */ public bool Valid { get; }
/* 130 */ public double Tsmom(int n)
/* 132 */ public double GapPct(double todayOpen)
/* 133 */ public int Days { get; }
/* 135 */ public string Configure()
/* 151 */ public void Reset()
/* 165 */ public void SetPrevWeek(double high, double low)
/* 171 */ public bool AddDay(DateTime date, double o, double h, double l, double c, bool firstDayOfNewContrac
t = false, double rollOffset = double.NaN, double rthClose = double.NaN)
/* 274 */ public static double HvOf(double[] ring, int next, int count, int n, int annualisation)
/* 288 */ public void OnRoll(DateTime date, double offset)
```

_RthCloseSelector

```
/* 311 */ public DateTime Day { get; private set; }
/* 312 */ public double Close { get; private set; }
/* 314 */ public double Observe(DateTime day, bool isRthValueBar, double close)
```

`_EventCalendar.Function.CS`

(file scope)

```
/* 27 */ public enum EEventKind : byte { None = 0, FomcDecision = 1, FomcPress = 2, FomcMinutes = 3, Cpi =
4, Ppi = 5, Nfp = 6, Jolts = 7, Claims = 8, Pce = 9, GdpAdvance = 10, GdpSecond = 11, GdpThird = 12, RetailSa
les = 13, IsmMfg = 14, IsmSvc = 15, JacksonHole = 16, ChairTestimony = 17, EiaPetroleum = 18,
QuarterlyExpiry = 30, RollWeek = 31, MonthlyOpex = 32, IndexRebalance = 33, HalfDay = 34, VixExpiry = 35, Rus
sellRecon = 36, Custom = 62 }
/* 42 */ public enum EEventTier : byte { None = 0, T3 = 1, T2 = 2, T1 = 3, C = 8 }
/* 46 */ public enum EEventCoverage : byte { NoEvent = 0, Event = 1, Unknown = 2 }
/* 48 */ public enum EEventFlags : ushort { None = 0, Sep = 1, Quad = 2, Unscheduled = 4, Scheduled = 8, Dela
yed = 16, Canceled = 32, Csv = 64, RuleOnly = 128, Pm = 256 }
/* 55 */ public enum EEventClassMask : byte { None = 0, Macro = 1, Calendar = 2, Energy = 4, All = 7 }
/* 58 */ public struct EventRec
/* 88 */ public struct EventDay
/* 102 */ public struct EventCorrection
/* 115 */ public sealed class EventConfig
/* 165 */ public static partial class EventRules
/* 476 */ public static partial class EventRules
/* 1092 */ internal sealed class _EventCalendar
```

EventRec

```
/* 60 */ public EEventKind kind;
/* 61 */ public EEventTier tier;
/* 62 */ public EEventFlags flags;
/* 63 */ public DateTime etTime;
/* 64 */ public DateTime etEnd;
/* 65 */ public DateTime utc;
/* 66 */ public DateTime tradingDay;
/* 67 */ public DateTime knownSinceUtc;
/* 68 */ public DateTime canceledSinceUtc;
/* 69 */ public string refPeriod;
/* 70 */ public string name;
/* 71 */ public string source;
/* 73 */ public bool IsSpan { get; }
/* 74 */ public EEventClassMask Class { get; }
/* 76 */ public static EventRec Make(EEventKind kind, DateTime et, EEventTier tier, EEventFlags flags = EEven
tFlags.None)
/* 84 */ public string KindName { get; }
```

EventDay

```
/* 90 */ public DateTime day;
/* 91 */ public EEventTier maxTier;
/* 92 */ public ulong kindsMask;
/* 93 */ public bool hasFomc, hasSep, isHalfDay, isRollWeek, isQuarterlyExpiry, isOpex, isRebalance;
/* 94 */ public TimeSpan? firstReleaseEt;
/* 95 */ public int count;
```

```
/* 96 */ public bool eiaRuleOnly;
/* 98 */ public bool Has(EEventKind k)
```

EventCorrection

```
/* 104 */ public EEventKind kind;
/* 105 */ public string refPeriod;
/* 106 */ public long originalEt;
/* 107 */ public long actualEt;
/* 108 */ public long announcedEt;
/* 109 */ public long canceledSinceEt;
/* 110 */ public EEventFlags flags;
/* 111 */ public string source;
```

EventConfig

```
/* 117 */ public bool enabled = false;
/* 118 */ public EEventClassMask classes = EEventClassMask.Macro | EEventClassMask.Calendar;
/* 119 */ public EEventTier minTier = EEventTier.T3;
/* 120 */ public Dictionary<EEventKind, EEventTier> tierOverride = new Dictionary<EEventKind, EEventTier>();
/* 121 */ public string csvPath = "";
/* 122 */ public bool allowPartialCoverage = false;
/* 123 */ public int blackoutPreMin = 0, blackoutPostMin = 0;
/* 124 */ public EEventTier blackoutTier = EEventTier.T1;
/* 125 */ public bool blackoutFlatten = false;
/* 126 */ public bool publish = false;
/* 128 */ public bool includeRussell = false;
/* 129 */ internal string csvSha = null;
/* 131 */ public static bool IsMacroTier(EEventTier t)
/* 134 */ public string Validate()
/* 149 */ public string Canon()
```

EventRules

```
/* 167 */ public static readonly TimeSpan T0830 = new TimeSpan(8, 30, 0), T0930 = new TimeSpan(9, 30, 0),
T1000 = new TimeSpan(10, 0, 0),
/* 176 */ public static readonly DateTime RollRuleMondayFrom = new DateTime(2022, 7, 1);
/* 216 */ public static EEventTier DefaultTier(EEventKind k)
/* 230 */ public static EEventClassMask ClassOf(EEventKind k, EEventTier tier)
/* 238 */ public static readonly EEventKind[] TableKinds =
/* 243 */ public static bool IsTableKind(EEventKind k)
/* 246 */ public static DateTime ThirdFriday(int y, int m)
/* 251 */ public static bool IsNyseHoliday(DateTime d)
/* 257 */ public static bool IsNyseBusinessDay(DateTime d)
/* 258 */ public static DateTime PrevBusinessDayOrSelf(DateTime d)
/* 259 */ public static DateTime NthBusinessDay(int y, int m, int n)
/* 263 */ public static bool IsFederalHoliday(DateTime d)
/* 276 */ public static DateTime OpexDate(int y, int m)
/* 281 */ public static DateTime ExpiryDate(int y, int m)
/* 283 */ public static DateTime RuleExpiryDate(int y, int m)
/* 286 */ public static DateTime RollDate(int y, int m)
/* 295 */ public static DateTime VixExpiry(int y, int m)
/* 303 */ public static DateTime FromYmdhm(long v)
/* 308 */ public static long ToYmdhm(DateTime d)
/* 311 */ public static List<EventRec> Generate(int y, bool includeRussell)
/* 378 */ public static long[] CoverageHoles()
/* 379 */ public static string HoleWhyFor(long kind, long fromEt, long toEt)
/* 385 */ public static List<EventRec> Table()
/* 397 */ public static EventCorrection[] Corrections { get; }
/* 400 */ public static string TableSha()
/* 405 */ public static string ExpectedTableSha { get; }
/* 406 */ public static int TableRowCount { get; }
/* 409 */ public static string TableIntegrity(int rows, string actualSha, string expectedSha)
/* 416 */ public static string ComputeTableSha(long[] et, byte[] kind, ushort[] flags, string[] refs, EventCo
rrection[] corr, long[] holes)
/* 424 */ public static string Sha256(byte[] b)
```

_EventCalendar

```
/* 1094 */ public EventConfig config;
/* 1095 */ public string label = "EVENT";
```

```
/* 1096 */ public Action<string> journal, warnOnce;
/* 1098 */ public Action<string> error;
/* 1100 */ public TimeSpan barDuration = TimeSpan.Zero;
/* 1102 */ public DateTime firstBarUtc = DateTime.MinValue;
/* 1104 */ public List<EventRec> table;
/* 1105 */ public EventCorrection[] corrections;
/* 1108 */ public long[] coverageHoles;
/* 1110 */ public string tableShaExpected;
/* 1112 */ sealed class Year
/* 1117 */ struct CsvRow
/* 1122 */ sealed class DayCache
/* 1136 */ struct CovSpan
/* 1148 */ public TimeZoneInfo Eastern { get; }
/* 1149 */ public DateTime NowUtc { get; }
/* 1150 */ public DateTime BarOpenUtc { get; }
/* 1151 */ public DateTime TradingDay { get; }
/* 1152 */ public bool Blackout { get; }
/* 1153 */ public bool FlattenNow { get; }
/* 1154 */ public bool DailyBars { get; }
/* 1155 */ public int RowsLoadedCsv { get; }
/* 1157 */ public bool Disabled { get; }
/* 1165 */ public string Configure()
/* 1260 */ public DateTime ToEt(DateTime utc)
/* 1264 */ public string LoadCsv(string text)
/* 1458 */ public DateTime TradingDayOf(DateTime utc)
/* 1472 */ public DateTime SessionOpenUtc(DateTime day)
/* 1483 */ public void Step(DateTime barTime, DateTime barOpenUtc, DateTime barCloseUtc, bool rollSplice)
/* 1566 */ public EEventCoverage Coverage(EEventKind kind, DateTime day)
/* 1576 */ public bool CoverageBlocks(DateTime day, EEventClassMask mask, EEventTier minTier, out EEventKind
kind)
/* 1583 */ public bool CoverageBlocks(DateTime day, EEventClassMask mask)
/* 1605 */ public List<string> CoverageReport(DateTime from, DateTime to, EEventClassMask mask, out bool flag
ged)
/* 1607 */ public List<string> CoverageReport(DateTime from, DateTime to, EEventClassMask mask, EEventTier mi
nTier, out bool flagged)
/* 1627 */ public List<string> CoverageStats()
/* 1661 */ public bool NextEvent(DateTime nowUtc, EEventTier minTier, EEventClassMask mask, out EventRec e)
/* 1673 */ public bool PrevEvent(DateTime nowUtc, EEventTier minTier, EEventClassMask mask, out EventRec e)
/* 1685 */ public bool NextEvent(EEventTier minTier, out EventRec e)
/* 1686 */ public bool PrevEvent(EEventTier minTier, out EventRec e)
/* 1689 */ public double MinutesTo(EEventTier minTier, EEventClassMask mask = EEventClassMask.Macro)
/* 1691 */ public double MinutesSince(EEventTier minTier, EEventClassMask mask = EEventClassMask.Macro)
/* 1693 */ public double MinutesSinceAt(DateTime nowUtc, EEventTier minTier, EEventClassMask mask = EEventCla
ssMask.Macro)
/* 1698 */ public bool IsInWindow(EEventTier minTier, int preMin, int postMin, EEventClassMask mask = EEventC
lassMask.Macro)
/* 1700 */ public bool IsInWindowAt(DateTime openUtc, DateTime closeUtc, EEventTier minTier, int preMin, int
postMin, EEventClassMask mask, out EventRec e)
/* 1742 */ public bool EventBlackout { get; }
/* 1743 */ public bool EventFlattenNow { get; }
/* 1746 */ public List<EventRec> EventsOn(DateTime day, EEventClassMask mask = EEventClassMask.All)
/* 1768 */ public List<EventRec> EventsOnExPost(DateTime day, bool includeCanceled = false, EEventClassMask m
ask = EEventClassMask.All)
/* 1780 */ public List<EventRec> EventsInRange(DateTime fromDay, DateTime toDay, EEventClassMask mask = EEven
tClassMask.All)
/* 1782 */ public List<EventRec> EventsInRangeExPost(DateTime fromDay, DateTime toDay, bool includeCanceled
= false, EEventClassMask mask = EEventClassMask.All)
/* 1786 */ public EventDay Classify(DateTime day)
/* 1787 */ public EventDay Classify(DateTime day, DateTime asOfUtc)
/* 1788 */ public EventDay ClassifyExPost(DateTime day)
/* 1826 */ public bool IsRollWeek(DateTime day)
/* 1832 */ public bool IsRollWeek()
/* 1834 */ public bool IsRollDay()
/* 1836 */ public int SessionsToExpiry(DateTime day)
/* 1844 */ public int SessionsToExpiry()
/* 1852 */ public static DateTime RollDateFor(DateTime expiry)
/* 1854 */ public static DateTime RuleRollDate(DateTime day)
/* 1857 */ public bool InSpan(EEventKind kind, DateTime utc)
/* 1865 */ public bool InSpan(EEventKind kind)
/* 1868 */ public bool LastT1Instant(out DateTime et)
```

_EventCalendar.Year

```
/* 1114 */ public EventRec[] recs;
/* 1115 */ public Dictionary<DateTime, int[]> dayIdx;
```

_EventCalendar.CsvRow

```
/* 1119 */ public int line; public string action; public EEventKind kind; public string name; public
DateTime date; public TimeSpan? time;
/* 1120 */ public EEventTier? tier; public EEventFlags flags; public string refPeriod; public DateTime? annou
nced; public string source;
```

`_FvgDetector.Function.CS`

(file scope)

```
/* 16 */ public enum EFvgFillOn { Close = 0, Trade = 1 }
/* 18 */ public sealed class FvgConfig
/* 48 */ public sealed class Fvg
/* 62 */ public sealed class _FvgDetector
```

FvgConfig

```
/* 20 */ public bool enabled = false;
/* 21 */ public int dataStream = 1;
/* 22 */ public int minTicks = 1;
/* 23 */ public double minAtrFrac = 0;
/* 24 */ public int atrLen = 14;
/* 25 */ public bool crossSessions = false;
/* 26 */ public bool rthOnly = false;
/* 27 */ public int maxLive = 20;
/* 28 */ public int maxAgeSessions = 5;
/* 29 */ public EFvgFillOn fillOn = EFvgFillOn.Close;
/* 31 */ public string Validate()
/* 42 */ public string Canon()
```

Fvg

```
/* 50 */ public int id, dir;
/* 51 */ public double lo, hi, start;
/* 52 */ public DateTime createdTime, bar1Time, firstTouchTime;
/* 53 */ public DateTime createdDay;
/* 54 */ public bool touched, filled, expired, invalid;
/* 55 */ public double fillPct;
/* 56 */ public int ageSessions;
/* 57 */ public double Size { get; }
/* 58 */ public bool IsLive { get; }
/* 59 */ public string Tag { get; }
```

_FvgDetector

```
/* 64 */ public readonly FvgConfig config;
/* 65 */ public readonly double tick;
/* 66 */ public readonly TimeSpan resolution;
/* 67 */ public Action<string> journal;
/* 75 */ public _FvgDetector(FvgConfig cfg, double tick, TimeSpan resolution)
/* 80 */ public IList<Fvg> Live { get; }
/* 81 */ public Fvg NewThisBar { get; private set; }
/* 82 */ public double Atr { get; }
/* 85 */ public void Add(RingBar b)
/* 164 */ public Fvg Nearest(double price, int dir)
/* 177 */ public LevelRef ToLevel(Fvg f, bool startEdge, int stream = 1)
```

`_Gate0.Function.CS`

(file scope)

```
/* 22 */ public enum EGate0Mode { Strategy = 0, EventStudy = 1, ControlRT = 2, ControlFL = 3, Canary = 4 }
/* 25 */ public enum EGate0Family { All = 0, PriorProfile = 1, NakedPoc = 2, IbEdge = 3, Overnight = 4,
Avwap = 5, Vwap = 6, Weekly = 7, CompositeVa = 8, SinglePrint = 9, DvSource = 10, HvnLvn = 11, Mixture = 12,
SigmaMixture = 13 }
```

```
/* 28 */ public enum EGate0Canary { VW08 = 0, A2_24_i = 1, A2_24_ii = 2, A2_24_iii = 3, DV10 = 4, A2_53 = 5,
DV01 = 6, PS03 = 7 }
/* 30 */ public enum EGate0Entry { None = 0, Market = 1, Limit = 2 }
/* 32 */ public struct Gate0Bar
/* 43 */ public sealed class Gate0Intent
/* 56 */ public sealed class Gate0Profile
/* 159 */ public sealed class Gate0Vwap
/* 170 */ public sealed class Gate0Atr
/* 188 */ public sealed class Gate0Bucket
/* 214 */ public sealed class Gate0Session
/* 237 */ public sealed class Gate0DvLevel
/* 243 */ public sealed class Gate0Result
/* 265 */ public sealed class Gate0Engine
```

Gate0Bar

```
/* 34 */ public DateTime et;
/* 35 */ public DateTime day;
/* 36 */ public double o, h, l, c, v;
/* 37 */ public bool roll;
/* 38 */ public Gate0Bar(DateTime et, DateTime day, double o, double h, double l, double c, double v, bool ro
ll = false)
```

Gate0Intent

```
/* 45 */ public int side;
/* 46 */ public EGate0Entry entry;
/* 47 */ public double entryPrice = double.NaN;
/* 48 */ public double stop = double.NaN, target = double.NaN;
/* 49 */ public double reverseStop = double.NaN;
/* 50 */ public bool flatten;
/* 51 */ public string tag = "";
/* 52 */ public void Clear()
```

Gate0Profile

```
/* 58 */ public readonly SortedDictionary<long, double> rows = new SortedDictionary<long, double>();
/* 59 */ public double tick;
/* 60 */ public double total;
/* 61 */ public Gate0Profile(double tick)
/* 62 */ public void Add(double h, double l, double v)
/* 71 */ public void Merge(Gate0Profile p)
/* 72 */ public bool Empty { get; }
/* 75 */ public bool ValueArea(double pct, out double poc, out double vah, out double val)
/* 93 */ public long FirstRow { get; }
/* 94 */ public long LastRow { get; }
/* 98 */ public static List<double[]> InteriorRuns(long first, long last, Func<long, bool> thin, int
minRows, double tick)
/* 111 */ public List<double[]> SinglePrintRuns(double frac, int minRows)
/* 120 */ public static List<double[]> TpoSinglePrints(IDictionary<long, int> tpo, int minRows, double tick)
/* 130 */ public void Nodes(double bwTicks, double prom, out List<double[]> hvn, out List<double[]> lvn)
```

Gate0Vwap

```
/* 161 */ public double pv, p2v, v;
/* 162 */ public void Reset()
/* 163 */ public void Add(double price, double vol)
/* 164 */ public bool Ready { get; }
/* 165 */ public double Value { get; }
/* 166 */ public double Sigma { get; }
```

Gate0Atr

```
/* 173 */ public double Value = double.NaN;
/* 174 */ public Gate0Atr(int len)
/* 176 */ public void ResetPrev()
/* 177 */ public void Add(double h, double l, double c)
```

Gate0Bucket

```
/* 190 */ public readonly int minutes;
/* 191 */ public bool open; public DateTime end; public double o, h, l, c, v;
/* 192 */ public Gate0Bucket(int minutes)
/* 193 */ public static DateTime EndOf(DateTime t, int minutes)
/* 198 */ public bool PreBar(DateTime t, out DateTime endOut, out double oo, out double hh, out double ll, ou
t double cc, out double vv)
/* 204 */ public void Add(DateTime t, double bo, double bh, double bl, double bc, double bv)
/* 210 */ public bool PostBar(DateTime t)
```

Gate0Session

```
/* 216 */ public DateTime day; public bool roll;
/* 217 */ public double rthO = double.NaN, rthH = double.MinValue, rthL = double.MaxValue, rthC = double.NaN;
/* 218 */ public double ibH = double.MinValue, ibL = double.MaxValue, onH = double.MinValue, onL = double.Max
Value;
/* 219 */ public Gate0Profile profile;
/* 220 */ public readonly Dictionary<long, int> tpo = new Dictionary<long, int>();
/* 221 */ internal readonly Dictionary<long, int> tpoLast = new Dictionary<long, int>();
/* 222 */ public Gate0Vwap vwap = new Gate0Vwap(), avHi = new Gate0Vwap(), avLo = new Gate0Vwap();
/* 223 */ public double poc = double.NaN, vah = double.NaN, val = double.NaN;
/* 224 */ public List<double> closes1m = new List<double>();
/* 225 */ public int bars1m;
/* 226 */ public List<RingBar> rth1m;
/* 227 */ public double hCv = double.NaN;
/* 228 */ public double pkSum; public int pkN;
/* 229 */ public bool pocNaked = true;
/* 230 */ public bool HasRth { get; }
/* 232 */ public double SigmaPrice { get; }
/* 233 */ public Gate0Session(DateTime day, double tick)
```

Gate0DvLevel

```
/* 239 */ public string type = ""; public double price; public int dir; public DateTime validFrom; public
int expirySessions; public bool touched;
```

Gate0Result

```
/* 245 */ public string type = ""; public int levels, touches, sessions;
/* 246 */ public int pairs; public double edge = double.NaN, t = double.NaN, p = double.NaN, pHolm = double.N
aN, ciLo = double.NaN, ciHi = double.NaN;
/* 247 */ public double halfLife = double.NaN; public double[] tByAge = new double[0]; public int[] nByAge =
new int[0]; public double[] tByTercile = new double[3];
/* 248 */ public string a2Verdict = "";
/* 249 */ public string verdict = "";
/* 250 */ public bool verdictRun = true;
/* 251 */ public string AgeList()
/* 257 */ public string Line()
```

Gate0Engine

```
/* 267 */ public static readonly DateTime DevEnd = new DateTime(2025, 1, 1);
/* 268 */ public static readonly DateTime CanaryA224From = new DateTime(2019, 1, 1);
/* 269 */ public const int BootstrapDraws = 500, BootstrapBlock = 5;
/* 270 */ public const int Controls = 20, ReactionBars5m = 12, MaxAge = 15;
/* 271 */ public const int PriorProfileMaxAge = 15, NakedPocMaxAge = 15;
/* 273 */ public const int CompositeSessions = 5;
/* 274 */ public const int SigmaCompositeSessions = 20, SigmaEwmaSpan = 10, ParkinsonMinBars = 60, GmmMaxK =
4;
/* 275 */ public const int HistoryCap = 26;
/* 276 */ public const double KdeBandwidthTicks = 6, KdeProminence = 0.10;
/* 279 */ public const int HcvDistribution = 2;
/* 282 */ public static string PreRegistration()
/* 290 */ public const int SinglePrintMinRows = 4; public const double SinglePrintVolFrac = 0.05;
/* 291 */ public const double DvRotW = 1.5, DvBreakAtr = 1.5, DvTrendLegAtr = 3.0, DvHvnMedianX = 2.0;
/* 292 */ public const int DvRotExpirySessions = 5, DvTrendExpirySessions = 10, DvTrendArmBars = 2, DvTrendLo
okback = 20, DvPendingBars = 26;
/* 293 */ public static readonly int[] DvRotBars = { 6, 12 };
/* 296 */ public static readonly string[] AllTypes = { "POC", "VAH", "VAL", "NPOC", "IBH", "IBL", "ONH", "ONL
", "AV_PH", "AV_PL", "AV_WK", "AV_MO", "AV_OPEN", "AV_PC",
/* 301 */ public EGate0Mode mode = EGate0Mode.EventStudy;
/* 302 */ public int variant;
```

```
/* 303 */ public double tick = 0.25;
/* 304 */ public long seed = 33;
/* 305 */ public double vaPct = 0.70;
/* 306 */ public Action<string> journal;
/* 308 */ public double a253Gap = 0.005; public int a253Bar = 1, a253Minutes = 30; public bool a253Filter = t
rue;
/* 309 */ public double dv01F = 0.15, dv01K = 1.0;
/* 310 */ public double dv10S = 0.5, dv10F = 0.20;
/* 312 */ internal readonly _LevelStudy study = new _LevelStudy();
/* 313 */ public readonly Gate0Intent intent = new Gate0Intent();
/* 314 */ public readonly List<DateTime> rollDays = new List<DateTime>();
/* 315 */ public readonly List<DateTime> sessionDays = new List<DateTime>();
/* 316 */ public int Trades { get; private set; }
/* 319 */ public int RollSkips { get; private set; }
/* 321 */ public bool VerdictRun { get; }
/* 345 */ public IList<Gate0Session> History { get; }
/* 346 */ public Gate0Session Current { get; }
/* 347 */ public IList<Gate0DvLevel> DvLevels { get; }
/* 348 */ public double SessionVwap { get; }
/* 349 */ public double Atr20d { get; }
/* 350 */ public double Atr30 { get; }
/* 354 */ public static bool DecodeA253(int v, out double gap, out int j, out int minutes, out bool filter)
/* 364 */ public string CanaryName { get; }
/* 366 */ public string Configure()
/* 388 */ public void Bar(Gate0Bar b, int mp = 0)
/* 621 */ public List<KeyValuePair<string, double[]>> Candidates()
/* 686 */ public static GmmSession ToGmmSession(Gate0Session s, double tick)
/* 697 */ public static SigmaMixtureResult SigmaCompositeFit(IList<Gate0Session> hist, double tick)
/* 747 */ public bool InRollWindow(DateTime day)
/* 756 */ public static void HolmFixed(List<LevelTypeResult> r, int m)
/* 764 */ public List<Gate0Result> Finish()
/* 806 */ public static double PairedReaction(LevelEvent lv, IList<LevelEvent> ctl)
/* 816 */ public static void ScoreEdge(Gate0Result r, IList<KeyValuePair<DateTime, double>> diffs, LevelTypeR
esult a2, long seed)
/* 853 */ public static string Verdict(int n, double t, string a2)
/* 900 */ public static bool InsideDay(double h, double l, double h1, double l1)
/* 908 */ public double Hv(int n)
```

`_Gmm.Function.CS`

(file scope)

```
/* 18 */ public enum EGmmTolMode { Relative = 0, PerSample = 1 }
/* 20 */ public sealed class GmmModel
/* 55 */ public sealed class GmmFitConfig
/* 87 */ public static class _Gmm
```

GmmModel

```
/* 22 */ public int K, D;
/* 23 */ public double[] w;
/* 24 */ public double[][] mu;
/* 25 */ public double[][,] cov;
/* 26 */ public double logLik = double.NaN, bic = double.NaN;
/* 27 */ public int iters;
/* 28 */ public bool converged;
/* 29 */ public int reseeds;
/* 31 */ public double nEff = double.NaN;
/* 32 */ public int restart = -1;
/* 33 */ public int cholRetries;
/* 34 */ public bool reseedInFinalIter;
/* 35 */ public GmmModel[] perK;
/* 36 */ public GmmModel init;
/* 38 */ public int Params { get; }
/* 41 */ public double Sd(int k)
/* 43 */ public GmmModel Clone()
```

GmmFitConfig

```
/* 57 */ public int kMin = 1, kMax = 3, restarts = 10, maxIter = 200;
/* 58 */ public double tol = 1e-6;
```

```
/* 59 */ public EGmmTolMode tolMode = EGmmTolMode.Relative;
/* 60 */ public double regCovar = 1e-6, minWeight = 1e-3, bicDelta = 10;
/* 61 */ public ulong seed;
/* 62 */ public int lloydIters = 10;
/* 63 */ public int cholRetries = 3;
/* 65 */ public string Validate()
/* 78 */ public string Canon()
/* 84 */ public GmmFitConfig Copy()
```

_Gmm

```
/* 89 */ public const int MaxD = 4;
/* 92 */ public static int ParamCount(int K, int D)
/* 99 */ public static GmmModel Fit(double[][] x, double[] weights, double nEff, GmmFitConfig cfg, int orderF
eature = 0, int tieFeature = -1)
/* 125 */ public static GmmModel Fit1D(IList<double> x, IList<double> weights, double nEff, GmmFitConfig cfg)
/* 158 */ public static GmmModel FitK(double[][] x, double[] om, double nEff, int K, GmmFitConfig cfg, int or
derFeature = 0, int tieFeature = -1)
/* 172 */ public static GmmModel FitOnce(double[][] x, double[] om, double nEff, int K, GmmFitConfig cfg,
int restart)
/* 365 */ public static bool CholeskyWithRetry(double[,] cov, int D, double regCovar, int retries, double[,]
L, ref int retriesUsed)
/* 380 */ public static bool Cholesky(double[,] a, int D, double[,] L)
/* 404 */ public static double LogPdf(double[] x, double[] mu, double[,] L, int D)
/* 418 */ public static double LogDensity(GmmModel m, double[] x)
/* 433 */ public static double Density1D(GmmModel m, double x)
/* 441 */ public static void Order(GmmModel m, int orderFeature, int tieFeature)
```

`_LevelRef.Function.CS`

(file scope)

```
/* 15 */ public enum ELevelKind { Poc, Vah, Val, Hvn, Lvn, RangePoc, RangeVah, RangeVal, WeekPoc, WeekVah,
WeekVal, FvgUpper, FvgLower, GmmMu, GmmEdge, GmmVoid, SigmaMu, VtPrice, BootstrapPocZoneLo, BootstrapPocZoneH
i }
/* 18 */ public sealed class LevelRef
/* 55 */ public sealed class _RollWindow
```

LevelRef

```
/* 20 */ public ELevelKind kind; public double price, lo, hi;
/* 21 */ public DateTime createdTime;
/* 22 */ public DateTime sourceT0, sourceT1;
/* 23 */ public int dataStream; public string tag;
/* 24 */ public double quality = double.NaN;
/* 25 */ public bool crossesRoll;
/* 26 */ public bool isRollWindow;
/* 28 */ public static LevelRef Line(ELevelKind kind, double price, DateTime created, DateTime t0, DateTime t
1, int stream, string tag, double quality = double.NaN)
/* 33 */ public static LevelRef Zone(ELevelKind kind, double price, double lo, double hi, DateTime created, D
ateTime t0, DateTime t1, int stream, string tag, double quality = double.NaN)
/* 39 */ public string StudyType { get; }
/* 41 */ public string StudyKey { get; }
/* 43 */ public override string ToString()
```

_RollWindow

```
/* 62 */ public void Observe(DateTime tradingDay, bool roll)
/* 73 */ public bool IsRollDay(DateTime tradingDay)
/* 76 */ public bool IsRollWindow(DateTime tradingDay)
/* 86 */ public int Days { get; }
```

`_LevelStudy.Function.CS`

(file scope)

```
/* 21 */ public sealed class LevelStudyConfig
/* 53 */ public sealed class LevelEvent
/* 62 */ public sealed class LevelTypeResult
/* 76 */ internal sealed class _LevelStudy
```

LevelStudyConfig

```
/* 23 */ public bool enabled = false;
/* 24 */ public int controls = 20;
/* 25 */ public double distLo = 0.9, distHi = 1.1;
/* 26 */ public int reactionBars = 60;
/* 27 */ public int sigmaBars = 60;
/* 28 */ public double touchTicks = 0;
/* 29 */ public int maxAge = 15;
/* 30 */ public double deathR = 1.0;
/* 31 */ public int seed = 33;
/* 32 */ public DateTime from = DateTime.MinValue, to = DateTime.MaxValue;
/* 33 */ public string csvPath = "";
/* 34 */ public bool controlsEitherSide = false;
/* 35 */ public bool reactionAfterTouch = false;
/* 37 */ public string Validate()
```

LevelEvent

```
/* 55 */ public DateTime day; public string type = ""; public string key = ""; public int age; public double
quality = double.NaN;
/* 56 */ public bool control; public int levelNo; public double price, refPrice, sigma;
/* 57 */ public bool touched; public int touchBar = -1; public int fromAbove;
/* 58 */ public double reaction = double.NaN, asymmetry = double.NaN, volAttraction = double.NaN; public int
barsSeen;
/* 59 */ internal double maxBounce, maxThrough, volSum; internal int volBars;
```

LevelTypeResult

```
/* 64 */ public string type = ""; public int levels, touchedLevels;
/* 65 */ public double touchEdge = double.NaN, touchT = double.NaN;
/* 66 */ public double edge = double.NaN, t = double.NaN, p = double.NaN, pHolm = double.NaN;
/* 67 */ public double asymEdge = double.NaN, volEdge = double.NaN;
/* 68 */ public double[] edgeByAge = new double[0]; public int[] nByAge = new int[0]; public double[] tByAge
= new double[0];
/* 69 */ public double halfLife = double.NaN;
/* 70 */ public double[] edgeByTercile = { double.NaN, double.NaN, double.NaN }; public double[] tByTercile
= { double.NaN, double.NaN, double.NaN };
/* 71 */ public double[] kmSurvival = new double[0];
/* 72 */ public double kmMedianAge = double.NaN;
/* 73 */ public string verdict = "";
```

_LevelStudy

```
/* 78 */ public LevelStudyConfig config = new LevelStudyConfig();
/* 79 */ public double tickSize = 0.25;
/* 80 */ public Action<string> journal;
/* 90 */ public IReadOnlyList<LevelEvent> Events { get; }
/* 91 */ public string Configure()
/* 94 */ public void BeginSession(DateTime day)
/* 104 */ public void SetSessionReference(double refPrice, double sigma)
/* 107 */ public bool AddLevel(string type, double price, int ageSessions, double quality = double.NaN, strin
g key = null)
/* 134 */ public void Bar(double o, double h, double l, double c, double v)
/* 184 */ public List<LevelTypeResult> Finish()
/* 188 */ public static List<LevelTypeResult> Analyse(IList<LevelEvent> events, LevelStudyConfig cfg)
/* 245 */ public static void meanT(IList<double> x, out double mean, out double t)
/* 254 */ public static double Phi(double z)
/* 262 */ public static void Holm(List<LevelTypeResult> r)
/* 269 */ public static double HalfLife(double[] edgeByAge, int[] n)
/* 283 */ public static double[] KaplanMeier(IList<KeyValuePair<double, bool>> obs, int maxT)
/* 296 */ public static string Verdict(LevelTypeResult r)
/* 309 */ public static string Summary(LevelTypeResult r)
/* 317 */ public void WriteCsv(TextWriter w)
```

`_MtfMath.Function.CS`

(file scope)

```
/* 11 */ public static class _MtfMath
```

_MtfMath

```
/* 14 */ public static int ClosedIndex(bool barZeroClosed, int ago)
/* 21 */ public static int MapIndex(Func<int, DateTime> timeAt, int first, DateTime t, int maxBack)
/* 28 */ public static string SymbolRoot(string name)
/* 41 */ public static bool IsStale(DateTime streamTime, DateTime data1Time, TimeSpan maxStale)
```

`_NyseHolidays.Function.CS`

(file scope)

```
/* 14 */ public sealed class _NyseHolidays : _Holidays
```

_NyseHolidays

```
/* 26 */ public static class Rules
/* 104 */ public _NyseHolidays(CStudyControl _master)
/* 105 */ public _NyseHolidays(CStudyControl _master, int _ds)
/* 112 */ protected override void refreshHolidays(int year)
```

_NyseHolidays.Rules

```
/* 44 */ public static List<DateTime> Holidays(int year)
/* 65 */ public static DateTime Observed(DateTime holiday)
/* 72 */ public static DateTime NthWeekday(int year, int month, DayOfWeek day, int n)
/* 79 */ public static DateTime LastWeekday(int year, int month, DayOfWeek day)
/* 86 */ public static DateTime Easter(int year)
```

`_OrderFlow.Function.CS`

(file scope)

```
/* 26 */ public sealed class _OrderFlow
```

_OrderFlow

```
/* 32 */ public enum FlowSource { Aggressor = 0, McBidAsk = 1, Quote = 2, TickRule = 3, BarUpDown = 4,
Unavailable = 5 }
/* 34 */ public enum EMcBidAsk { Off = 0, Prefer = 1, CrossCheckOnly = 2 }
/* 36 */ public const string McBidAskMethod = "McBidAsk(quote-classified, not aggressor; MC rule: inside-
spread trade -> last recorded side, UM p35/p159)";
/* 38 */ public enum FlowQuality { Exact = 0, High = 1, Medium = 2, Low = 3, Unknown = 4 }
/* 39 */ public enum Side { Unknown = 0, Buy = 1, Sell = -1 }
/* 41 */ public enum ResetMode { Session = 0, Rth = 1 }
/* 43 */ public enum VolumeBasis { Unknown = 0, TradeVolume = 1, TickCount = 2 }
/* 45 */ public enum Regime { TickStream = 0, IogRealTime = 1, IogMagnifierOneTick = 2,
IogHistoryNoMagnifier = 3, BarsOnly = 4 }
/* 54 */ public struct FlowValue
/* 68 */ public sealed class FlowConfig
/* 86 */ public struct BarInput
/* 96 */ public struct Imbalance
/* 104 */ public sealed class StackedResult
/* 113 */ public sealed class FootprintBar
/* 155 */ public sealed class TickRule
/* 190 */ public sealed class FlowEngine
/* 426 */ public static List<Imbalance> Imbalances(FootprintBar fp, double ratio, double minVol, bool exclude
Extremes)
```

```
/* 455 */ public static StackedResult Stacked(FootprintBar fp, double ratio, double minVol, int minStack, boo
l excludeExtremes)
```

_OrderFlow.FlowValue

```
/* 56 */ public double Value;
/* 57 */ public FlowSource Source;
/* 58 */ public FlowQuality Quality;
/* 59 */ public bool Developing;
/* 60 */ public static FlowValue Unavailable(bool developing = false)
/* 64 */ public bool IsAvailable { get; }
/* 65 */ public override string ToString()
```

_OrderFlow.FlowConfig

```
/* 70 */ public double MaxQuoteAgeMs = 1000;
/* 71 */ public ResetMode ResetMode = ResetMode.Session;
/* 73 */ public bool AllowMixed = false;
/* 74 */ public bool QuotesExpected = true;
/* 75 */ public VolumeBasis BuildVolumeOn = VolumeBasis.Unknown;
/* 76 */ public bool ExcludeExtremesTickRule = true;
/* 77 */ public int MaxBars = 5000;
/* 80 */ public EMcBidAsk McBidAsk = EMcBidAsk.Off;
/* 82 */ public double mcVolTolerance = 0.005;
```

_OrderFlow.BarInput

```
/* 88 */ public int BarNumber;
/* 89 */ public DateTime CloseTime;
/* 90 */ public DateTime SessionKey;
/* 91 */ public DateTime ResetKey;
/* 92 */ public Regime Regime;
/* 93 */ public double High, Low, Close, Volume, UpTicks, DownTicks;
```

_OrderFlow.Imbalance

```
/* 98 */ public double Price;
/* 99 */ public Side Side;
/* 100 */ public double Ratio;
/* 101 */ public FlowQuality Quality;
```

_OrderFlow.StackedResult

```
/* 106 */ public readonly List<double[]> Runs = new List<double[]>();
/* 107 */ public FlowQuality Quality = FlowQuality.Unknown;
/* 108 */ public int Longest { get; }
/* 109 */ public bool Found { get; }
```

_OrderFlow.FootprintBar

```
/* 115 */ public int BarNumber;
/* 116 */ public DateTime CloseTime, SessionKey, ResetKey;
/* 117 */ public FlowSource Source = FlowSource.Unavailable;
/* 118 */ public FlowQuality Quality = FlowQuality.Unknown;
/* 119 */ public double Buy, Sell, Volume;
/* 120 */ public int Trades, StaleQuoteTrades;
/* 121 */ public double McDelta = double.NaN;
/* 122 */ public double ChainDelta = double.NaN;
/* 123 */ public string McMiss;
/* 124 */ public bool FirstAfterConnect;
/* 125 */ public double TradeHigh = double.NaN, TradeLow = double.NaN, BarHigh, BarLow, BarClose;
/* 126 */ internal readonly SortedDictionary<long, double[]> Ladder = new SortedDictionary<long, double[]>();
/* 127 */ public double Tick;
/* 128 */ public bool HasLadder { get; }
/* 129 */ public double Delta { get; }
/* 130 */ public long Row(double price)
/* 131 */ public double Bid(double price)
/* 132 */ public double Ask(double price)
```

```
/* 134 */ public List<double[]> Rows()
/* 141 */ internal void add(double price, double size, Side side)
```

_OrderFlow.TickRule

```
/* 160 */ public TickRule(double tick)
/* 161 */ public Side Peek(double price)
/* 168 */ public void Observe(double price, Side side)
/* 169 */ public Side Classify(double price)
```

_OrderFlow.FlowEngine

```
/* 192 */ public readonly FlowConfig Cfg;
/* 193 */ public readonly double Tick;
/* 205 */ public int McUsed, McMissing, McVolMismatch;
/* 208 */ public void AddMcLevels(DateTime barTime, double[] price, double[] ask, double[] bid)
/* 213 */ public int PendingMcSets { get; }
/* 254 */ public FlowEngine(double tick, FlowConfig cfg)
/* 256 */ public void AddTrade(DateTime time, double price, double size)
/* 257 */ public void AddQuote(DateTime time, bool isBid, double price)
/* 258 */ public int PendingTrades { get; }
/* 259 */ public int Count { get; }
/* 262 */ public FootprintBar Closed(int ago)
/* 263 */ public FootprintBar ByNumber(int bar)
/* 276 */ public FootprintBar CloseBar(BarInput b)
/* 345 */ public static bool BarUpDownQualifies(VolumeBasis basis, double volume, double up, double down)
/* 351 */ public FlowValue BarDelta(int ago)
/* 359 */ public FlowValue CumDelta(int ago)
/* 375 */ public List<Imbalance> DiagonalImbalances(int ago, double ratio = 3.0, double minVol = 10)
/* 380 */ public StackedResult StackedImbalances(int ago, double ratio = 3.0, double minVol = 10, int minStac
k = 3)
/* 391 */ public FlowValue Absorption(int ago, Side side, int edgeTicks = 2, double volMult = 2.0, int holdTi
cks = 1)
```

`_PortMM.Function.CS`

(file scope)

```
/* 19 */ public static class _PortKeys
/* 40 */ public enum EPortState { Flat = 0, Armed = 1, Long = 2, Short = 3, Exiting = 4, Halted = 5 }
/* 41 */ public enum EPortSizing { CodeSize = 0, RiskParity = 1, VolTarget = 2 }
/* 44 */ public sealed class _PortReturns
/* 68 */ public sealed class PortDdStep
/* 71 */ public sealed class PortMmConfig
/* 138 */ public static class PortMmConfigLoader
/* 268 */ public sealed class PortStratView
/* 289 */ public sealed class PortDecision
/* 300 */ public static class _PortMath
/* 374 */ internal sealed class _PortMM
```

_PortKeys

```
/* 21 */ public const int Version = 1;
/* 22 */ public const string Ver = "pm.ver", Id = "pm.id", Family = "pm.family", Cluster = "pm.cluster";
/* 23 */ public const string Eq = "pm.eq", Pnl = "pm.pnl", Ret = "pm.ret", URet = "pm.uRet", Vol = "pm.vol";
/* 24 */ public const string RDollar = "pm.R$", QtyCode = "pm.qtyCode", RiskNew = "pm.riskNew", RiskOpen = "p
m.riskOpen", OpenR = "pm.openR";
/* 25 */ public const string State = "pm.state", Side = "pm.side", Regime = "pm.regime", Score = "pm.score",
Stamp = "pm.stamp", Day = "pm.day", Pos = "pm.pos";
/* 28 */ public const string RegimeP = "pm.regimeP";
/* 32 */ public const string EvtTier = "pm.evtTier";
/* 33 */ public const string EvtMask = "pm.evtMask";
/* 34 */ public const string EvtWin = "pm.evtWin";
/* 35 */ public const string EvtNext = "pm.evtNext";
/* 37 */ public const string Mult = "pm.mult", Heat = "pm.heat", Dd = "pm.dd", Rho = "pm.rho";
```

_PortReturns

```
/* 46 */ public const int DefaultCapacity = 260;
/* 48 */ public int Count { get; }
/* 49 */ public int Capacity { get; private set; }
/* 50 */ public DateTime LastDay { get; }
/* 51 */ public _PortReturns(int capacity = DefaultCapacity)
/* 53 */ public DateTime Day(int i)
/* 54 */ public double Value(int i)
/* 56 */ public _PortReturns Append(DateTime day, double value)
/* 65 */ public Dictionary<DateTime, double> ToMap()
```

PortMmConfig

```
/* 73 */ public EPortSizing sizing = EPortSizing.RiskParity;
/* 74 */ public double bookVolTarget = 0.10;
/* 75 */ public double riskPerTrade = 0.005;
/* 76 */ public double heatCap = 0.04;
/* 77 */ public double clusterCap = 0.02;
/* 78 */ public double rhoMerge = 0.9;
/* 79 */ public int rhoWin = 60;
/* 80 */ public int rhoMinDays = 40;
/* 81 */ public int maxPositions = 6;
/* 82 */ public PortDdStep[] ddSteps = { new PortDdStep(0.05, 0.5), new PortDdStep(0.10, 0.25), new PortDdSte
p(0.15, 0.0) };
/* 83 */ public int ddRecoverHighDays = 20;
/* 84 */ public double dailyLossStop = 0.02;
/* 85 */ public double dailyLossFlatten = 0.03;
/* 86 */ public double stratKillR = -8;
/* 87 */ public int stratKillTrades = 20;
/* 88 */ public int staleBars = 3;
/* 89 */ public int minContracts = 1;
/* 90 */ public double idmCap = 2.5;
/* 92 */ public string Validate()
/* 116 */ public PortMmConfig Copy()
/* 123 */ public override string ToString()
```

PortMmConfigLoader

```
/* 141 */ public static PortMmConfig LoadFile(string path, PortMmConfig defaults, List<string> warnings,
List<string> info)
/* 157 */ public static PortMmConfig Parse(string json, PortMmConfig defaults, List<string> warnings, string
source = "json")
/* 214 */ public static PortMmConfig Clone(PortMmConfig d)
```

PortStratView

```
/* 270 */ public int index;
/* 271 */ public string id = "";
/* 272 */ public bool publishes;
/* 273 */ public bool stale;
/* 274 */ public bool currencyOk = true;
/* 275 */ public bool paused;
/* 276 */ public EPortState state;
/* 277 */ public int side;
/* 278 */ public double rDollar = double.NaN;
/* 279 */ public int qtyCode;
/* 280 */ public double riskOpen;
/* 281 */ public double vol = double.NaN;
/* 282 */ public string cluster = "";
/* 283 */ public int position;
/* 284 */ public double pnl;
/* 285 */ public double score = double.NaN;
/* 286 */ public _PortReturns ret, uRet;
```

PortDecision

```
/* 291 */ public int index;
/* 292 */ public bool allowEntries, allowLong, allowShort;
/* 293 */ public int entryContracts = -1;
/* 294 */ public bool forceClose, pause;
```

```
/* 295 */ public string status = "";
/* 296 */ public string reason = "";
```

_PortMath

```
/* 303 */ public static double Pearson(_PortReturns a, _PortReturns b, int win, int minDays, Func<DateTime,
bool> dayFilter = null)
/* 316 */ public static double Corr(IList<double> x, IList<double> y)
/* 325 */ public static int[] Clusters(string[] seeds, double[,] rho, double threshold)
/* 337 */ public static double Idm(int[] active, double[,] rho, int[] cluster, double cap)
/* 352 */ public static int DdStage(PortDdStep[] steps, double dd)
/* 358 */ public static int Size(PortMmConfig c, double mult, double book, double rDollar, int qtyCode, doubl
e vol, int k, double idm)
```

_PortMM

```
/* 376 */ public PortMmConfig config = new PortMmConfig();
/* 377 */ public Action<string> journal;
/* 393 */ public double Mult { get; private set; }
/* 394 */ public double Heat { get; private set; }
/* 395 */ public double Drawdown { get; private set; }
/* 396 */ public double Book { get; private set; }
/* 397 */ public double[,] Rho { get; }
/* 398 */ public int DdStageNow { get; }
/* 399 */ public bool DayStopped { get; }
/* 401 */ public string Configure()
/* 407 */ public PortDecision[] Decide(DateTime day, double initialCapital, bool currencyOk, IList<PortStratV
iew> v)
```

`_PriceActionEngine.Function.CS`

(file scope)

```
/* 24 */ public enum EPivotMode { Fractal = 1, AtrZigZag = 2 }
/* 25 */ public enum EPivotTieRule { Strict = 0, LeftInclusive = 1 }
/* 26 */ public enum EPivotLabel { None = 0, HH, LH, HL, LL }
/* 27 */ public enum EStructure { Unknown = 0, Up, Down, Range }
/* 31 */ public enum EPriceActionPreset { Library = 0, Futures = 1 }
/* 35 */ public enum ECandle { None = 0, Bull = 1 << 0, Bear = 1 << 1, Doji = 1 << 2, BodyAtHigh = 1 << 3, Bo
dyAtLow = 1 << 4, Inside = 1 << 5, Outside = 1 << 6, Engulfing = 1 << 7, BullReversal = 1 << 8, BearReversal
= 1 << 9, Strong = 1 << 10, VolAboveAvg = 1 << 11, VolPlus1SD = 1 << 12, VolPlus2SD = 1 << 13, VolPlus3SD =
1 << 14, VolBelowAvg = 1 << 15, VolMinus1SD = 1 << 16, VolMinus2SD = 1 << 17, VolMinus3SD = 1 << 18, Breakout
Up = 1 << 19, BreakoutDown = 1 << 20 }
/* 49 */ public sealed class PriceActionConfig
/* 139 */ public sealed class _Pivot
/* 162 */ internal sealed class _Ring<T>
/* 175 */ internal struct _PaBar
/* 177 */ internal sealed class _PriceActionEngine
```

PriceActionConfig

```
/* 51 */ public bool enabled = false;
/* 52 */ public int dataStream = 1;
/* 53 */ public EPivotMode pivotMode = EPivotMode.Fractal;
/* 54 */ public EPivotTieRule tieRule = EPivotTieRule.Strict;
/* 55 */ public int layers = 2;
/* 56 */ public int strength = 1;
/* 57 */ public int atrLength = 20;
/* 58 */ public int volumeLength = 20;
/* 59 */ public bool volumeIncludesCurrent = true;
/* 60 */ public double atrMultiple = 1.5;
/* 61 */ public double factorDoji = 0.1;
/* 62 */ public double factorClose = 0.1;
/* 63 */ public double factorStrong = 1.0;
/* 64 */ public double factorEpsilon = 0.05;
/* 65 */ public double equalFactor = 0.1;
/* 66 */ public int lookbackPivots = 5;
/* 67 */ public int minBreakoutLayer = 1;
/* 68 */ public bool breakoutNeedsStrong = true;
/* 69 */ public int history = 50;
```

```
/* 70 */ public bool candles = true;
/* 71 */ public EPriceActionPreset preset = EPriceActionPreset.Library;
/* 73 */ public string role = "";
/* 75 */ public bool presetApplied;
/* 78 */ public PriceActionConfig ApplyPreset(EPriceActionPreset p)
/* 91 */ public PriceActionConfig ApplyPresetDefaults(EPriceActionPreset p)
/* 101 */ public static PriceActionConfig Futures()
/* 103 */ public static PriceActionConfig Library()
/* 107 */ public string Canon()
/* 116 */ public string Validate()
```

_Pivot

```
/* 141 */ public readonly bool isHigh;
/* 142 */ public readonly int layer;
/* 143 */ public readonly long seq;
/* 144 */ public readonly double price;
/* 145 */ public readonly DateTime extremeTime;
/* 146 */ public readonly int extremeBar;
/* 147 */ public readonly DateTime confirmTime;
/* 148 */ public readonly int confirmBar;
/* 149 */ public readonly EPivotLabel label;
/* 150 */ public readonly bool isEqual;
/* 151 */ public readonly long equalToSeq;
/* 152 */ public readonly double legSizeAtr;
/* 153 */ public _Pivot(bool isHigh, int layer, long seq, double price, DateTime extremeTime, int
extremeBar, DateTime confirmTime, int confirmBar, EPivotLabel label, bool isEqual, long equalToSeq, double le
gSizeAtr)
```

_Ring

```
/* 165 */ public _Ring(int capacity)
/* 166 */ public int Count { get; }
/* 167 */ public int Capacity { get; }
/* 168 */ public void Clear()
/* 169 */ public void Add(T v)
/* 171 */ public T Ago(int ago)
/* 172 */ public T Oldest()
```

_PriceActionEngine

```
/* 180 */ public PriceActionConfig config = new PriceActionConfig();
/* 181 */ public string label = "PA";
/* 182 */ public Action<string> trace;
/* 183 */ public Action<string> journal;
/* 184 */ public Action<string> warnOnce;
/* 185 */ public Func<double> volumeZOverride;
/* 188 */ public bool Configured { get; private set; }
/* 189 */ public bool IsWarm { get; private set; }
/* 190 */ public double Atr { get; }
/* 191 */ public double VolumeMean { get; private set; }
/* 192 */ public double VolumeSigma { get; private set; }
/* 193 */ public ECandle Candle { get; private set; }
/* 194 */ public ECandle Changed { get; private set; }
/* 195 */ public bool NewPivotHigh { get; private set; }
/* 196 */ public bool NewPivotLow { get; private set; }
/* 197 */ public bool BosUp { get; private set; }
/* 198 */ public bool BosDown { get; private set; }
/* 199 */ public bool ChochUp { get; private set; }
/* 200 */ public bool ChochDown { get; private set; }
/* 201 */ public DateTime LastBarTime { get; private set; }
/* 203 */ public int LastBar { get; }
/* 224 */ public string Configure()
/* 242 */ public void Reset()
/* 252 */ public void OnRoll(DateTime barTime)
/* 262 */ public void Step(IInstrument bars)
/* 270 */ public bool Add(int barNumber, DateTime time, double o, double h, double l, double c, double v)
/* 295 */ public _Pivot LastHigh(int layer = 0)
/* 296 */ public _Pivot LastLow(int layer = 0)
/* 297 */ public _Pivot High(int layer, int ago)
/* 298 */ public _Pivot Low(int layer, int ago)
/* 299 */ public int PivotCount(int layer, bool isHigh)
/* 300 */ public EStructure Structure(int layer = 0)
```

```
/* 301 */ public bool DoubleTop(int layer = 0)
/* 302 */ public bool DoubleBottom(int layer = 0)
/* 303 */ public bool Has(ECandle f)
/* 306 */ public const double SMALL_BODY_RATIO = 1.0 / 3.0;
/* 307 */ public const double TAIL_MIN_RATIO = 1.0 / 3.0;
/* 308 */ public const double TAIL_MAX_RATIO = 1.0 / 2.0;
/* 311 */ internal static ECandle ClassifyCandle(_PaBar b, _PaBar p, double atr, PriceActionConfig k)
```

`_ProfileMath.Function.CS`

(file scope)

```
/* 14 */ public class _ProfileMath
```

_ProfileMath

```
/* 16 */ protected _ProfileMath()
/* 18 */ public enum ValueRel { Unknown = 0, Higher, Lower, OverlappingHigher, OverlappingLower, Inside, Outs
ide }
/* 19 */ public enum Shape { Unknown = 0, D, P, b, Thin, Double }
/* 22 */ public const double SHAPE_SKEW_SHARE = 0.50;
/* 23 */ public const double SHAPE_THIN_PEAK_TO_MEAN = 1.5;
/* 24 */ public const double SHAPE_DOUBLE_PEAK = 0.60;
/* 25 */ public const double SHAPE_DOUBLE_VALLEY = 0.50;
/* 26 */ public const double SHAPE_DOUBLE_SEPARATION = 0.25;
/* 28 */ public static long Key(double price, double tick)
/* 31 */ public static bool ToDense(IList<double> prices, IList<double> vols, double tick, out double[] dense
Prices, out double[] denseVols)
/* 53 */ public static int PocIndex(double[] vols)
/* 71 */ public static bool ValueAreaExpansion(double[] vols, double percent, out int lo, out int hi, out
int poc)
/* 93 */ public static bool ValueAreaSigma(double[] prices, double[] vols, double k, out double vah, out doub
le val)
/* 110 */ public static ValueRel Relationship(double prevVah, double prevVal, double curVah, double curVal)
/* 124 */ public static bool IsPoorExtreme(double[] vols, bool top, double fraction, int minRows)
/* 144 */ public static List<int[]> SinglePrints(double[] vols, int vaLo, int vaHi, double pct, int minRows)
/* 164 */ public static Shape ProfileShape(double[] vols, double vaPercent)
/* 191 */ public enum MigrationState { Unknown = 0, Down, Neutral, Up }
/* 194 */ public static int MigrationStep(double a, double b, double d)
/* 206 */ public static int? MigrationScore(IList<double> pocsOldestToNewest, int n, double d, double?
devPoc)
/* 217 */ public static MigrationState ClassifyMigration(int? v, int upAt = 2, int downAt = 2)
/* 226 */ public static bool IsBalanced(IList<double> vahs, IList<double> vals, double minOverlap)
/* 247 */ public static bool ValueMigrationScore(IList<double> completedPocs, int n, double developingPoc, do
uble minChange, out int score)
/* 270 */ public sealed class Accumulator
```

_ProfileMath.Accumulator

```
/* 272 */ public readonly SortedDictionary<long, double> rows = new SortedDictionary<long, double>();
/* 273 */ public double tick;
/* 274 */ public int bars;
/* 275 */ public double open, high, low, close;
/* 276 */ public int firstBar = -1, lastBar = -1;
/* 277 */ public DateTime firstTime, lastTime;
/* 279 */ public Accumulator(double tick)
/* 281 */ public void Reset()
/* 283 */ public void Add(int barIndex, DateTime time, double o, double h, double l, double c, double
volume, int mode)
/* 302 */ public void AddRow(long k, double v)
/* 312 */ public void Dense(out double[] prices, out double[] vols)
```

`_ProfileStats.Function.CS`

(file scope)

```
/* 27 */ public enum EEntropyRows { DenseRange = 0, TradedRows = 1 }
/* 28 */ public enum EMmN { Bars = 0, Trades = 1, Contracts = 2 }
```

```
/* 29 */ public enum EKdeVerdict { Unknown = 0, Stable, Flat, Bimodal }
/* 30 */ public enum ECvFolding { Contiguous = 0, Interleaved = 1 }
/* 32 */ public sealed class ProfileStatsConfig
/* 71 */ public struct EntropyResult
/* 81 */ public sealed class BootstrapResult
/* 88 */ public sealed class KdeResult
/* 107 */ public sealed class VtResult
/* 117 */ public enum EGmmNeff { Bars = 0, Rows = 1, Contracts = 2 }
/* 119 */ public sealed class ProfileMixtureResult
/* 146 */ public sealed class GmmSession
/* 172 */ public sealed class SigmaMixtureResult
/* 184 */ public static class _ProfileStats
/* 861 */ public sealed class _PriorDayValue
/* 884 */ public sealed class _EntropyTracker
/* 928 */ public sealed class _GmmProfileTracker
```

ProfileStatsConfig

```
/* 34 */ public bool enabled = false;
/* 35 */ public EEntropyRows entropyRows = EEntropyRows.DenseRange;
/* 36 */ public EMmN mmN = EMmN.Bars;
/* 37 */ public double klEpsilon = 1e-6;
/* 38 */ public int parkinsonMinBars = 60;
/* 39 */ public int ewmaSessions = 10;
/* 40 */ public int pctLookbackA229 = 250;
/* 41 */ public int pctLookbackA230 = 60;
/* 42 */ public double pctMinFill = 0.8;
/* 43 */ public int bootstrapBlock = 5, bootstrapDraws = 500;
/* 44 */ public double kdeMedFloorTicks = 1;
/* 45 */ public int kdeFolds = 5;
/* 46 */ public ECvFolding kdeFolding = ECvFolding.Contiguous;
/* 47 */ public int ringBars = 1440;
/* 49 */ public string Validate()
/* 64 */ public string Canon()
```

EntropyResult

```
/* 73 */ public double H, Hnorm, Hmm, HnormMm, Neff, N;
/* 74 */ public int n, m;
/* 75 */ public override string ToString()
```

BootstrapResult

```
/* 83 */ public double medianPoc = double.NaN, ci90Ticks = double.NaN, stability = double.NaN, zoneLo =
double.NaN, zoneHi = double.NaN;
/* 84 */ public double[] pocs = new double[0];
/* 85 */ public int bars, draws, block; public ulong seed;
```

KdeResult

```
/* 90 */ public EKdeVerdict verdict = EKdeVerdict.Unknown;
/* 91 */ public double sigma = double.NaN, maxJ = double.NaN, medJ = double.NaN, range = double.NaN, hCv = do
uble.NaN, cvPoc = double.NaN;
/* 92 */ public ECvFolding folding = ECvFolding.Contiguous;
/* 93 */ public double hCvAlt = double.NaN;
/* 94 */ public double HcvContiguous { get; }
/* 95 */ public double HcvInterleaved { get; }
/* 97 */ public string Journal()
/* 102 */ public bool disagree;
/* 103 */ public double[] h = new double[0], poc = new double[0];
/* 104 */ public double[] cvH = new double[0], cvLL = new double[0];
```

VtResult

```
/* 109 */ public double volPoc = double.NaN, tpoPoc = double.NaN, atr = double.NaN;
/* 110 */ public readonly List<double> prices = new List<double>(), ratios = new List<double>();
```

ProfileMixtureResult

```
/* 121 */ public GmmModel model;
/* 122 */ public double[] mu = new double[0], sd = new double[0], pi = new double[0], voidPrice = new
double[0], voidDepth = new double[0];
/* 123 */ public double nEff = double.NaN, tick = double.NaN;
/* 124 */ public ulong seed;
/* 125 */ public int K { get; }
/* 128 */ public string Journal(DateTime date)
/* 137 */ internal static string join(double[] a, string fmt)
```

GmmSession

```
/* 148 */ public DateTime day, closeTime;
/* 149 */ public string contract;
/* 150 */ public bool roll;
/* 151 */ public double[] prices = new double[0], vols = new double[0];
/* 152 */ public int bars;
/* 153 */ public double sVwap = double.NaN, sigma = double.NaN, totalVol;
/* 156 */ public static GmmSession FromBars(DateTime tradingDay, IList<RingBar> rth1m, double tick, bool
roll, string contract, int minBars = 60)
```

SigmaMixtureResult

```
/* 174 */ public bool ok;
/* 175 */ public string reason;
/* 176 */ public ProfileMixtureResult mix;
/* 177 */ public double priorVwap = double.NaN, sigmaHat = double.NaN;
/* 178 */ public double[] levels = new double[0];
/* 179 */ public int sessions;
/* 180 */ public string contract;
/* 181 */ public List<DateTime> days = new List<DateTime>();
```

_ProfileStats

```
/* 193 */ public static EntropyResult Entropy(double[] vols, EEntropyRows rows, double mmN)
/* 217 */ public static double MmN(EMmN mode, int bars, double trades, double contracts)
/* 233 */ public static double KL(double[] pPrices, double[] pVols, double[] qPrices, double[] qVols, double
tick, double eps)
/* 247 */ public static double KLKdePrior(double[] pPrices, double[] pVols, double[] qPrices, double[]
qVols, double tick, double priorHcv, double eps)
/* 256 */ public static double KLKde(double[] pPrices, double[] pVols, double[] qPrices, double[] qVols, doub
le tick, double h, double eps)
/* 303 */ public static double[] KdeOnGrid(double[] prices, double[] vols, double gridLo, int gridN, double t
ick, double h, double trunc)
/* 325 */ public static double ParkinsonPerBar(IList<RingBar> bars, out int n)
/* 337 */ public static double ParkinsonPerBar(IList<RingBar> bars)
/* 346 */ public static double ParkinsonSessionPrice(IList<RingBar> bars, double refPrice, int minBars)
/* 354 */ public static double EwmaForecast(IList<double> valuesOldestFirst, int span)
/* 375 */ public static List<RingBar> AggregateBars(IList<RingBar> bars, TimeSpan size)
/* 406 */ public static double WilderAtr(IList<RingBar> bars, int len)
/* 423 */ public static void BarRows(RingBar b, double tick, int mode, Action<long, double> add)
/* 437 */ public static bool Histogram(IList<RingBar> bars, IList<int> idx, double tick, int mode, out
double[] prices, out double[] vols)
/* 447 */ public static double WeightedSd(double[] prices, double[] vols)
/* 459 */ public static double Percentile(IList<double> values, double q)
/* 476 */ public static double MidRank(IList<double> values, double x)
/* 486 */ public const int BOOTSTRAP_BLOCK = 5, BOOTSTRAP_DRAWS = 500;
/* 489 */ public static BootstrapResult BootstrapPoc(IList<RingBar> bars, double tick, int distribution, ulon
g seed)
/* 495 */ public static ulong BootstrapSeed(DateTime tradingDay, int block, int draws)
/* 503 */ public static BootstrapResult BootstrapPoc(IList<RingBar> bars, double tick, int distribution, int
block, int draws, ulong seed)
/* 540 */ public const int KDE_SWEEP_POINTS = 30, KDE_CV_POINTS = 25;
/* 541 */ public const double KDE_CV_LO = 0.15, KDE_CV_HI = 1.2;
/* 544 */ public static double[] KdeCvGrid(double sigma)
/* 559 */ public static KdeResult KdeSweep(IList<RingBar> bars, double tick, int distribution, double mcPoc
= double.NaN, double medFloorTicks = 1, int folds = 5, ECvFolding folding = ECvFolding.Contiguous)
/* 588 */ public static KdeResult KdeSweep(double[] prices, double[] vols, double tick, double medFloorTicks
= 1)
/* 626 */ public static double[] CvLogLik(IList<RingBar> bars, double tick, int distribution, double[] hs, in
t folds, ECvFolding folding)
/* 672 */ public static VtResult VtPrices(IList<RingBar> bars, double tick, int distribution, double atr30, i
nt top = 3)
```

```
/* 700 */ public static List<LevelRef> BootstrapLevels(BootstrapResult b, DateTime created, DateTime t0, Date
Time t1, int stream, string tag)
/* 709 */ public static List<LevelRef> VtLevels(VtResult vt, DateTime created, DateTime t0, DateTime t1, int
stream, string tag)
/* 719 */ public const int GMMP_COMPONENT = 27;
/* 720 */ public const int A232_WINDOW = 20, A232_EWMA_SPAN = 10, A232_MIN_BARS = 60, A232_KMAX = 4;
/* 721 */ public const double A232_BIN = 0.05;
/* 724 */ public static ulong MixtureSeed(DateTime tradingDay)
/* 727 */ public static double MixtureNeff(EGmmNeff mode, int bars, double[] vols)
/* 742 */ public static ProfileMixtureResult ProfileMixture(double[] prices, double[] vols, int kMax, ulong s
eed, EGmmNeff neff, int bars, double tick, GmmFitConfig cfg = null)
/* 763 */ public static void MixtureVoids(GmmModel m, double tick, out double[] voidPrice, out double[] voidD
epth)
/* 790 */ public static List<LevelRef> MixtureLevels(ProfileMixtureResult r, double edgeC, DateTime created,
DateTime t0, DateTime t1, int stream, string tag)
/* 812 */ public static SigmaMixtureResult SigmaMixture(IList<GmmSession> sessions, ulong seed, int window =
A232_WINDOW, int ewmaSpan = A232_EWMA_SPAN, double bin = A232_BIN, int kMax = A232_KMAX, GmmFitConfig cfg = n
ull)
/* 851 */ public static List<LevelRef> SigmaLevels(SigmaMixtureResult r, DateTime created, DateTime t0, DateT
ime t1, int stream, string tag)
```

_PriorDayValue

```
/* 864 */ public int keep = 30;
/* 865 */ public void Set(DateTime day, double x)
/* 872 */ public double Before(DateTime day)
```

_EntropyTracker

```
/* 886 */ public readonly int lookback; public readonly double minFill;
/* 890 */ public _EntropyTracker(int lookback, double minFill, IEnumerable<int> slots)
/* 896 */ public bool HasSlot(int slot)
/* 897 */ public int Count(int slot)
/* 900 */ public bool Push(int slot, DateTime day, double x)
/* 912 */ public double Percentile(int slot, double x)
/* 920 */ public static int Slot(DateTime t)
```

_GmmProfileTracker

```
/* 930 */ public double tick;
/* 931 */ public int stream = 1;
/* 932 */ public int kMaxPrice = 3, window = _ProfileStats.A232_WINDOW, ewmaSpan = _ProfileStats.A232_EWMA_SP
AN, minBars = _ProfileStats.A232_MIN_BARS, kMaxSigma = _ProfileStats.A232_KMAX;
/* 933 */ public double bin = _ProfileStats.A232_BIN, edgeC = 1.0;
/* 934 */ public EGmmNeff neff = EGmmNeff.Bars;
/* 935 */ public bool priceMixture = true, sigmaMixture = true;
/* 936 */ public int keepSessions = 40;
/* 937 */ public Action<string> journal;
/* 944 */ public ProfileMixtureResult LastPrice { get; private set; }
/* 945 */ public SigmaMixtureResult LastSigma { get; private set; }
/* 946 */ public int Fits { get; private set; }
/* 948 */ public _GmmProfileTracker(double tick)
/* 951 */ public void OnSessionClose(DateTime tradingDay, IList<RingBar> rth1m, bool roll, string contract =
null)
/* 969 */ public int SkippedSessions(string contract)
/* 971 */ public IDictionary<string, int> SkippedByContract { get; }
/* 974 */ public List<LevelRef> Levels()
```

`_RangeProfile.Function.CS`

(file scope)

```
/* 17 */ public enum ERangeRollPolicy { Reject = 0, Truncate = 1 }
/* 19 */ public sealed class RangeProfileConfig
/* 55 */ public sealed class RangeHistogram
/* 69 */ public sealed class RangeStats
/* 76 */ public sealed class RangeAccumulator
/* 135 */ public static class RangeProfileBuilder
/* 209 */ public sealed class RangeProfileHandle
```

RangeProfileConfig

```
/* 21 */ public bool enabled = false;
/* 22 */ public int[] streams = { 1 };
/* 23 */ public int distribution = 2;
/* 24 */ public bool useTickLadder = false;
/* 25 */ public bool rthOnly = false;
/* 26 */ public ERangeRollPolicy rollPolicy = ERangeRollPolicy.Reject;
/* 27 */ public double valueAreaPercent = double.NaN;
/* 28 */ public int maxRows = 4000;
/* 29 */ public int cacheSize = 8;
/* 30 */ public int maxOpenHandles = 8;
/* 31 */ public int ringBars = 2880;
/* 33 */ public string Validate()
/* 47 */ public string Canon()
```

RangeHistogram

```
/* 57 */ public double tick;
/* 58 */ public double[] prices = new double[0], vols = new double[0];
/* 59 */ public double[] ask, bid;
/* 60 */ public int bars, ladderBars;
/* 61 */ public int startBar = -1, endBar = -1;
/* 62 */ public DateTime t0, t1, firstTime, lastTime;
/* 63 */ public double open, high, low, close, totalVol;
/* 64 */ public bool isPartial, crossesRoll, approximate = true, tooManyRows;
/* 65 */ public string tag;
/* 66 */ public int Rows { get; }
```

RangeStats

```
/* 71 */ public int bars, expectedBars, ladderBars, rows; public double coverage = double.NaN, totalVol;
/* 72 */ public override string ToString()
```

RangeAccumulator

```
/* 78 */ public readonly _ProfileMath.Accumulator acc;
/* 80 */ public readonly RangeProfileConfig cfg;
/* 81 */ public int ladderBars, bidAskBars;
/* 82 */ public bool crossesRoll, isPartial;
/* 83 */ public DateTime t0, t1;
/* 85 */ public RangeAccumulator(double tick, RangeProfileConfig cfg)
/* 87 */ public int Bars { get; }
/* 89 */ public void Add(RingBar b, LadderBar lb)
/* 112 */ public RangeHistogram Build(string tag = null)
```

RangeProfileBuilder

```
/* 145 */ public static RangeHistogram Build(_BarRing ring, DateTime t0, DateTime t1, double tick,
RangeProfileConfig cfg, Func<RingBar, LadderBar> ladder, bool t0Exclusive = false, string tag = null)
/* 168 */ public static RangeHistogram BuildBars(_BarRing ring, int barsBack, int count, double tick, RangePr
ofileConfig cfg, Func<RingBar, LadderBar> ladder, string tag = null)
/* 183 */ public static int ExpectedSlots(DateTime t0, DateTime t1, TimeSpan res, Func<DateTime, bool>
isOpen, bool t0Exclusive = false)
/* 192 */ public static RangeStats Stats(RangeHistogram h, int expectedBars)
/* 199 */ public static string Tag(DateTime t0, DateTime t1)
```

RangeProfileHandle

```
/* 214 */ public readonly int stream;
/* 215 */ public readonly DateTime t0;
/* 216 */ public readonly string Tag;
/* 217 */ public bool IsClosed { get; private set; }
/* 218 */ public DateTime ClosedAt { get; private set; }
/* 219 */ public DateTime ClosedDay { get; private set; }
/* 220 */ public bool CrossesRoll { get; }
/* 221 */ public int Bars { get; }
/* 223 */ public int LastBar { get; }
/* 225 */ public Action<RangeProfileHandle> beforeRead;
```

```
/* 229 */ public RangeProfileHandle(int stream, DateTime t0, string tag, double tick, RangeProfileConfig cfg)
/* 236 */ public bool Step(RingBar b, LadderBar lb)
/* 251 */ public void Backfill(_BarRing ring, Func<RingBar, LadderBar> ladder)
/* 259 */ public void Close(DateTime t1)
/* 265 */ public RangeHistogram Profile
```

`_RegimeEngine.Function.CS`

(file scope)

```
/* 27 */ public enum ERegimeMethod { Hmm = 0, Gmm = 1 }
/* 28 */ public enum ERegimeKSelect { Fixed = 0, Bic = 1 }
/* 29 */ public enum ERegimeRefit { Monthly = 0, EveryNSessions = 1, Quarterly = 2 }
/* 30 */ public enum ERegimeFallback { AtrTercile = 0, Unknown = 1 }
/* 31 */ public enum ERegimeFeature { Return = 0, LogHv20 = 1, Tsmom63z = 2, Parkinson10 = 3 }
/* 34 */ public sealed class RegimeConfig
/* 93 */ public sealed class RegimeState
/* 113 */ public sealed class RegimeFit
/* 144 */ public sealed class _RegimeEngine
```

RegimeConfig

```
/* 36 */ public bool enabled = false;
/* 37 */ public ERegimeMethod method = ERegimeMethod.Hmm;
/* 38 */ public int K = 3;
/* 39 */ public ERegimeKSelect kSelect = ERegimeKSelect.Fixed;
/* 40 */ public ERegimeFeature[] features = { ERegimeFeature.Return, ERegimeFeature.LogHv20, ERegimeFeature.T
smom63z };
/* 41 */ public int fitSessions = 1000;
/* 42 */ public int minSessions = 500;
/* 43 */ public ERegimeRefit refit = ERegimeRefit.Monthly;
/* 44 */ public int refitEveryN = 21;
/* 45 */ public int restarts = 10;
/* 46 */ public int maxIter = 200;
/* 47 */ public double tol = 1e-6;
/* 48 */ public double regCovar = 1e-6;
/* 49 */ public double transPrior = 1;
/* 50 */ public int warmFilter = 63;
/* 51 */ public double switchProb = 0.60;
/* 52 */ public double switchMargin = 0.10;
/* 53 */ public ulong seed = 20261021;
/* 54 */ public ERegimeFallback fallback = ERegimeFallback.AtrTercile;
/* 55 */ public bool publish = false;
/* 57 */ public int atrPctLookback = 250;
/* 58 */ public int parkinsonBars = 10;
/* 59 */ public bool bicDiagnostic = true;
/* 60 */ public double rollSuspectZ = 5;
/* 61 */ public int gapBusinessDays = 4;
/* 63 */ public int D { get; }
/* 65 */ public string Validate()
/* 87 */ public RegimeConfig Copy()
/* 89 */ public int IndexOf(ERegimeFeature f)
```

RegimeState

```
/* 95 */ public bool Valid;
/* 96 */ public int Label = -1;
/* 97 */ public int RawArgmax = -1;
/* 98 */ public double[] Probs = new double[0];
/* 99 */ public DateTime AsOf = DateTime.MinValue;
/* 100 */ public DateTime FittedThrough = DateTime.MinValue;
/* 101 */ public int K;
/* 102 */ public double LogLik = double.NaN;
/* 103 */ public int Refits;
/* 104 */ public bool UsingFallback;
/* 105 */ public int RelabelConflicts;
/* 107 */ public RegimeState Copy()
/* 109 */ public static RegimeState Neutral(int K)
```

RegimeFit

```
/* 115 */ public ERegimeMethod method;
/* 116 */ public int K, D;
/* 117 */ public double[] pi;
/* 118 */ public double[,] A;
/* 119 */ public double[][] mu;
/* 120 */ public double[][,] cov;
/* 121 */ public double[][,] L;
/* 122 */ public double[] zMean, zSd;
/* 123 */ public double logLik = double.NaN, bic = double.NaN;
/* 124 */ public int iters, gmmIters;
/* 125 */ public bool converged;
/* 126 */ public DateTime fittedThrough = DateTime.MinValue, windowStart = DateTime.MinValue;
/* 127 */ public int window;
/* 128 */ public string failReason;
/* 130 */ public GmmModel gmmInit, gmm;
/* 131 */ public double[] hmmInitPi; public double[,] hmmInitA; public double[][] hmmInitMu; public double[]
[,] hmmInitCov;
/* 132 */ public double[][] gamma;
/* 134 */ public static int ParamCount(ERegimeMethod m, int K, int D)
/* 137 */ public void Raw(int k, out double[] m, out double[,] c)
```

_RegimeEngine

```
/* 146 */ public RegimeConfig config = new RegimeConfig();
/* 147 */ public string label = "REGIME";
/* 148 */ public Action<string> journal;
/* 149 */ public Action<string> warnOnce;
/* 151 */ public Action<RegimeFit> onRefit;
/* 166 */ public bool Configured { get; private set; }
/* 168 */ public RegimeState State { get; }
/* 169 */ public RegimeFit Model { get; }
/* 170 */ public int Refits { get; }
/* 171 */ public int RelabelConflicts { get; }
/* 172 */ public int ValidObservations { get; }
/* 174 */ public string Configure()
/* 182 */ public void Reset()
/* 207 */ public void AddDay(DateTime date, double adjClose, double hv20, double hv63, double high, double lo
w, double atr20d, bool rollDay)
/* 299 */ public static double AtrPercentile(double[] ring, int next, int count, double cur)
/* 308 */ public static int AtrBucket(double pct, int K)
/* 311 */ public static int Hysteresis(int current, double[] p, double switchProb, double switchMargin)
/* 320 */ public static int Argmax(double[] p)
/* 434 */ public static void ZStats(double[][] x, out double[] mean, out double[] sd)
/* 445 */ public static double[] Standardise(RegimeFit m, double[] raw)
/* 453 */ public static double[] FilterStep(RegimeFit m, double[] alpha, double[] z)
/* 475 */ public static RegimeFit FitWindow(double[][] z, int K, RegimeConfig cfg, ulong seed, int orderFeatu
re, int tieFeature)
/* 609 */ public static void OrderStates(RegimeFit r, int of, int tf)
/* 625 */ public static double Bhattacharyya(double[] m1, double[,] c1, double[] m2, double[,] c2)
/* 642 */ public static int[] BestMatch(RegimeFit oldFit, RegimeFit newFit)
```

`_RegimeGate.Function.CS`

(file scope)

```
/* 25 */ public enum ERegimeGateArm { U = 0, G = 1, A = 2 }
/* 28 */ public sealed class RegimeGateConfig
/* 41 */ public sealed class A263Config
/* 55 */ public struct RegimeGateDecision
/* 68 */ public enum ERegimeGateReason { Ok = 0, Warmup = 1, Invalid = 2, Fallback = 3 }
/* 72 */ public sealed class RegimeGateCounts
/* 88 */ public static class RegimeGate
/* 216 */ public sealed class AtrTercileTracker
/* 232 */ public struct A263Action
/* 250 */ public sealed class A263Core
```

RegimeGateConfig

```
/* 30 */ public ERegimeGateArm arm = ERegimeGateArm.U;
/* 32 */ public ulong placeboSeed = RegimeGate.PlaceboSeed;
/* 33 */ public string Validate()
```

A263Config

```
/* 43 */ public double k = 0.5;
/* 44 */ public int atrLength = 20;
/* 45 */ public TimeSpan rthOpen = new TimeSpan(9, 30, 0);
/* 46 */ public TimeSpan activeEnd = new TimeSpan(15, 30, 0);
/* 47 */ public TimeSpan exitTime = new TimeSpan(15, 55, 0);
/* 48 */ public TimeSpan rthClose = new TimeSpan(16, 0, 0);
/* 49 */ public int contracts = 1;
/* 51 */ public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);
```

RegimeGateDecision

```
/* 57 */ public DateTime session, asOf;
/* 58 */ public ERegimeGateArm arm;
/* 59 */ public int label, raw, atrTercile;
/* 60 */ public double pFilt;
/* 61 */ public bool fb, usable, wouldBlock, allow;
/* 65 */ public ERegimeGateReason reason;
```

RegimeGateCounts

```
/* 74 */ public int sessions, usable, warmup, invalid, fallback, blocked, noRth;
/* 75 */ public void Add(RegimeGateDecision d, bool noRthDay = false)
/* 85 */ public string Suffix()
```

RegimeGate

```
/* 90 */ public const int BlockedLabel = 2;
/* 91 */ public const ulong PlaceboSeed = 20261022;
/* 92 */ public const int ArmAFitSessions = 100000;
/* 93 */ public const string Version = "A263-RG-2";
/* 96 */ public static void ApplyArm(RegimeGateConfig g, RegimeConfig r)
/* 108 */ public static RegimeGateDecision Decide(ERegimeGateArm arm, RegimeState s, DateTime session, int at
rTercile)
/* 130 */ public static string Line(RegimeGateDecision d, bool entrySignal, string cfgHash, ulong
placeboSeed)
/* 138 */ public static IDictionary<string, string> TradeFields(RegimeGateDecision d, string cfgHash, ulong p
laceboSeed)
/* 153 */ public static readonly string[] HashFields =
/* 164 */ public static Dictionary<string, string> CanonFields(RegimeConfig r, RegimeGateConfig g,
A263Config h)
/* 186 */ public static string Canon(IDictionary<string, string> fields)
/* 198 */ public static string Canon(RegimeConfig r, RegimeGateConfig g, A263Config h)
/* 201 */ public static string Fnv1a64(string s)
/* 208 */ public static string Hash(RegimeConfig r, RegimeGateConfig g, A263Config h)
/* 209 */ public static string Hash(IDictionary<string, string> fields)
```

AtrTercileTracker

```
/* 219 */ public DateTime LastDate { get; private set; }
/* 220 */ public int Tercile { get; private set; }
/* 221 */ public AtrTercileTracker(int lookback, int K)
/* 222 */ public void AddDay(DateTime date, double atr20d)
```

A263Action

```
/* 234 */ public bool latched;
/* 235 */ public bool sendStop;
/* 236 */ public double stop;
/* 237 */ public bool exit;
```

A263Core

```
/* 252 */ public readonly A263Config config;
/* 253 */ public A263Core(A263Config c)
/* 254 */ public DateTime Session { get; private set; }
/* 255 */ public double Open { get; private set; }
/* 256 */ public double Atr { get; private set; }
/* 257 */ public DateTime AtrAsOf { get; private set; }
/* 258 */ public double Threshold { get; private set; }
/* 259 */ public TimeSpan ExitAt { get; private set; }
/* 260 */ public TimeSpan? EarlyClose { get; private set; }
/* 261 */ public bool NoRth { get; private set; }
/* 262 */ public bool AtrRejected { get; private set; }
/* 263 */ public bool Entered { get; private set; }
/* 264 */ public bool Allowed { get; private set; }
/* 267 */ public bool IsLatchBar(DateTime etClose)
/* 275 */ public void Latch(DateTime etClose, double barOpen, double atr20d, DateTime atrAsOf, bool allow, Ti
meSpan? earlyCloseEt = null, bool noRth = false)
/* 285 */ public bool HasSignal { get; }
/* 287 */ public A263Action Step(DateTime etClose, bool isLong)
/* 299 */ public string Suffix()
```

`_Rng.Function.CS`

(file scope)

```
/* 12 */ public sealed class _Rng
```

_Rng

```
/* 16 */ public _Rng(ulong seed)
/* 18 */ public ulong State { get; }
/* 21 */ public ulong Next()
/* 30 */ public static ulong Mix(ulong z)
/* 41 */ public double NextDouble()
/* 44 */ public int NextInt(int n)
/* 68 */ public static ulong Seed(params long[] parts)
/* 80 */ public static long Hash(string s)
```

`_RollDetector.Function.CS`

(file scope)

```
/* 16 */ public sealed class _RollDetector
/* 58 */ public sealed class _RollPending
```

_RollDetector

```
/* 18 */ public bool series;
/* 19 */ public bool table;
/* 20 */ public int gapTicks = 8;
/* 21 */ public ICollection<DateTime> dates;
/* 29 */ public IList<int> RollBars { get; }
/* 30 */ public int LastRollBar { get; }
/* 32 */ public void Reset()
/* 36 */ public bool Step(int bar, DateTime tradingDay, double open, double prevClose, string name, double ti
ck)
```

_RollPending

```
/* 62 */ public bool Armed { get; }
/* 63 */ public void Arm(DateTime data1RollTime)
/* 64 */ public bool Fire(int bar, DateTime time)
/* 71 */ public void Reset()
```

`_TickStream.Function.CS`

(file scope)

```
/* 15 */ public sealed class TickConfig
/* 42 */ public sealed class TickBarStats
/* 55 */ internal sealed class _TickStream
```

TickConfig

```
/* 17 */ public bool enabled = false;
/* 18 */ public int historyTicks = 2000000;
/* 19 */ public bool realTime = true;
/* 20 */ public double burstZ = 3.0;
/* 21 */ public int burstNormLength = 200;
/* 22 */ public int burstMinSamples = 30;
/* 23 */ public double[] sizeBuckets = { 1, 2, 5, 10, 20, 50 };
/* 24 */ public double largePrintSize = 20;
/* 25 */ public bool sizeOutputsVerified = false;
/* 27 */ public string Validate()
```

TickBarStats

```
/* 44 */ public bool valid;
/* 45 */ public string quality = "Unknown";
/* 46 */ public int prints;
/* 47 */ public int maxPrintsPerSecond;
/* 48 */ public double interPrintMedianMs = double.NaN, interPrintP10Ms = double.NaN;
/* 49 */ public double burstZ = double.NaN; public bool burst;
/* 50 */ public int[] sizeHistogram;
/* 51 */ public int largePrints = -1;
/* 52 */ public double volume = double.NaN;
```

_TickStream

```
/* 57 */ public TickConfig config = new TickConfig();
/* 58 */ public Action<string> journal;
/* 70 */ public bool Configured { get; private set; }
/* 71 */ public bool HistoryComplete { get; }
/* 72 */ public TickBarStats Last { get; private set; }
/* 74 */ public string Configure()
/* 80 */ public void Reset()
/* 88 */ public void Enqueue(DateTime time, double price, double size)
/* 93 */ public void MarkHistoryComplete()
/* 114 */ public TickBarStats Aggregate(DateTime barStart, DateTime barEnd)
/* 174 */ public static double Quantile(double[] sorted, double q)
```

`_TodNormaliser.Function.CS`

(file scope)

```
/* 15 */ public enum ETodMetric { None = 0, Volume = 1 << 0, Delta = 1 << 1, Range = 1 << 2, TrueRange = 1
<< 3 }
/* 18 */ public struct TodSample
/* 28 */ internal sealed class _TodNormaliser
```

TodSample

```
/* 20 */ public DateTime sessionDate;
/* 21 */ public int minuteOfSession;
/* 22 */ public bool halfDay;
/* 23 */ public bool splice;
/* 24 */ public double volume, delta, range, trueRange;
/* 25 */ public int deltaTier;
```

_TodNormaliser

```
/* 30 */ public ETodMetric metrics = ETodMetric.None;
/* 31 */ public int sessions = 20;
/* 32 */ public int minSessions = 10;
/* 33 */ public int bucketMinutes = 1;
/* 34 */ public string label = "TOD";
/* 35 */ public Action<string> journal;
/* 36 */ public Action<string> warnOnce;
/* 53 */ public bool Configured { get; private set; }
/* 55 */ public string Validate()
/* 65 */ public string Configure()
/* 79 */ public void Reset()
/* 96 */ public bool Valid(ETodMetric m)
/* 98 */ public double Z(ETodMetric m)
/* 99 */ public double Median(ETodMetric m)
/* 101 */ public double Pct(ETodMetric m)
/* 103 */ public double ZCum(ETodMetric m)
/* 105 */ public int Sessions(ETodMetric m)
/* 109 */ public IList<double> VolumePool(int minuteOfSession)
/* 120 */ public bool Add(int barNumber, TodSample s)
```

`_VolumeProfileBase.Strategy.CS`

(file scope)

```
/* 24 */ public abstract class _VolumeProfileBase : _CustomSignal
```

_VolumeProfileBase

```
/* 27 */ public const double DEFAULT_HVN_PERCENTILE = 0.8;
/* 28 */ public const double DEFAULT_LVN_PERCENTILE = 0.2;
/* 29 */ public const int DEFAULT_SMOOTH_TICKS = 3;
/* 30 */ public const int DEFAULT_MIN_NODE_TICKS = 3;
/* 31 */ public const double PRICE_EPSILON = 1e-12;
/* 32 */ public const double DEFAULT_MAX_LVN_PCT_OF_RANGE = 0.12;
/* 33 */ public const int DEFAULT_VWAP_LENGTH = 20;
/* 34 */ public const int DEFAULT_CONFLUENCE_LOOKBACK = 4;
/* 35 */ public const int DEFAULT_CONFLUENCE_ALIGN_TICKS = 4;
/* 36 */ public const int DEFAULT_CONFLUENCE_TOP_COUNT = 5;
/* 37 */ public const int DEFAULT_CONFLUENCE_TARGET_CLEAR_TICKS = 1;
/* 38 */ public const double DEFAULT_CONFLUENCE_STREAM_WEIGHT_1 = 1.00;
/* 39 */ public const double DEFAULT_CONFLUENCE_STREAM_WEIGHT_2 = 1.25;
/* 40 */ public const double DEFAULT_CONFLUENCE_STREAM_WEIGHT_3 = 1.50;
/* 41 */ public const double DEFAULT_CONFLUENCE_NAKED_MULTIPLIER = 1.15;
/* 42 */ public const double DEFAULT_CONFLUENCE_REFINEMENT = 0.25;
/* 43 */ internal const double MAX_PARENT_CREDIT_MULTIPLIER = 1.25;
/* 44 */ public const double DEFAULT_CONFLUENCE_OVERLAY_FACTOR = 0.50;
/* 45 */ public const double DEFAULT_CONFLUENCE_SHARE_PIVOT = 0.15;
/* 46 */ public const double DEFAULT_EXCEPTIONAL_VOLUME_PERCENTILE = 0.95;
/* 47 */ public const int MAX_DENSE_ROWS = 20000;
/* 49 */ internal const double FALLBACK_TICK = 0.01;
/* 50 */ internal const double CLUSTER_SPLIT_TOL_MULTIPLE = 4.0;
/* 51 */ internal const int CLUSTER_MIN_SPLIT_COUNT = 3;
/* 52 */ internal const int ZONE_TOP_SURVIVORS = 3;
/* 53 */ internal const double FALLBACK_TIMEFRAME_WEIGHT_STEP = 0.25;
/* 54 */ internal const double SHARE_FACTOR_MIN = 0.5;
/* 55 */ internal const double SHARE_FACTOR_MAX = 1.5;
/* 56 */ internal const double LEVEL_MATCH_TICK_FRACTION = 0.51;
/* 57 */ public const int DEFAULT_LVN_HVN_MAX_GAP_TICKS = 4;
/* 63 */ public enum EVwapType { Disabled = 0, Session = 1, Length = 2, AdaptiveFromDate = 3, Week = 4,
Month = 5, Quarter = 6, RollDate = 7 }
/* 79 */ internal struct Row
/* 88 */ public struct Node
/* 135 */ public class VpExtra
/* 259 */ public struct LevelVol
/* 281 */ public enum EConfluenceKind { Poc = 0, Vah = 1, Val = 2, Hvn = 3, Vwap1 = 4, Vwap2 = 5, Vwap3 = 6,
NakedPoc = 7, Lvn = 8 }
/* 298 */ public enum EConfluenceRole { Acceptance = 0, Rejection = 1, Magnet = 2, Mixed = 3 }
/* 309 */ public enum ESuggestedUse { Target = 0, Entry = 1, Conflict = 2 }
/* 319 */ public enum EConfluenceLocation { Unknown = 0, Premium = 1, Discount = 2, InsideValue = 3 }
/* 331 */ public enum EVpDirection { Bearish = -1, Bullish = 1 }
/* 340 */ public struct ConfluenceHit
/* 359 */ public class ConfluenceDrop
```

```
/* 372 */ public class ConfluenceZone
/* 417 */ public class ConfluenceTarget
/* 491 */ [Input] public double HvnPercentile { get; set; }
/* 496 */ [Input] public double LvnPercentile { get; set; }
/* 497 */ [Input] public int MinNodeTicks { get; set; }
/* 498 */ [Input] public bool TrackNakedPoc { get; set; }
/* 499 */ [Input] public int NakedPocTickTolerance { get; set; }
/* 502 */ [Input] public EVwapType Vwap1Type { get; set; }
/* 503 */ [Input] public int Vwap1Length { get; set; }
/* 504 */ [Input] public string Vwap1FromDate { get; set; }
/* 505 */ [Input] public int Vwap1DataStream { get; set; }
/* 507 */ [Input] public EVwapType Vwap2Type { get; set; }
/* 508 */ [Input] public int Vwap2Length { get; set; }
/* 509 */ [Input] public string Vwap2FromDate { get; set; }
/* 510 */ [Input] public int Vwap2DataStream { get; set; }
/* 512 */ [Input] public EVwapType Vwap3Type { get; set; }
/* 513 */ [Input] public int Vwap3Length { get; set; }
/* 514 */ [Input] public string Vwap3FromDate { get; set; }
/* 515 */ [Input] public int Vwap3DataStream { get; set; }
/* 538 */ public int ConfluenceDataStream1
/* 548 */ public int ConfluenceDataStream2
/* 557 */ public int ConfluenceDataStream3
/* 566 */ public double ConfluenceWeight1
/* 575 */ public double ConfluenceWeight2
/* 584 */ public double ConfluenceWeight3
/* 594 */ public int ConfluenceLookbackProfiles
/* 603 */ public int ConfluenceAlignTicks
/* 611 */ public int ConfluenceTopCount
/* 619 */ public bool ConfluenceIncludeHvns { get; set; }
/* 621 */ public double MaxLvnFractionOfRange { get; set; }
/* 625 */ public bool ConfluenceIncludeVwapSlots { get; set; }
/* 629 */ public double ConfluenceTargetMinZScore { get; set; }
/* 634 */ public int ConfluenceTargetClearTicks
/* 642 */ public int ConfluenceTargetCount
/* 668 */ internal DateTime vwapSessionDate(DateTime barTime)
/* 670 */ internal bool vwapRollOn(int stream)
/* 687 */ internal _VpVwapEngine m_vwap1;
/* 688 */ internal _VpVwapEngine m_vwap2;
/* 689 */ internal _VpVwapEngine m_vwap3;
/* 690 */ internal readonly List<ConfluenceZone> m_confluenceZones = new List<ConfluenceZone>();
/* 709 */ public sealed class ProfileMath : _ProfileMath
/* 716 */ public enum EDevelopingSource { MultiCharts = 0, Own = 1 }
/* 718 */ public enum EDevProfileSourceMode { Auto = 0, VolumeAtPrice = 1, BarUniform = 2, BarTypical = 3 }
/* 720 */ public enum EValueAreaMethod { MultiCharts = 0, VolumeExpansion = 1, Sigma = 2 }
/* 722 */ public const double DEFAULT_VALUE_AREA_PERCENT = 0.70;
/* 723 */ public const double DEFAULT_VALUE_AREA_SIGMA = 1.0;
/* 728 */ [Input] public EDevelopingSource DevelopingSource { get; set; }
/* 730 */ [Input] public EDevProfileSourceMode DevProfileSourceMode { get; set; }
/* 732 */ [Input] public int DevProfileDataStream { get; set; }
/* 734 */ [Input] public EValueAreaMethod ValueAreaMethod { get; set; }
/* 736 */ [Input] public bool LookaheadAudit { get; set; }
/* 739 */ public double ValueAreaPercent { get; set; }
/* 741 */ public double ValueAreaSigma { get; set; }
/* 743 */ protected bool AuditPlantLeak { get; set; }
/* 789 */ protected virtual bool isNewProfileWindow(int stream, int barIndex, DateTime barTime, DateTime prev
BarTime)
/* 795 */ protected virtual bool includeBarInProfile(DateTime barTime)
/* 884 */ protected void fillFromHistogram(VpExtra e, double[] prices, double[] vols)
/* 908 */ public VpExtra DevelopingExtra(int stream)
/* 919 */ public List<VpExtra> OwnCompletedProfiles(int count)
/* 1053 */ public ProfileMath.ValueRel ValueRelationship(VpExtra prev, VpExtra cur)
/* 1060 */ public bool IsPoorHigh(VpExtra e, double fraction = 0.5, int minRows = 2)
/* 1067 */ public bool IsPoorLow(VpExtra e, double fraction = 0.5, int minRows = 2)
/* 1074 */ public bool HasUnfinishedAuction(VpExtra e, bool high)
/* 1087 */ public List<double[]> SinglePrints(VpExtra e, int minTicks, double pct = 0.2)
/* 1099 */ public ProfileMath.Shape ProfileShape(VpExtra e)
/* 1110 */ public bool IsLevelNaked(VpExtra e, double level, int asOfBar)
/* 1129 */ public bool IsNakedVah(VpExtra e, int asOfBar)
/* 1130 */ public bool IsNakedVal(VpExtra e, int asOfBar)
/* 1136 */ public int AcceptanceOutsideValue(VpExtra reference, bool above, int minCloses)
/* 1158 */ protected virtual List<VpExtra> migrationProfiles(int stream, int count)
/* 1170 */ public int? ValueMigrationScore(int stream, int n = 3, double minChangeAtrFraction = 0.10, int atr
Len = 20, bool includeDeveloping = true, double dailyAtr = 0)
/* 1204 */ public ProfileMath.MigrationState ValueMigrationState(int stream, int n = 3, double minChangeAtrFr
action = 0.10, int atrLen = 20, bool includeDeveloping = true, double dailyAtr = 0, int upAt = 2, int downAt
= 2)
/* 1211 */ protected virtual bool profileCrossesRoll(VpExtra e)
```

```
/* 1221 */ public VpExtra Composite(int stream, int n)
/* 1247 */ public bool IsBalanced(int stream, int n, double minVaOverlap = 0.6)
/* 1257 */ public bool BalanceRange(int stream, int n, bool union, out double high, out double low)
/* 1281 */ public int ValueMigrationScore(int stream, double atr20Daily, int n = 3, double minChangeFraction
= 0.10, bool includeDeveloping = true)
/* 1299 */ public _VolumeProfileBase(object _ctx)
/* 1301 */ protected override void Create()
/* 1356 */ protected override void Destroy()
/* 1362 */ protected override void StartCalc()
/* 1394 */ protected override void CalcBar()
/* 1415 */ public VpExtra ExtraVolumeProfileForBar(int barIndex)
/* 1426 */ public VpExtra ExtraVolumeProfileForBar(int datastream, int barIndex)
/* 1468 */ public List<VpExtra> RecentCompletedProfiles(int datastream, int count, int asOfBar)
/* 1486 */ public List<VpExtra> NakedPocs(int datastream)
/* 1497 */ public List<VpExtra> NakedPocs(int datastream, int asOfBar)
/* 1520 */ public bool IsNewVolumeProfile()
/* 1528 */ public bool IsNewVolumeProfile(int datastream)
/* 1540 */ public bool IsNewVolumeProfile(int datastream, int barIndex)
/* 1575 */ public List<ConfluenceZone> GetConfluenceZones()
/* 1587 */ public List<ConfluenceZone> GetTopConfluenceZones()
/* 1596 */ public List<ConfluenceZone> GetTopConfluenceZones(int count)
/* 1613 */ public List<ConfluenceTarget> GetConfluenceTargets(double price, EVpDirection direction)
/* 1624 */ public List<ConfluenceTarget> GetConfluenceTargets(double price, EVpDirection direction, int
count)
/* 1638 */ public List<ConfluenceZone> GetConfluenceTargetZones(double price, EVpDirection direction)
/* 1646 */ public List<ConfluenceZone> GetConfluenceTargetZones(double price, EVpDirection direction, int cou
nt)
/* 1658 */ public void InvalidateConfluenceZones()
/* 1669 */ public void RefreshVolumePercentiles()
/* 1681 */ public double CurrentVwap1 { get; }
/* 1683 */ public double CurrentVwap2 { get; }
/* 1685 */ public double CurrentVwap3 { get; }
/* 1687 */ public double CurrentVwapSigma1 { get; }
/* 1690 */ public double CurrentVwap1Sigma { get; }
/* 1691 */ public double CurrentVwap2Sigma { get; }
/* 1693 */ public double CurrentVwap3Sigma { get; }
/* 1695 */ public bool Vwap1Active { get; }
/* 1697 */ public bool Vwap2Active { get; }
/* 1699 */ public bool Vwap3Active { get; }
/* 1701 */ public int CurrentVwap1Stream { get; }
/* 1703 */ public int CurrentVwap2Stream { get; }
/* 1705 */ public int CurrentVwap3Stream { get; }
/* 1776 */ internal int streamItemBarIndex(int stream)
/* 1790 */ internal static int toNativeItemBar(int chartBar)
/* 2322 */ internal List<VpExtra> confluenceLookbackExtras(int datastream)
/* 2408 */ internal double helperTickSize()
/* 2410 */ internal DateTime barTime(int stream, int chartBar)
/* 2611 */ protected void buildNodesFromLevels(VpExtra extra)
/* 2943 */ protected RangeProfileConfig RangeProfiles { get; }
/* 2945 */ protected WeeklyProfileConfig WeeklyProfiles { get; }
/* 2947 */ protected ProfileStatsConfig ProfileStats { get; }
/* 2949 */ protected BarLadderConfig BarLadders { get; }
/* 2974 */ protected virtual bool sessionOpenAt(DateTime slotStart)
/* 2976 */ protected virtual _OrderFlow.FootprintBar footprintForLadder(int barNumber)
/* 2978 */ protected virtual bool ladderHasFlow { get; }
/* 3053 */ public VpExtra RangeProfile(int stream, DateTime t0, DateTime t1)
/* 3057 */ public VpExtra RangeProfileAfter(int stream, DateTime t0, DateTime t1)
/* 3087 */ public VpExtra RangeProfileBars(int stream, int barsBack, int count)
/* 3097 */ public RangeProfileHandle OpenRange(int stream, DateTime t0, string tag)
/* 3116 */ public void CloseRange(RangeProfileHandle h, DateTime t1)
/* 3124 */ public VpExtra RangeHandleProfile(RangeProfileHandle h)
/* 3134 */ public RangeStats RangeStatsOf(VpExtra e)
/* 3141 */ public List<LevelRef> RangeLevels(VpExtra e, DateTime t0, DateTime t1, string tag)
/* 3278 */ public VpExtra Week(int weeksAgo)
/* 3302 */ public VpExtra WeeklyComposite(int nWeeks)
/* 3308 */ public int CompletedWeeks { get; }
/* 3309 */ public DateTime WeekKey(int weeksAgo)
/* 3312 */ public List<LevelRef> WeekLevels(int weeksAgo)
/* 3352 */ public LadderBar Ladder(int ago)
/* 3354 */ public LadderBar Ladder(int ago, _OrderFlow.FlowQuality minQuality)
/* 3364 */ public LadderBar LadderAt(DateTime barTime)
/* 3373 */ public LadderBar HtfLadder(int htfStream, int ago)
/* 3398 */ public VpExtra LadderProfile(LadderBar b)
/* 3419 */ public EntropyResult ProfileEntropy(VpExtra e, double mmN)
/* 3429 */ public List<RingBar> SessionBars(DateTime tradingDay, bool rthOnly)
```

_VolumeProfileBase.Row

```
/* 81 */ public double price;
/* 82 */ public double volume;
```

_VolumeProfileBase.Node

```
/* 90 */ public double low;
/* 91 */ public double high;
/* 92 */ public double peak;
/* 93 */ public double peakVol;
/* 94 */ public double strength;
/* 95 */ public bool isPoc;
```

_VolumeProfileBase.VpExtra

```
/* 137 */ public int dataStream;
/* 138 */ public int startBar;
/* 139 */ public int endBar;
/* 140 */ public double open;
/* 141 */ public double high;
/* 142 */ public double low;
/* 143 */ public double close;
/* 144 */ public double poc;
/* 145 */ public double pocVol;
/* 146 */ public bool pocComplete;
/* 147 */ public bool pocStale;
/* 148 */ public bool isDeveloping;
/* 149 */ public bool approximate;
/* 150 */ public bool isOwn;
/* 151 */ public bool isPartial;
/* 152 */ public bool crossesRoll;
/* 153 */ public bool pocNaked;
/* 154 */ public int pocTouchedBar;
/* 164 */ internal int lastNakedScanBar;
/* 165 */ internal bool nakedTracked;
/* 166 */ internal int ohlcStartBar;
/* 167 */ internal int ohlcFoldedThrough;
/* 168 */ internal int ohlcEndBar;
/* 169 */ internal bool nodesBuilt;
/* 170 */ internal int nodesBuiltForStart;
/* 171 */ internal int nodesBuiltGeneration = -1;
/* 172 */ internal int filledAtBar = -1;
/* 173 */ internal bool filledAtClose;
/* 174 */ internal int filledGeneration = -1;
/* 176 */ internal bool inClosedSet;
/* 177 */ internal int pctStampedFill = -1;
/* 178 */ internal int pctStampedClosedGen = -1;
/* 185 */ public bool IsNakedPoc(int asOfBar)
/* 190 */ public double vah;
/* 191 */ public double val;
/* 192 */ public double vwap1;
/* 193 */ public double vwap2;
/* 194 */ public double vwap3;
/* 195 */ public double vwap1Sigma;
/* 196 */ public double vwap2Sigma;
/* 197 */ public double vwap3Sigma;
/* 198 */ public int vwap1Stream;
/* 199 */ public int vwap2Stream;
/* 200 */ public int vwap3Stream;
/* 201 */ public readonly List<Node> hvns = new List<Node>();
/* 202 */ public readonly List<Node> lvns = new List<Node>();
/* 209 */ public readonly List<LevelVol> levels = new List<LevelVol>();
/* 210 */ public double maxTotalPercentile;
/* 211 */ public double maxAskPercentile;
/* 212 */ public double maxBidPercentile;
/* 214 */ public bool HasExceptionalVolume(double minPercentile)
/* 222 */ public bool HasExceptionalVolume()
/* 227 */ public List<LevelVol> ExceptionalLevels(double minPercentile)
/* 242 */ public LevelVol PeakTotalLevel()
```

_VolumeProfileBase.LevelVol

```
/* 261 */ public double price;
/* 262 */ public double total;
/* 263 */ public double ask;
/* 264 */ public double bid;
/* 265 */ public double totalPercentile;
/* 266 */ public double askPercentile;
/* 267 */ public double bidPercentile;
/* 269 */ public bool IsExceptional(double minPercentile)
```

_VolumeProfileBase.ConfluenceHit

```
/* 342 */ public int dataStream;
/* 343 */ public int profileStartBar;
/* 344 */ public int profileAge;
/* 345 */ public EConfluenceKind kind;
/* 346 */ public double price;
/* 347 */ public double weight;
/* 348 */ public string sourceVolumeId;
/* 349 */ public string absorbedLabels;
/* 350 */ public double credit;
/* 351 */ public double volumeShare;
/* 352 */ public bool naked;
/* 353 */ public bool independent;
```

_VolumeProfileBase.ConfluenceDrop

```
/* 361 */ public int dataStream;
/* 362 */ public int profileStartBar;
/* 363 */ public EConfluenceKind kind;
/* 364 */ public double price;
/* 365 */ public string sourceVolumeId;
/* 366 */ public string reason;
```

_VolumeProfileBase.ConfluenceZone

```
/* 374 */ public double low;
/* 375 */ public double high;
/* 376 */ public double center;
/* 377 */ public double rawScore;
/* 378 */ public double zScore;
/* 379 */ public double anchor;
/* 380 */ public double acceptanceScore;
/* 381 */ public double rejectionScore;
/* 382 */ public double contextScore;
/* 383 */ public int independentSources;
/* 384 */ public EConfluenceRole role;
/* 385 */ public EConfluenceLocation location;
/* 386 */ public ESuggestedUse suggestedUse;
/* 387 */ public double exitPrice;
/* 388 */ public double entryPrice;
/* 389 */ public double invalidationBelow;
/* 390 */ public double invalidationAbove;
/* 391 */ public int hitCount;
/* 392 */ public int streamCount;
/* 393 */ public int profileCount;
/* 394 */ public readonly List<ConfluenceHit> hits = new List<ConfluenceHit>();
/* 395 */ public readonly List<ConfluenceDrop> dropped = new List<ConfluenceDrop>();
/* 397 */ public bool HasStream(int dataStream)
/* 404 */ public bool HasKind(EConfluenceKind kind)
```

_VolumeProfileBase.ConfluenceTarget

```
/* 419 */ public double price;
/* 420 */ public double touch;
/* 421 */ public double distance;
/* 422 */ public EVpDirection direction;
/* 423 */ public ConfluenceZone zone;
```

`_VolumeProfileFuturesBase.Strategy.CS`

(file scope)

```
/* 49 */ public abstract class _VolumeProfileFuturesBase : _VolumeProfileBase
```

_VolumeProfileFuturesBase

```
/* 52 */ public const string DEFAULT_ASIA_START = "18:00";
/* 53 */ public const string DEFAULT_ASIA_END = "03:00";
/* 54 */ public const string DEFAULT_EUROPE_START = "03:00";
/* 55 */ public const string DEFAULT_EUROPE_END = "11:30";
/* 56 */ public const string DEFAULT_NEWYORK_START = "08:00";
/* 57 */ public const string DEFAULT_NEWYORK_END = "17:00";
/* 58 */ public const string DEFAULT_RTH_START = "09:30";
/* 59 */ public const string DEFAULT_RTH_END = "16:15";
/* 60 */ public const string DEFAULT_HALT_START = "17:00";
/* 61 */ public const string DEFAULT_HALT_END = "18:00";
/* 62 */ public const string DEFAULT_GLOBEX_OPEN = "18:00";
/* 63 */ public const string DEFAULT_GLOBEX_CLOSE = "17:00";
/* 66 */ public static readonly TimeSpan DefaultAsiaStart = new TimeSpan(18, 0, 0);
/* 67 */ public static readonly TimeSpan DefaultAsiaEnd = new TimeSpan(3, 0, 0);
/* 68 */ public static readonly TimeSpan DefaultEuropeStart = new TimeSpan(3, 0, 0);
/* 69 */ public static readonly TimeSpan DefaultEuropeEnd = new TimeSpan(11, 30, 0);
/* 70 */ public static readonly TimeSpan DefaultNewYorkStart = new TimeSpan(8, 0, 0);
/* 71 */ public static readonly TimeSpan DefaultNewYorkEnd = new TimeSpan(17, 0, 0);
/* 72 */ public static readonly TimeSpan DefaultRthStart = new TimeSpan(9, 30, 0);
/* 73 */ public static readonly TimeSpan DefaultRthEnd = new TimeSpan(16, 15, 0);
/* 74 */ public static readonly TimeSpan DefaultHaltStart = new TimeSpan(17, 0, 0);
/* 75 */ public static readonly TimeSpan DefaultHaltEnd = new TimeSpan(18, 0, 0);
/* 76 */ public static readonly TimeSpan DefaultGlobexOpen = new TimeSpan(18, 0, 0);
/* 77 */ public static readonly TimeSpan DefaultGlobexClose = new TimeSpan(17, 0, 0);
/* 84 */ public enum MarketSession { None = 0, Asia = 1 << 0, Europe = 1 << 1, NewYork = 1 << 2, Rth = 1 <<
3, Premarket = 1 << 4, PostMarket = 1 << 5 }
/* 98 */ public enum GlobexState { Closed, Halt, Open }
/* 107 */ protected TimeSpan m_asiaStart, m_asiaEnd, m_euStart, m_euEnd, m_nyStart, m_nyEnd;
/* 108 */ protected TimeSpan m_rthStart, m_rthEnd, m_haltStart, m_haltEnd, m_globexOpen, m_globexClose;
/* 113 */ public _VolumeProfileFuturesBase(object _ctx)
/* 115 */ protected override void Create()
/* 125 */ protected override void presetExtraPriceAction(PriceActionConfig c)
/* 127 */ protected override void StartCalc()
/* 138 */ protected override void CalcBar()
/* 153 */ protected virtual void configureSessions()
/* 175 */ protected _CmeSessionClock SessionClock { get; }
/* 204 */ protected DateTime ToEastern(DateTime barTime)
/* 212 */ protected DateTime EasternNow()
/* 214 */ protected GlobexState GetGlobexState()
/* 217 */ protected GlobexState GetGlobexState(DateTime barTime)
/* 223 */ protected MarketSession GetSessions()
/* 226 */ protected MarketSession GetSessions(DateTime barTime)
/* 228 */ protected bool InSession(MarketSession session)
/* 233 */ protected bool InAny(MarketSession sessions)
/* 238 */ protected bool IsRth()
/* 239 */ protected bool IsAsia()
/* 240 */ protected bool IsEurope()
/* 241 */ protected bool IsNewYork()
/* 243 */ protected bool IsLondonNyOverlap()
/* 255 */ protected DateTime GlobexDate(DateTime barTime)
/* 261 */ protected bool IsNewGlobexDay()
/* 272 */ public MarketSession PrimarySession()
/* 278 */ public MarketSession PrimarySession(DateTime eastern)
/* 281 */ public TimeSpan TimeSinceOpen(MarketSession session)
/* 292 */ public TimeSpan TimeSinceOpen(DateTime eastern, MarketSession session)
/* 294 */ public int MinutesSinceOpen(MarketSession session)
/* 299 */ public int MinutesSinceOpen(DateTime eastern, MarketSession session)
/* 301 */ public TimeSpan TimeUntilClose(MarketSession session)
/* 306 */ public TimeSpan TimeUntilClose(DateTime eastern, MarketSession session)
/* 308 */ public int MinutesUntilClose(MarketSession session)
/* 313 */ public int MinutesUntilClose(DateTime eastern, MarketSession session)
/* 315 */ public TimeSpan SessionLength(MarketSession session)
/* 321 */ public bool IsOpeningRange(MarketSession session, int minutes)
/* 326 */ public bool IsOpeningRange(DateTime eastern, MarketSession session, int minutes)
/* 329 */ public bool IsRthAfterInitialBalance(int ibMinutes)
/* 334 */ public bool IsRthAfterInitialBalance(DateTime eastern, int ibMinutes)
```

```
/* 340 */ public bool TrySessionBounds( DateTime eastern, MarketSession session, out DateTime open, out DateTime close)
/* 356 */ protected override DateTime tradingDayKey(DateTime barTime)
/* 359 */ protected override TimeSpan sessionClock(DateTime barTime)
/* 369 */ public sealed class SessionStats
/* 390 */ protected int IbMinutes { get; set; }
/* 392 */ protected TimeSpan m_rthValueEnd = DefaultRthValueEnd;
/* 393 */ public static readonly TimeSpan DefaultRthValueEnd = new TimeSpan(16, 0, 0);
/* 399 */ public SessionStats Session { get; }
/* 402 */ protected TimeSpan RthValueEndEt(DateTime globexDate)
/* 405 */ protected bool IsRthValueBarEt(DateTime et)
/* 467 */ public enum EOwnProfileWindow { MultiChartsProfile = 0, GlobexDay = 1, Rth = 2 }
/* 473 */ protected EOwnProfileWindow OwnProfileWindow { get; set; }
/* 475 */ protected override bool isNewProfileWindow(int stream, int barIndex, DateTime barTime, DateTime prevBarTime)
/* 484 */ protected override int todMinuteOfSession(DateTime barTime)
/* 491 */ protected override bool todHalfDay(DateTime sessionDate)
/* 498 */ protected override double todDelta(out int tier)
/* 512 */ protected override bool isRthBar(DateTime barTime)
/* 514 */ protected override bool includeBarInProfile(DateTime barTime)
/* 521 */ public VpExtra RthProfile()
/* 527 */ public VpExtra GlobexProfile()
/* 533 */ protected override List<VpExtra> migrationProfiles(int stream, int count)
/* 545 */ public VpExtra PriorRthProfile()
/* 561 */ protected override bool profileCrossesRoll(VpExtra e)
/* 575 */ public enum EFlowTickSource { DataLoader = 0, Iog = 1, None = 2 }
/* 585 */ protected bool OrderFlowEnabled { get; set; }
/* 588 */ protected _OrderFlow.FlowConfig FlowCfg { get; }
/* 589 */ protected EFlowTickSource FlowTickSource { get; set; }
/* 591 */ protected bool IogMagnifierOneTick { get; set; }
/* 603 */ protected string McBidAskChartTypeName { get; set; }
/* 609 */ protected _OrderFlow.FlowEngine Flow { get; }
/* 624 */ protected override void StopCalc()
/* 853 */ protected override _OrderFlow.FootprintBar footprintForLadder(int barNumber)
/* 858 */ protected override bool ladderHasFlow { get; }
/* 860 */ protected override bool sessionOpenAt(DateTime slotStart)
/* 863 */ public _OrderFlow.FootprintBar Footprint(int barsAgo)
/* 871 */ public _OrderFlow.FlowValue BarDelta(int barsAgo)
/* 879 */ public _OrderFlow.FlowValue CumDelta(int barsAgo)
/* 887 */ public IList<_OrderFlow.Imbalance> DiagonalImbalances(int barsAgo, double ratio = 3.0, double minVol = 10)
/* 895 */ public _OrderFlow.StackedResult StackedImbalances(int barsAgo, double ratio = 3.0, double minVol = 10, int minStack = 3)
/* 903 */ public _OrderFlow.FlowValue Absorption(int barsAgo, _OrderFlow.Side side, int edgeTicks = 2, double volMult = 2.0, int holdTicks = 1)
/* 919 */ protected bool UseCmeHolidayTable { get; set; }
/* 927 */ protected virtual bool IsGlobexClosedDay(DateTime globexDate)
/* 936 */ protected virtual bool IsNoRthDay(DateTime globexDate)
/* 946 */ protected virtual TimeSpan? HolidayHaltEt(DateTime globexDate)
/* 955 */ protected virtual TimeSpan? RthEarlyCloseEt(DateTime globexDate)
/* 961 */ protected override bool isTradingHoliday(DateTime date)
```

_VolumeProfileFuturesBase.SessionStats

```
/* 371 */ public DateTime globexDate;
/* 372 */ public double globexHigh, globexLow, globexVolume;
/* 373 */ public double ethHigh, ethLow, ethVolume;
/* 374 */ public double rthOpen, rthHigh, rthLow, rthClose, rthVolume;
/* 375 */ public double ibHigh, ibLow;
/* 376 */ public bool ibComplete;
/* 377 */ public bool rthComplete;
/* 378 */ public double priorRthHigh, priorRthLow, priorRthClose;
/* 379 */ public double priorGlobexHigh, priorGlobexLow;
/* 380 */ public double IbRange { get; }
/* 382 */ public double IbExtensionUp { get; }
/* 384 */ public double IbExtensionDown { get; }
/* 386 */ public double Gap { get; }
```

`_WeeklyProfiles.Function.CS`

(file scope)

```
/* 19 */ public enum EWeeklySource { RthSessions = 0, GlobexSessions = 1 }
/* 20 */ public enum EWeeklyRollPolicy { Reset = 0, CashTermsReserved = 1, BackAdjustedReserved = 2 }
/* 22 */ public sealed class WeeklyProfileConfig
/* 47 */ public sealed class WeekSession
/* 52 */ public sealed class WeekProfile
/* 103 */ public sealed class _WeeklyProfiles
```

WeeklyProfileConfig

```
/* 24 */ public bool enabled = false;
/* 25 */ public int dataStream = 1;
/* 26 */ public EWeeklySource source = EWeeklySource.RthSessions;
/* 27 */ public int keepWeeks = 8;
/* 28 */ public int minSessions = 3;
/* 29 */ public bool includeDeveloping = false;
/* 30 */ public EWeeklyRollPolicy rollPolicy = EWeeklyRollPolicy.Reset;
/* 31 */ public int distribution = 2;
/* 33 */ public string Validate()
/* 43 */ public string Canon()
```

WeekSession

```
/* 49 */ public DateTime day; public bool roll, partial, isRollWindow; public double volume; public int bars;
```

WeekProfile

```
/* 54 */ public DateTime key;
/* 55 */ public double tick;
/* 56 */ public readonly SortedDictionary<long, double[]> rows = new SortedDictionary<long, double[]>();
/* 57 */ public readonly List<WeekSession> sessions = new List<WeekSession>();
/* 58 */ public bool isPartial, crossesRoll, complete, bidAskValid = true;
/* 59 */ public double high = double.MinValue, low = double.MaxValue, open = double.NaN, close = double.NaN;
/* 60 */ public int Sessions { get; }
/* 61 */ public bool IsRollWindow { get; }
/* 64 */ public bool Dense(out double[] prices, out double[] total, out double[] ask, out double[] bid)
/* 83 */ internal void merge(WeekProfile o)
/* 93 */ internal void addRow(long k, double total, double ask, double bid)
/* 100 */ public string Tag { get; }
```

_WeeklyProfiles

```
/* 105 */ public readonly WeeklyProfileConfig config;
/* 106 */ public readonly double tick;
/* 107 */ public Action<string> journal;
/* 113 */ public _WeeklyProfiles(WeeklyProfileConfig cfg, double tick)
/* 116 */ public static DateTime WeekKeyOf(DateTime tradingDay)
/* 118 */ public int CompletedWeeks { get; }
/* 124 */ public void AddSession(DateTime day, IDictionary<long, double[]> rows, bool roll, bool partial, dou
ble open, double high, double low, double close, int bars, bool isRollWindow = false)
/* 158 */ public void RefreshRollWindow(Func<DateTime, bool> isRollWindow)
/* 166 */ public void Flush()
/* 182 */ public WeekProfile Week(int weeksAgo)
/* 190 */ public DateTime WeekKey(int weeksAgo)
/* 194 */ public WeekProfile Composite(int n)
```

`_XmktLeg.Function.CS`

(file scope)

```
/* 14 */ public sealed class XmktLegConfig
/* 36 */ internal sealed class _XmktLeg
```

XmktLegConfig

```
/* 16 */ public string role = "leg";
/* 17 */ public int dataStream = 2;
```

```
/* 18 */ public int maxStaleSec = 0;
/* 19 */ public int returnBars = 1;
/* 20 */ public int corrLength = 60;
/* 21 */ public int rsLength = 60;
/* 22 */ public bool leaderSwing = false;
/* 24 */ public string Validate()
```

_XmktLeg

```
/* 38 */ public XmktLegConfig config = new XmktLegConfig();
/* 39 */ public TimeSpan maxStale = TimeSpan.Zero;
/* 40 */ public Action<string> journal;
/* 50 */ public bool Configured { get; private set; }
/* 52 */ public bool Available { get; }
/* 53 */ public DateTime LastLegTime { get; }
/* 54 */ public double LastClose { get; }
/* 56 */ public string Configure()
/* 66 */ public void Reset()
/* 83 */ public void Step(DateTime d1Time, double d1Close, int legBar, DateTime legTime, double legClose)
/* 110 */ public void OnRoll()
/* 120 */ public double LogReturn { get; }
/* 122 */ public double Data1LogReturn { get; }
/* 124 */ public double Correlation { get; }
/* 126 */ public double Beta { get; }
/* 128 */ public double RsZ { get; }
```

F. Open VERIFY items and discrepancies

Still VERIFY (could not be settled from the code; most need MultiCharts itself):

1. **TraceLevel override.** Whether the Format Signal dialog value of `TraceLevel` overrides a value set in code in `Create()`.
2. **`_Test6` trading day.** `_Test6` (and example 6) do not override `tradingDayKey`. On a Globex template their M-DAILY days are calendar days, not Globex days. Decide if that is intended.
3. **EventBlackout timing.** `EventBlackout` is computed on the bar close for `[open, close)`. Whether an order sent on that bar (filled next bar) should also check the next bar's window.
4. **M-TICK reflection.** The DataLoader member names that `_TickStream` reaches by reflection (they only resolve inside MC).
5. **HTF bar-end stamping (MC Probe P2).** Examples 5, 9 and 10 assume HTF and 1m bars are stamped at their end. Run Probe P2 first, as the catalogue says.
6. **G0-a cost model.** The A2-40 cost model used by Gate 0 (including the 2-tick slippage for 09:30–09:35 fills) is VERIFY against the catalogue's A2-40/A2-59 text (§2.5).
7. **Example 10 zone break.** After the touch and before the 60m trigger, one RTH 1m close below the pullback zone kills the week's setup. That is a very strict coding choice; the catalogue is silent.
8. **MC API stubs.** The examples compile against stubs I wrote for the MC API. A member MC names differently only shows in the PowerLanguage editor.

Discrepancies found (code vs catalogue vs v1 manual):

1. **M-DAILY at rolls.** v1 said "roll gaps are removed" from the ATR. On `main`, a roll reseeds the ATR: `Atr20d` is NaN for `atrLength` sessions after each roll (only when RollDetection is on). The catalogue's A2-63 line "Atr20d, roll-gap removed" is also inaccurate: with RollDetection Off the gap enters the ATR; with it on, there is no ATR for 20 sessions.
2. **PS-13's 200-day ATR.** The catalogue's M-DAILY `atrLength = 200` never becomes valid with RollDetection on, because @ES rolls about every 63 sessions. Example 9 computes its own ATR200d instead.
3. **DV-03 follow-through** (≥ 1 box width before the retest) is not coded in example 9 (the PS-13 version).
4. **A2-29 gating.** The G0-d table lists the A2-25…A2-32 range as G0-d-gated, but A2-29's own entry says "Gate (v2): none (classifier)".
5. **`_Test1` vs `_TestGate0`.** The catalogue (§3.0: "A2-33 and A2-39 run in `_Test1`"; also the Batch-1 slot table) names `_Test1` as the host, and so does the `LevelStudyConfig` comment in `_CustomSignal`

("e.g. `_Test1`"). `_LevelStudy` itself doesn't name a host. `_Test1` is a plain `SignalObject`; Gate 0 is hosted by `_TestGate0`.

6. **KD-02 canary.** The catalogue's G0-b canary list includes KD-02; `EGate0Canary` does not have it.
7. **Cost per round trip (G0-a conflict).** The `_TestGate0` header's MC settings are $2.25 commission and 1 tick slippage, each per side, so an ES round trip costs $4.50 + $25.00 = **$29.50**. The catalogue's cost floor (§2.1) assumes **$16.98** (IBKR round trip + 1 tick in total), and v2 of this manual wrongly summed the header to $17.00. At $29.50 the 14-tick floor keeps costs at about 0.17 R, not ≤ 0.10 R. To match the catalogue instead, use 0.5 tick per side ($6.25), which gives $17.00. Not changed in code; decide which one G0-a means.
8. **Example file names.** The example files keep the numbers they were written under (`_Ex06`… `_Ex10` do not match the reading order); each §3 heading now gives its file name.
9. **Cost floor: max or skip.** Catalogue §2.1 says both "every idea's stop is max(structural stop, cost floor)" and "trades whose structural stop sits under the floor are logged as skipped rather than resized". The examples follow the second (skip and journal, 14 ticks, R from the planned entry). The first would widen the stop instead.
10. **AV-08 at rolls (example 3).** The catalogue says multi-day anchors reset at rolls; the Week/Month VWAP slots don't (`_VpVwapEngine` resets them only at a new week or month). Example 3 turns the layer off until the next clean period (`wtdClean`/`mtdClean`). Re-anchoring at `rollOn(1)` is the alternative.
11. **A2-57 at rolls (example 5).** The catalogue is silent on the 60m EMA at a roll. Example 5 re-seeds it at `rollOn(1)` (3 × 13 closed 60m bars of warm-up); ignoring turns for that long without re-seeding is the alternative.
12. **TS-01 market (example 2).** The catalogue tests MES on 30m/60m bars (kill $2.47). Example 2 is written for @ES on 1m.
13. **Example 10 zone break.** See VERIFY item 7: one 1m close below the zone after the touch cancels the week, which is stricter than the catalogue's text.

G. EV-4.6b — event-table coverage (PRs #28/#29)

What changed. Before EV-4.6b, any coverage gap in the event table disabled the whole engine. Now:

- The engine is **disabled only** when the compiled table is missing or its hash differs from `EventRules.ExpectedTableSha` (`"228cdee716a6eafa"` on 629da0d). It writes an Error line ending `; engine disabled`, does not throw, and every query answers neutrally (no events, no blackout).
- Known holes in the table (`EventRules.CoverageHoles()`, part of the hash) become a third state per kind and day: `EEventCoverage { NoEvent = 0, Event = 1, Unknown = 2 }`, read with `Coverage(kind, day)`.

Using it.

```
// in CalcBar, before acting on an event filter
_EventCalendar ev = EventCalendar;
if (ev == null || ev.Disabled) { /* fail closed: a disabled engine says "no event" */ return; }
EEventKind hole;
if (ev.CoverageBlocks(day, EEventClassMask.Macro, EEventTier.T1, out hole)) { /* unknown coverage: skip the day */ return; }
```

- `CoverageBlocks(day, mask, minTier, out kind)` returns true if some kind in `mask` is Unknown on that day and its (effective) tier passes `minTier`. It returns false when the engine is disabled or `EventConfig.allowPartialCoverage` is true. `CoverageBlocks(day, mask)` uses `EEventTier.None`.
- **PR #29:** an Unknown kind whose tier is below the query's `minTier` never blocks, and the tier test uses the **effective** tier (after `EventConfig.tierOverride`).
- `CoverageReport(from, to, mask[, minTier], out flagged)` reports, per kind, the share of Unknown-blocked trading days in `[from, to]`. `CoverageStats()` returns one `COVSHARE run {Kind} blocked {n}/{days} {p}%` line per kind, with `FLAG` above 5%.

Journal. Once at start: `COVGAP {Kind} {from}..{to} Unknown …`. Then, per affected trading day: `COVGAP day {yyyy-MM-dd} Unknown Kind(blocks|belowMinTier|NoEvent)`.

Where the manual uses it. Example 2 (TS-01) skips Unknown T1 macro days and fails closed when the engine is disabled. Example 4 (A2-50) tags Unknown days in its journal instead of calling them report or non-report days. The catalogue's TS-01 caveat about GDP/PCE holes in 2016–24 is the case this handles.